Parallel Processing and Distributed Systems

Introduction

Mahboob Qaosar
Lecturer

Dept. of CSE
R.U.

Reference Books

- Computer Architecture and Parallel Processing
  - Kai Hwang
  - Faye A. Briggs
- Advanced Computer Architecture (Parallelism, Scalability, Programmability)
  - Kai Hwang
- Database System Concepts (Chapter 22)
  - Silberschatz, Korth and Sudarshan
- Hand Notes…

Parallel Processing

Parallel processing is an efficient form of information processing which emphasize the exploitation of concurrent events in the computing process.

Concurrency implies parallelism, simultaneity and pipelining.

Distributed System

A distributed system is one in which components are located at networked computers, communicate and coordinate their actions only by passing messages

Use of computer from application point of view

1. Data Processing
2. Information Processing
3. Knowledge Processing
4. Intelligence Processing

From OS point of view, Computer System have improved chronologically in four phases

1. Batch processing
2. Multiprogramming
3. Time Sharing [2 & 3 Software approach]
4. Multiprocessing

Level of Parallel Processing

1. Job or program level
2. Task or procedure level
3. Inter-instruction level
4. Intra-instruction level

Parallelism in Uniprocessor Systems

- Multiplicity of functional unit
- Parallelism and pipelining within the CPU
- Overlapped CPU and I/O operation
- Use of a hierarchical memory system
- Balancing of subsystem bandwidths
- Multiprogramming and time sharing

Multiplicity of functional unit

Many of the functions of the ALU can be distributed to multiple and specialized functional unit which can operate in parallel

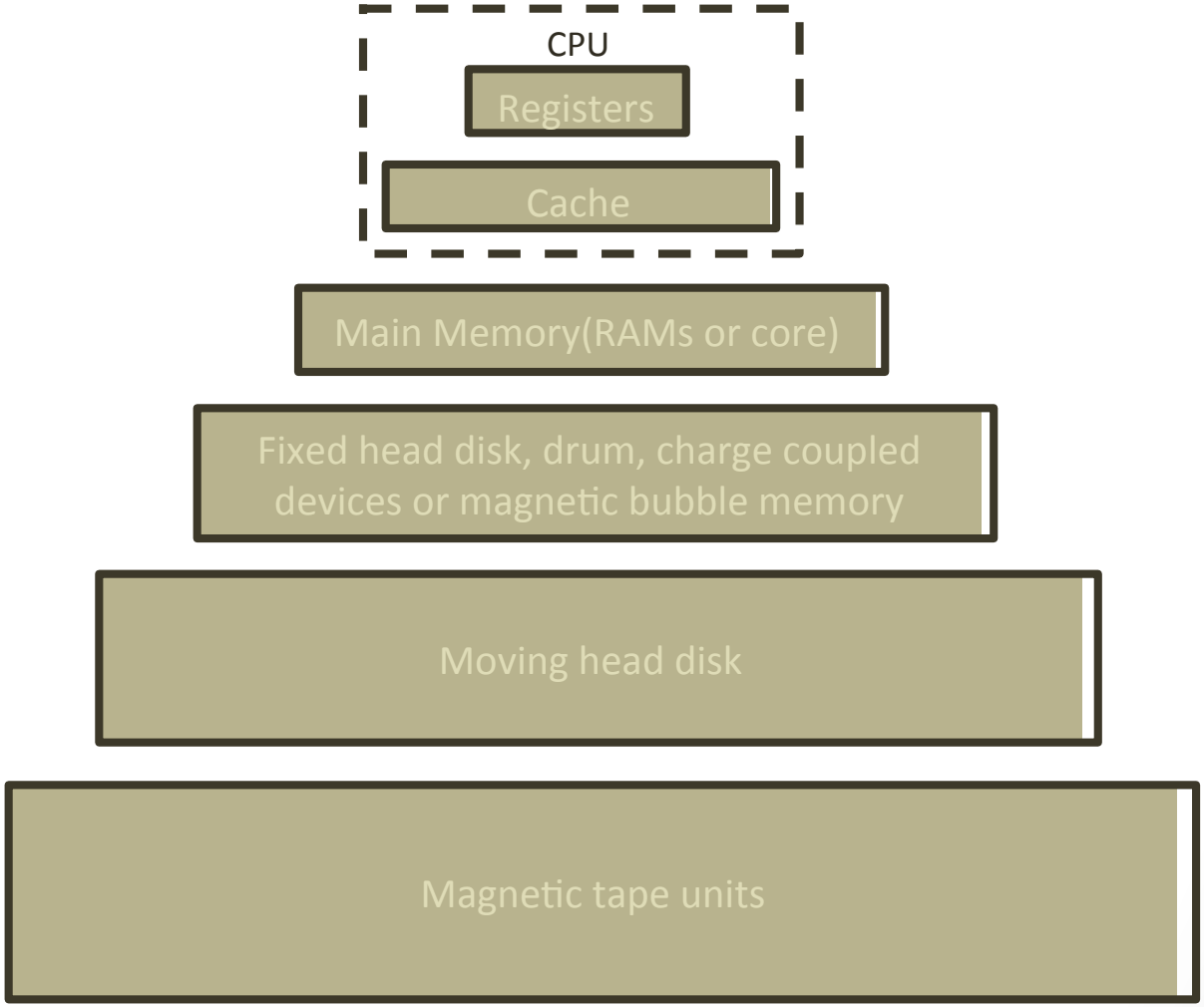
Parallelism and pipelining within the CPU

- Parallel adder such as carry-look ahead and carry-save adder can be utilized
- High speed multiplier and convergence division techniques can exploit parallelism

Overlapped CPU and I/O operations

DMA channel can be used to provide direct information transfer between the I/O devices and the main memory

Use of hierarchical memory system



Balancing of Subsystem Bandwidth

- $t_d > t_m > t_p$ where
  - t_d = I/O device access time
  - t_m = Memory cycle time
  - t_p = Processor cycle time
- $B_m = W / t_m$, B_m = *Memory bandwidth*
- $B^u_m = B_m / sqrt(M)$
  - B^u_m = *Utilized memory bandwidth*
  - *M = No. of memory module*

- $B^u_p = R_w / T_p$
  - B^u_p = Utilized processor bandwidth
  - R_w = No. of result word
  - T_p = Total time required to produce R_w
- $B_m \geq B^u_m \geq B_p \geq B^u_p > B_d$
- Bandwidth balancing between CPU and I/O device
  - Using fast cache memory between CPU and main memory
- Bandwidth balancing between memory and I/O device
  - I/O channels with different speeds can be used between the slow I/O device and main memory
  - We wish to achieve a balanced system, where
    - $B^u_p + B_d = B^u_m$

Multiprogramming and Time Sharing

Software approach

Parallel Computer Structure

Pipeline Computers

Array Computers

Multiprocessor Systems

Architectural Classification Schemes

- Flynn’s classification based on multiplicity of instruction streams and data streams in a computer system
- Feng’s scheme(1972) based on serial versus parallel processing
- Handler’s classification(1977) determined by the degree of parallelism and pipelining in various subsystem levels

Multiplicity of Instruction-Data Streams

Single instruction streams – single data streams(SISD)

Single instruction streams – multiple data streams(SIMD)

parallel

Multiple instruction streams – single data streams(MISD)

Multiple instruction streams – multiple data streams(MIMD)

Serial versus Parallel Processing

- Word length = *n*
- Bit-slice length = *m*
- Maximum Parallelism Degree, P(C) = n * m

Serial versus Parallel Processing

- Word-serial and bit-serial (WSBS) (n=1, m=1)
  - MINIMA(1, 1)
- Word-parallel and bit-serial (WPBS) (n=1, m>1)
  - STARAN(1, 256)
  - MPP(1, 16384)
  - DAP(1, 4096)
- Word-serial and bit-parallel (WSBP) (n>1, m=1)
  - IBM 370/168 UP(64, 1)
  - CDC6600(60, 1)
  - Burrough 7700(48, 1)
- Word-parallel and bit-parallel (WPBP) (n>1, m>1)
  - Illiac IV(64, 64)
  - TI-ASC(64, 32)

Parallel Processing

Hardware Technology

Advanced Processor Technology

Major Processor Families

- **CISC Processor**
- **RISC Processor**
- **Super-Scalar Processor**
  - **A special subclass of RISC Processor**
  - **Allow multiple instructions to be issued simultaneously during each cycle**

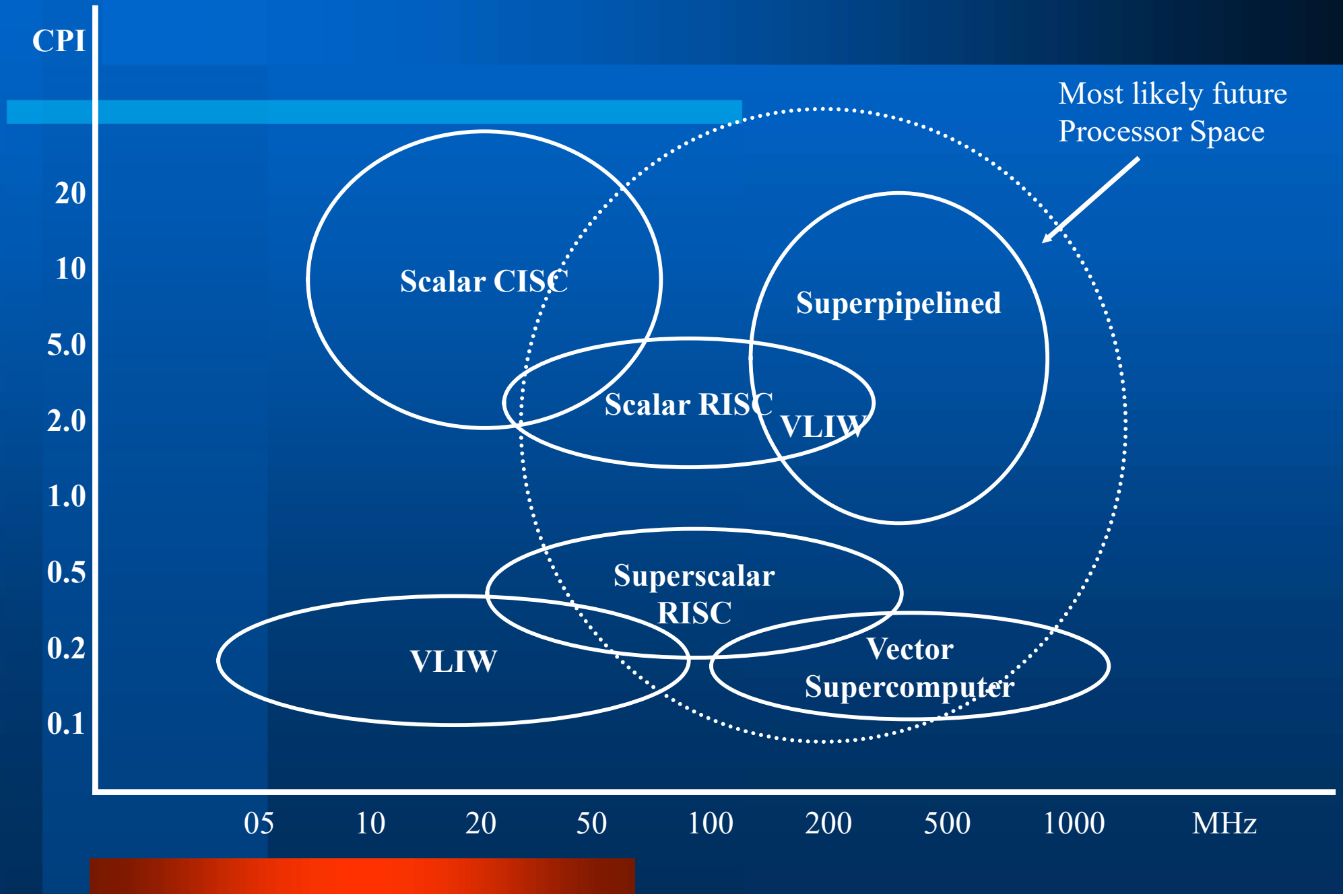
Advanced Processor Technology

- **VLIW Processor**
  - **Has more functional unit than that of a superscalar processor**
- **Super-Pipelined Processor**
  - **Uses multiphase clock**
- **Vector Processor**
  - **Uses multiple functional units for concurrent scalar and vector operations**
- **Symbolic Processor**

Advanced Processor Technology

- **Design Space of Processors**
  - Design space
  - Instruction Pipelines
  - Processors and Coprocessors
- **Instruction-Set Architectures**
  - Complex Instruction Set
  - Reduced Instruction Set
  - Architectural Distinction

The Design Space

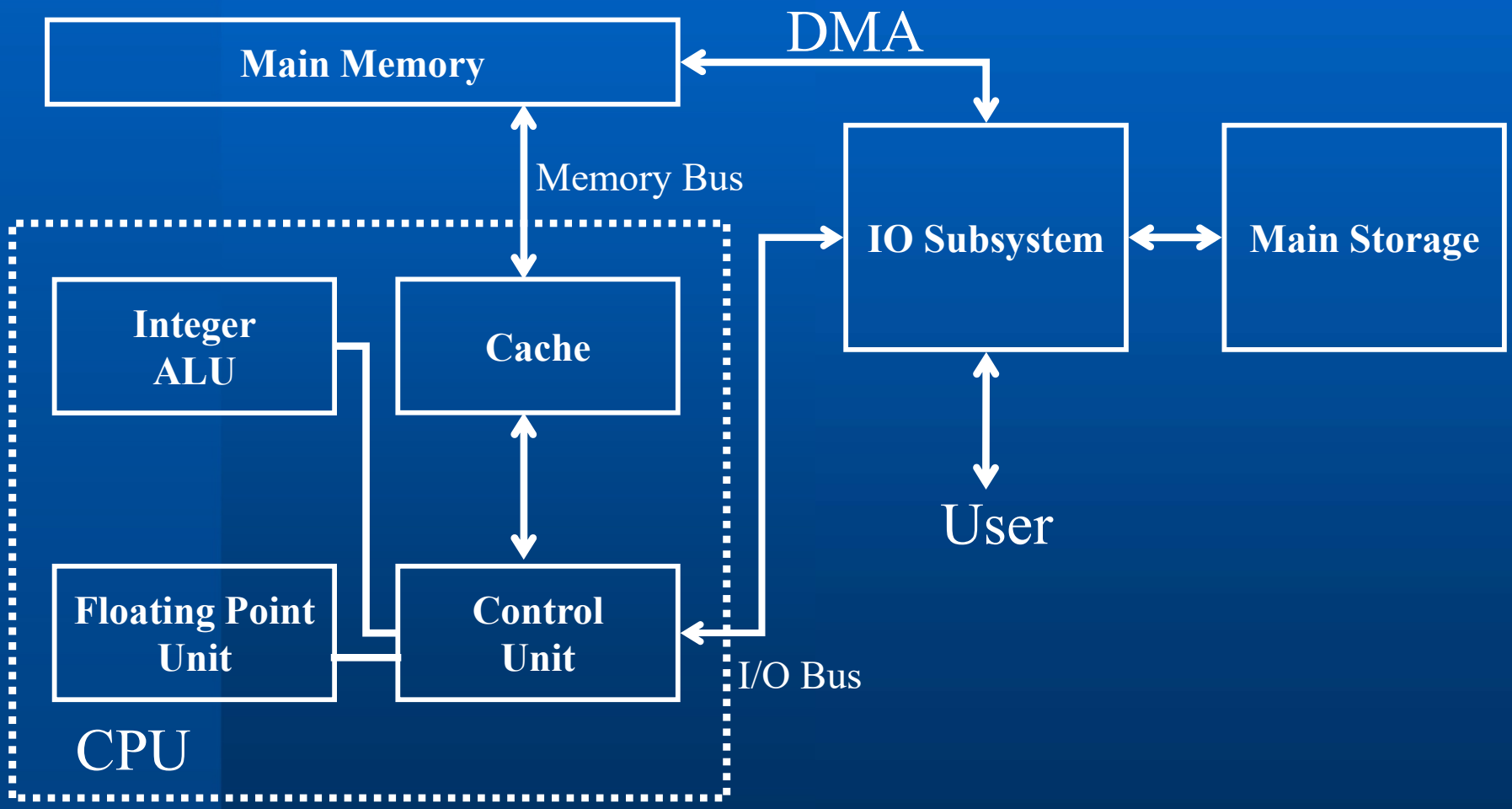


Instruction Pipeline

Some definitions

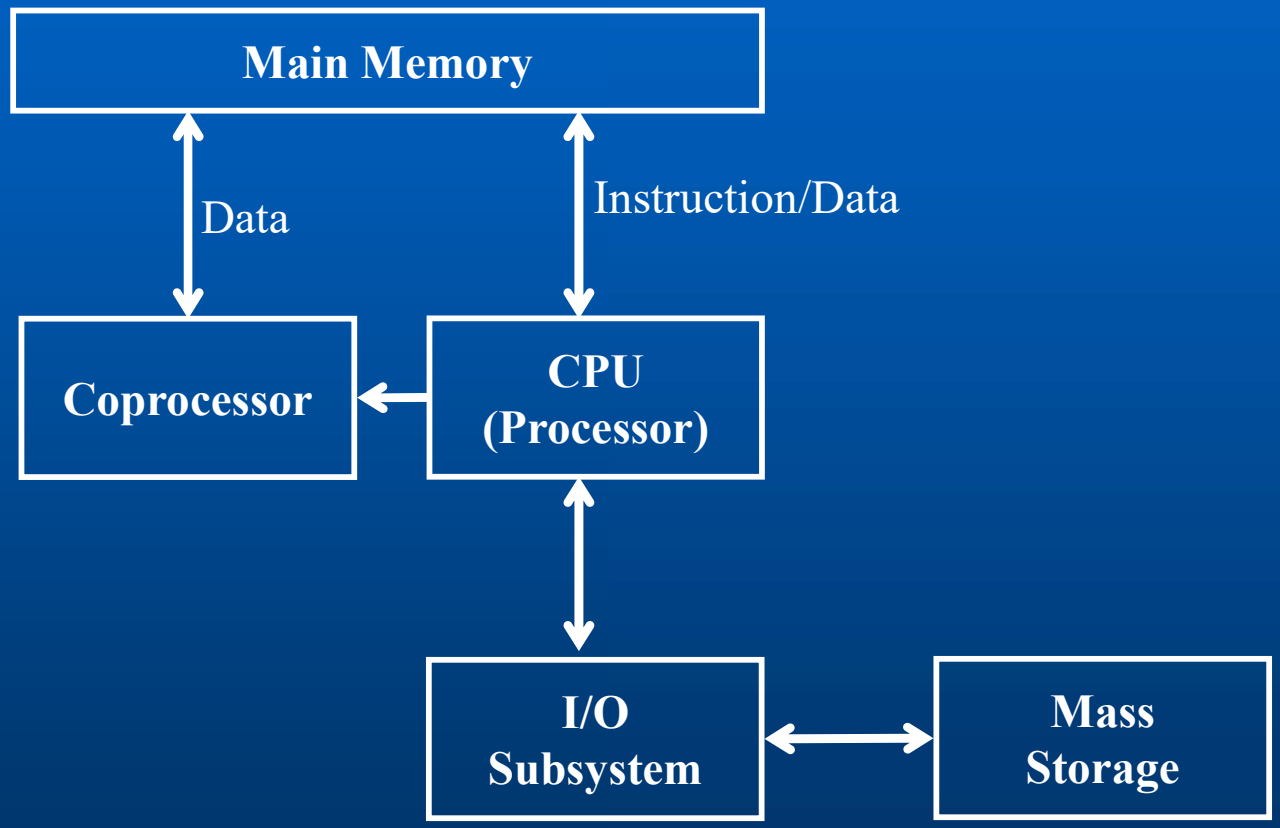
- **Instruction pipeline cycle**
  - **Clock period of the instruction pipeline**
- **Instruction issue latency**
- **Instruction issue rate**
- **Simple operation latency**
- **Resource conflict**

Processors and Coprocessors



CPU with built-in floating point unit

Processor and Coprocessor



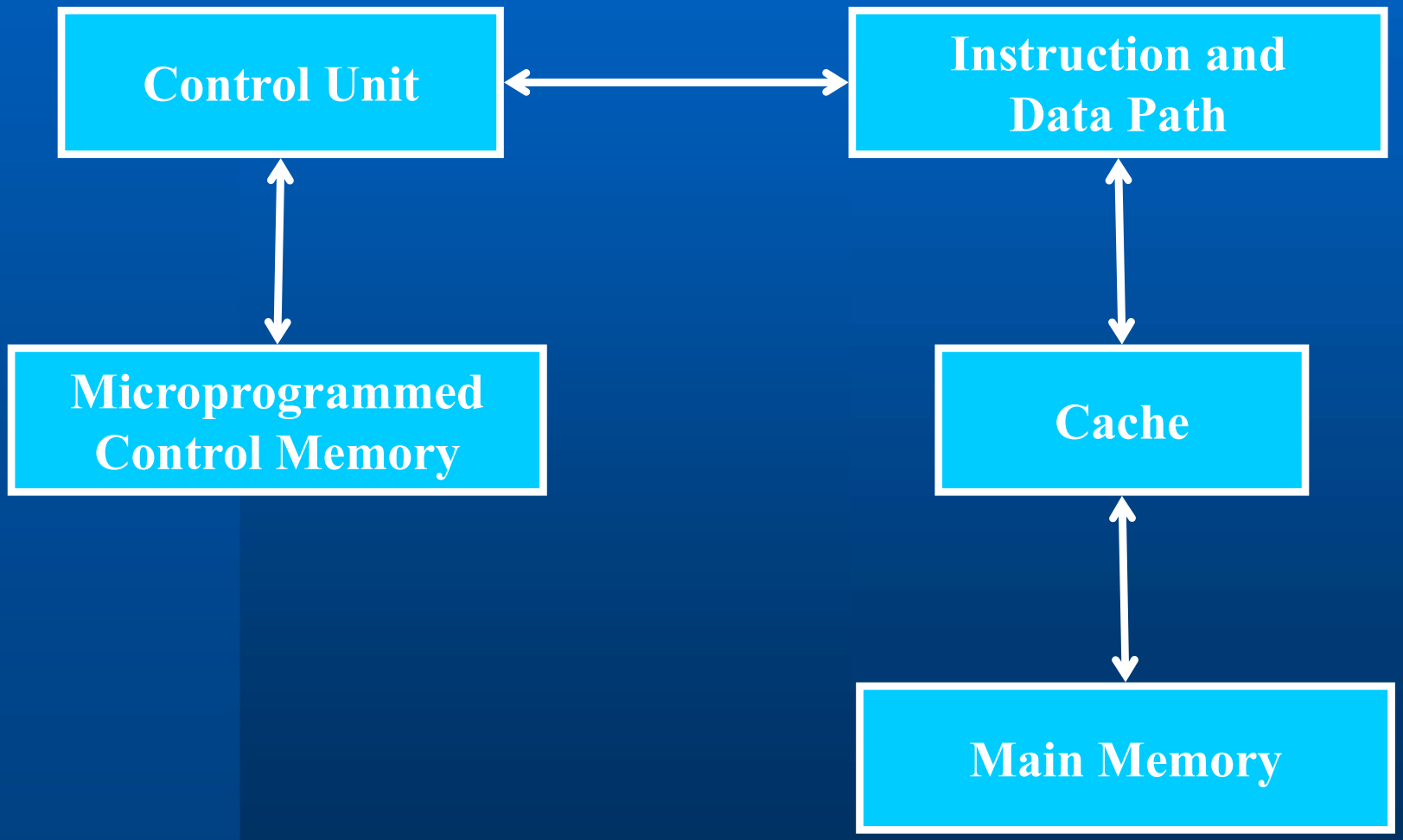
CPU with attached coprocessor

Instruction-Set Architectures

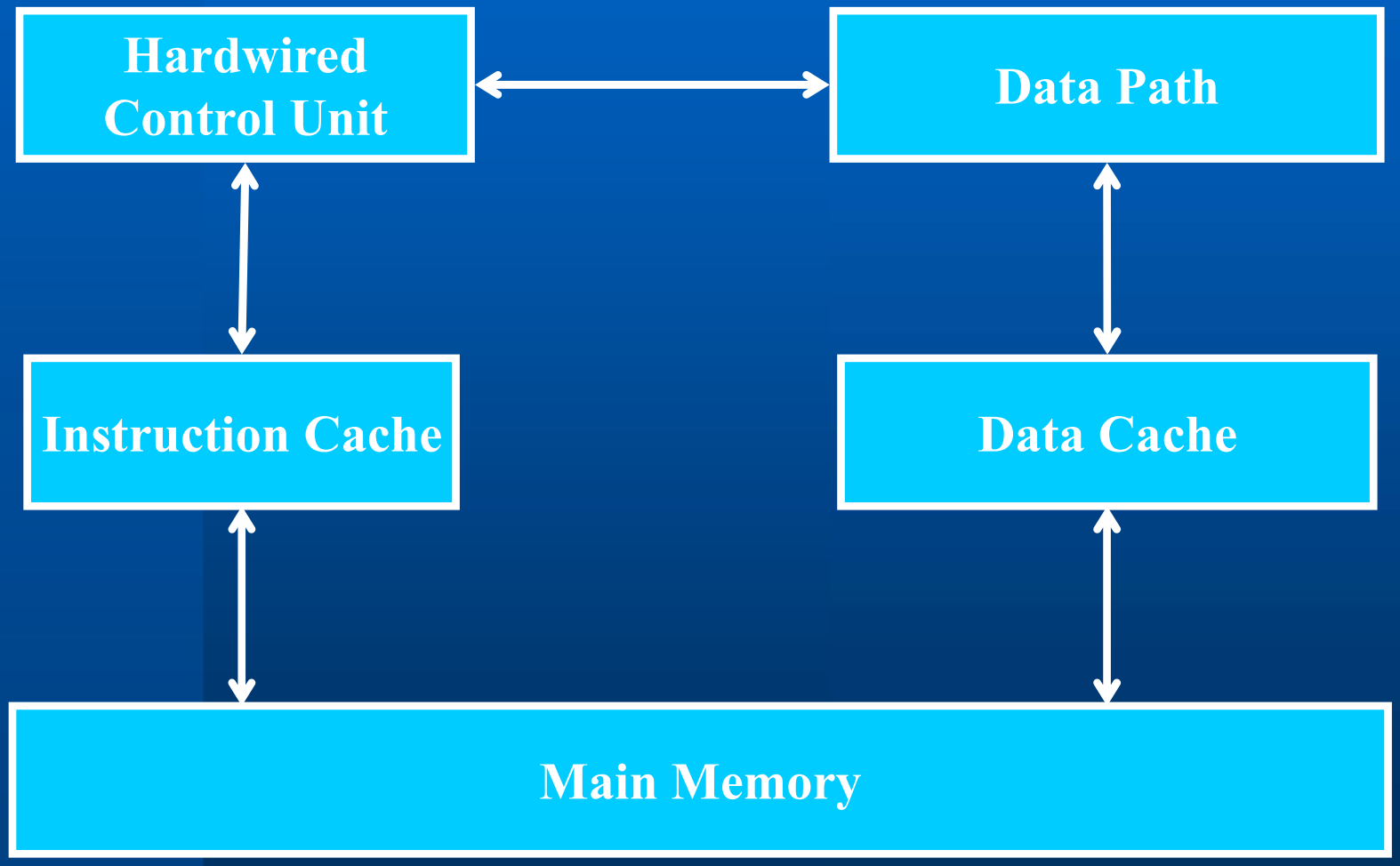
Specifies the primitive commands or machine instructions

- **Complex Instruction Sets**
  - **Most functions have been built into the hardware.**
  - **Has large instruction set.**
- **Reduced Instruction Sets**
  - **Has small instruction set.**

Typical CISC Processor Architecture



Typical RISC Processor Architecture

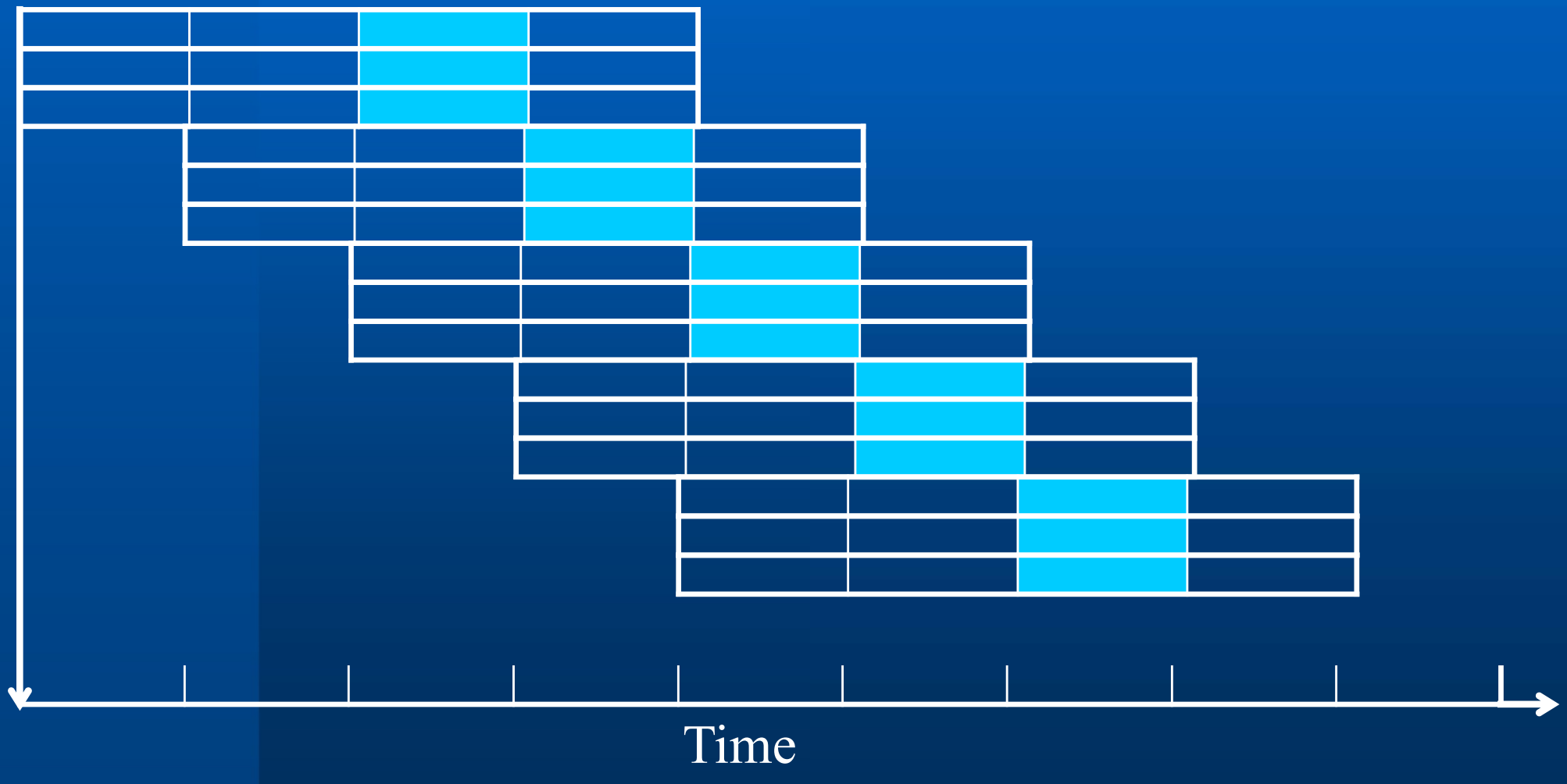


Differences between CISC & RISC architecture

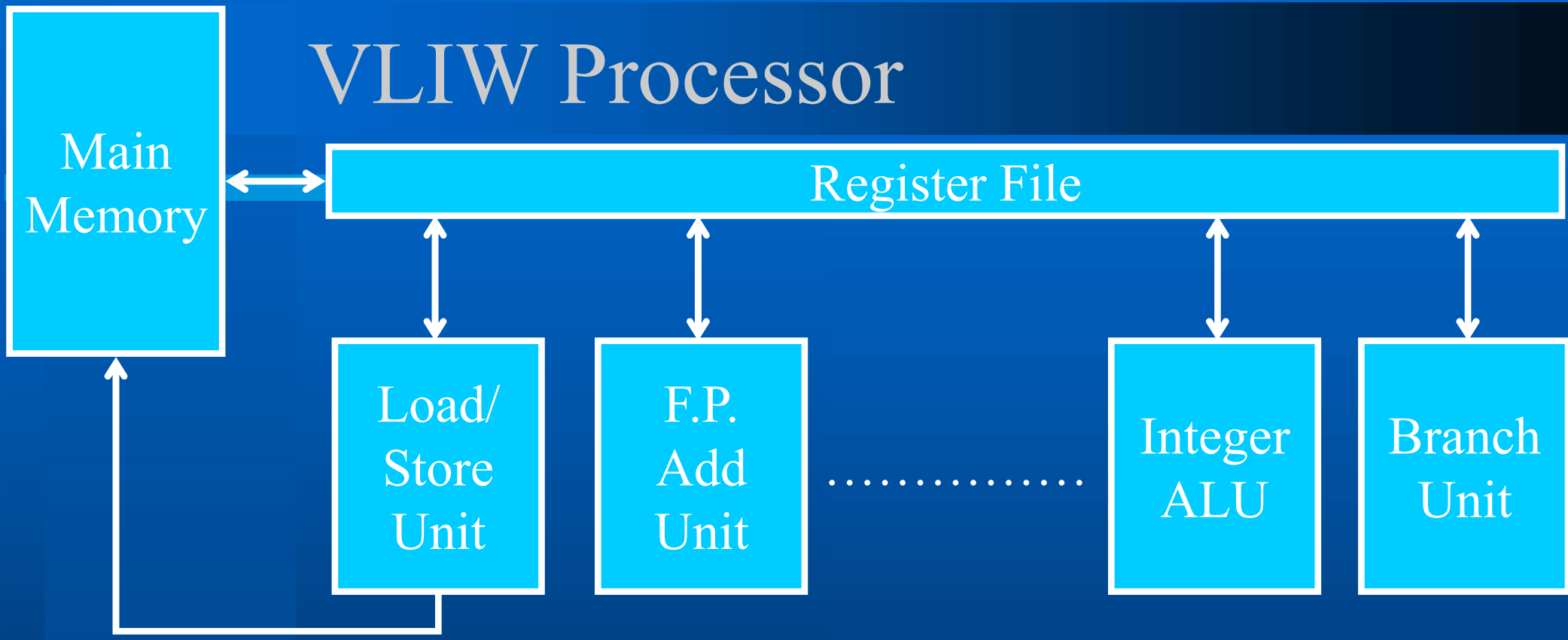
Characteristic	CISC	RISC
Instruction set size & instruction format	Large Set of Instruction with variable formats	Small set of instructions with fixed format
Addressing modes	12-24	Limited to 3-5
General purpose registers and cache design	8-24 GPRs, mostly with a unified cache for instructions and data, recent designs also use split caches	Large numbers (32-192) of GPRs with mostly split data cache and instruction cache
Clock rate and CPI	33-50 MHz in 1992 with a CPI between 2 and 15	50-150 MHz in 1993 with one cycle for almost all instructions and an average CPI < 1.5
CPU Control	Most microcoded using control memory (ROM), but modern CISC also uses hardwired control	Most hardwired without control memory

Superscalar Processor

- **Designed to exploit more instruction level parallelism in user programs.**



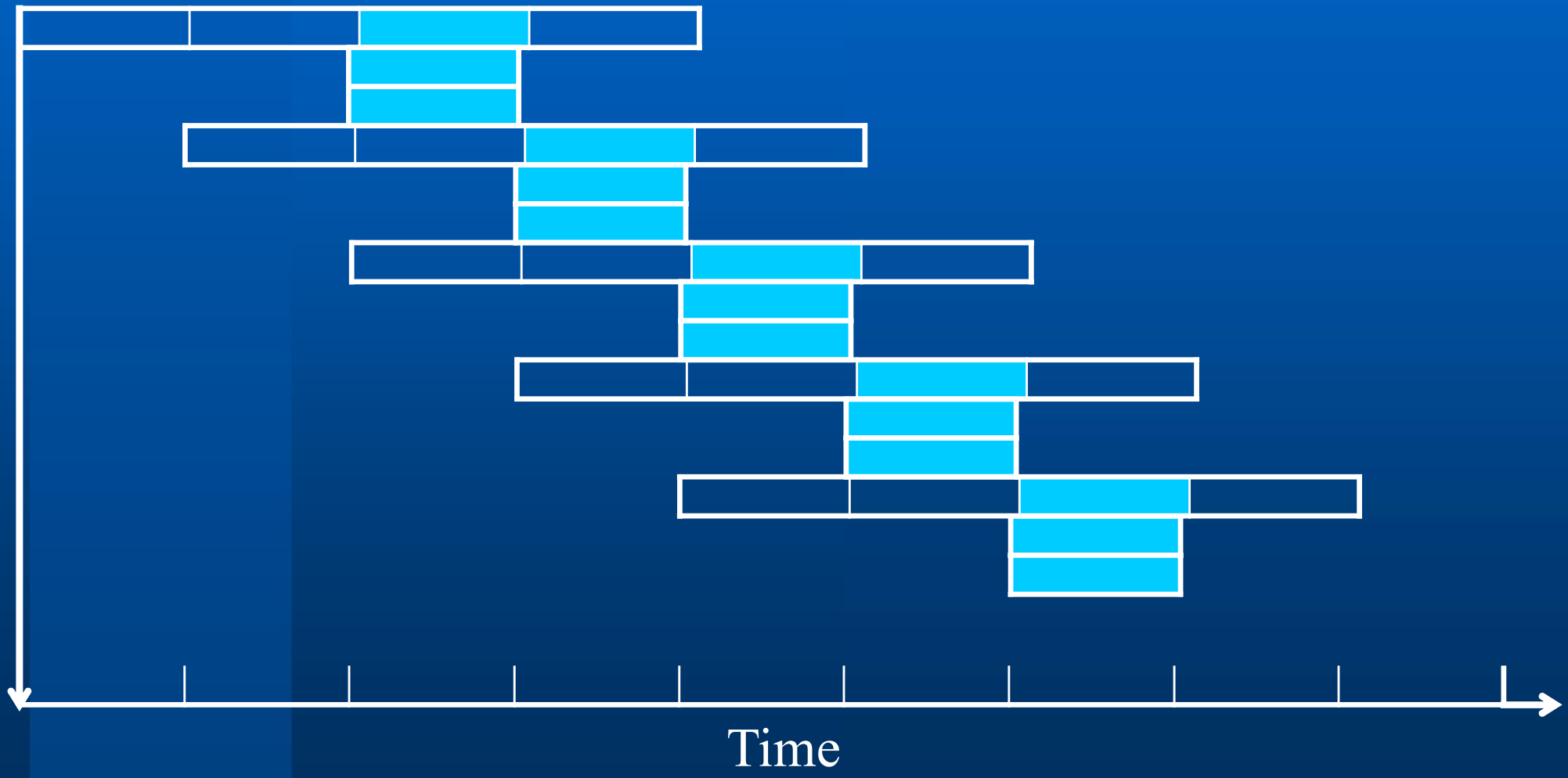
VLIW Processor



Load/Store	FP Add	FP Multiply	Branch	.....	Integer ALU

Typical VLIW processor & instruction format

VLIW Execution



Vector Processor

- **~ is a coprocessor specially designed to perform vector computation**

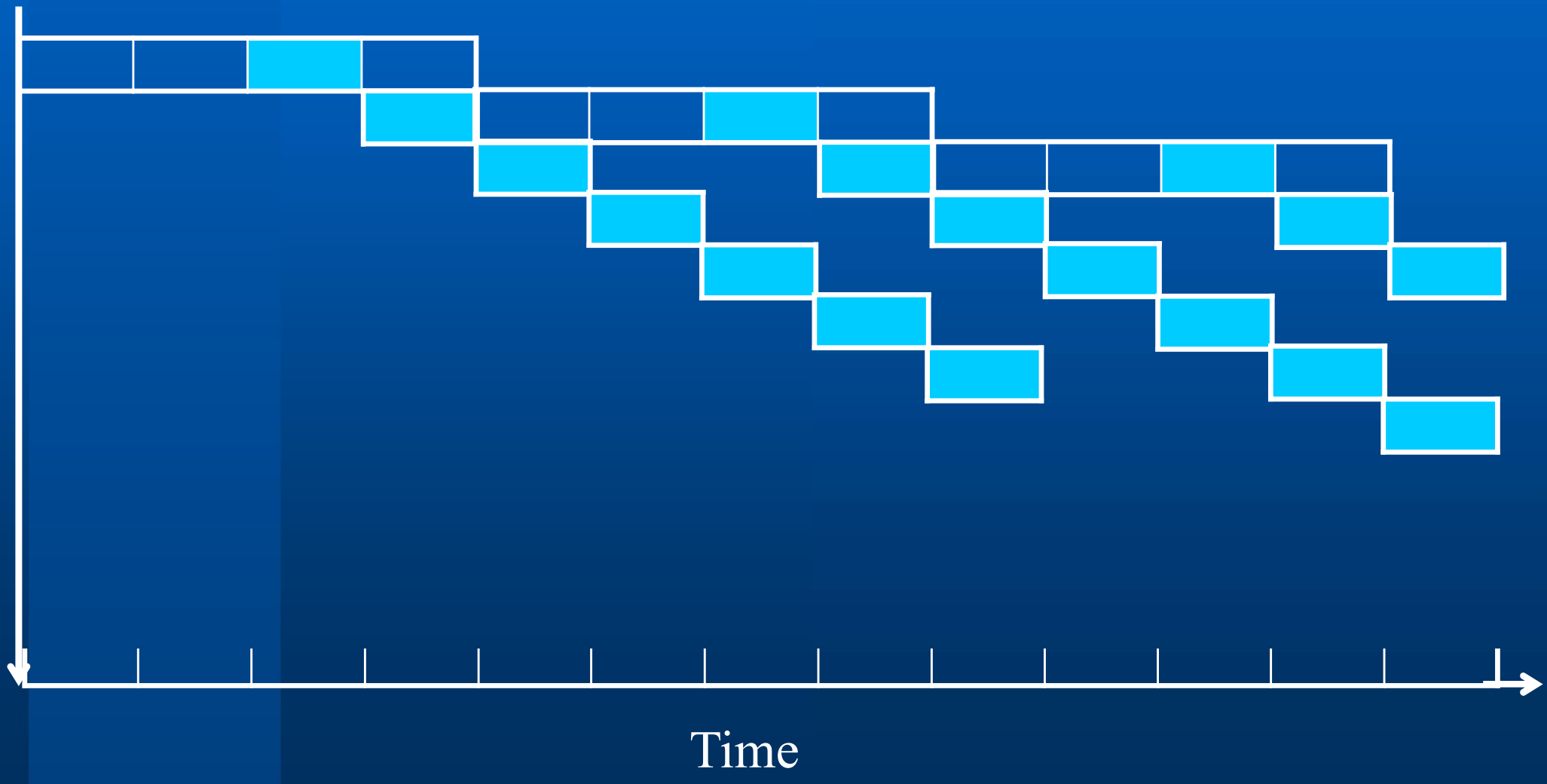
Vector Instructions

- V_1 o $V_2 \rightarrow V_3$ (binary vector)
- s_1 o $V_2 \rightarrow V_3$ (scaling)
- V_1 o $V_2 \rightarrow s_1$ (binary reduction)
- M(1:n) $\rightarrow V_1$ (vector load)
- $V_1 \rightarrow$ M(1:n) (vector store)
- o $V_1 \rightarrow V_2$ (unary vector)
- o $V_1 \rightarrow s_1$ (unary reduction)

Memory based vector operation

- $M_1(1{:}n)$ o $M_2(1{:}n) \rightarrow M_3(1{:}n)$
- s_1 o $M_1(1{:}n) \rightarrow M_2(1{:}n)$
- o $M_1(1{:}n) \rightarrow M_2(1{:}n)$
- $M(1{:}n) \rightarrow M(k)$

Vector Pipeline Execution

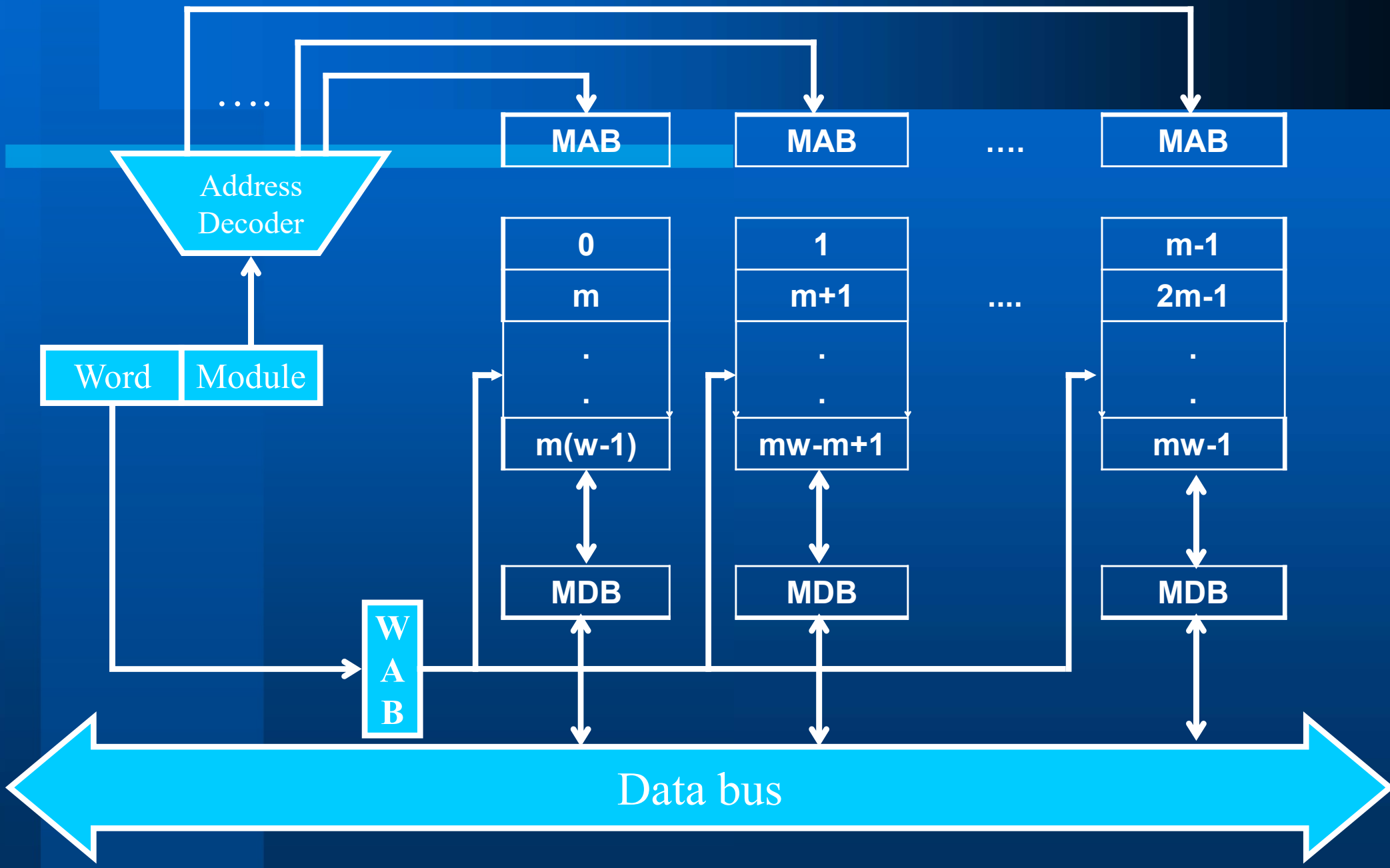


- CISC Scalar Processor
  - Figure 4.5: VAX 8600 CPU
- RISC Scalar Processor
  - Figure 4.9: i860 Processor
- Figure 4.12: A typical superscalar RISC processor
- Figure 4.13: IBM RS/6000 processor

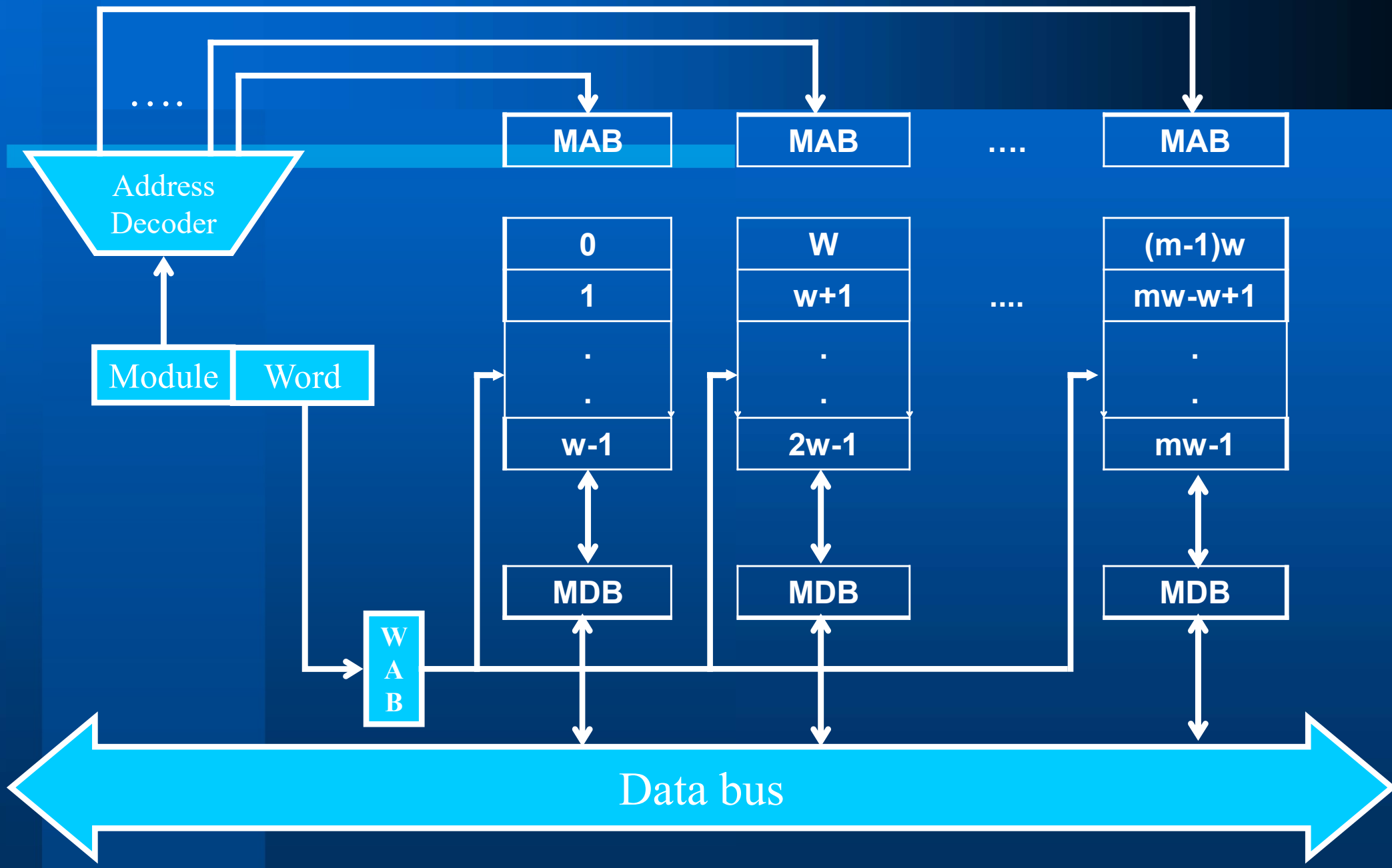
Interleaved Memory Organization

- **Allows pipelined access of the parallel memory modules**
- **Provides a higher bandwidth for pipelined access of contiguous memory location.**

Low-order interleaving



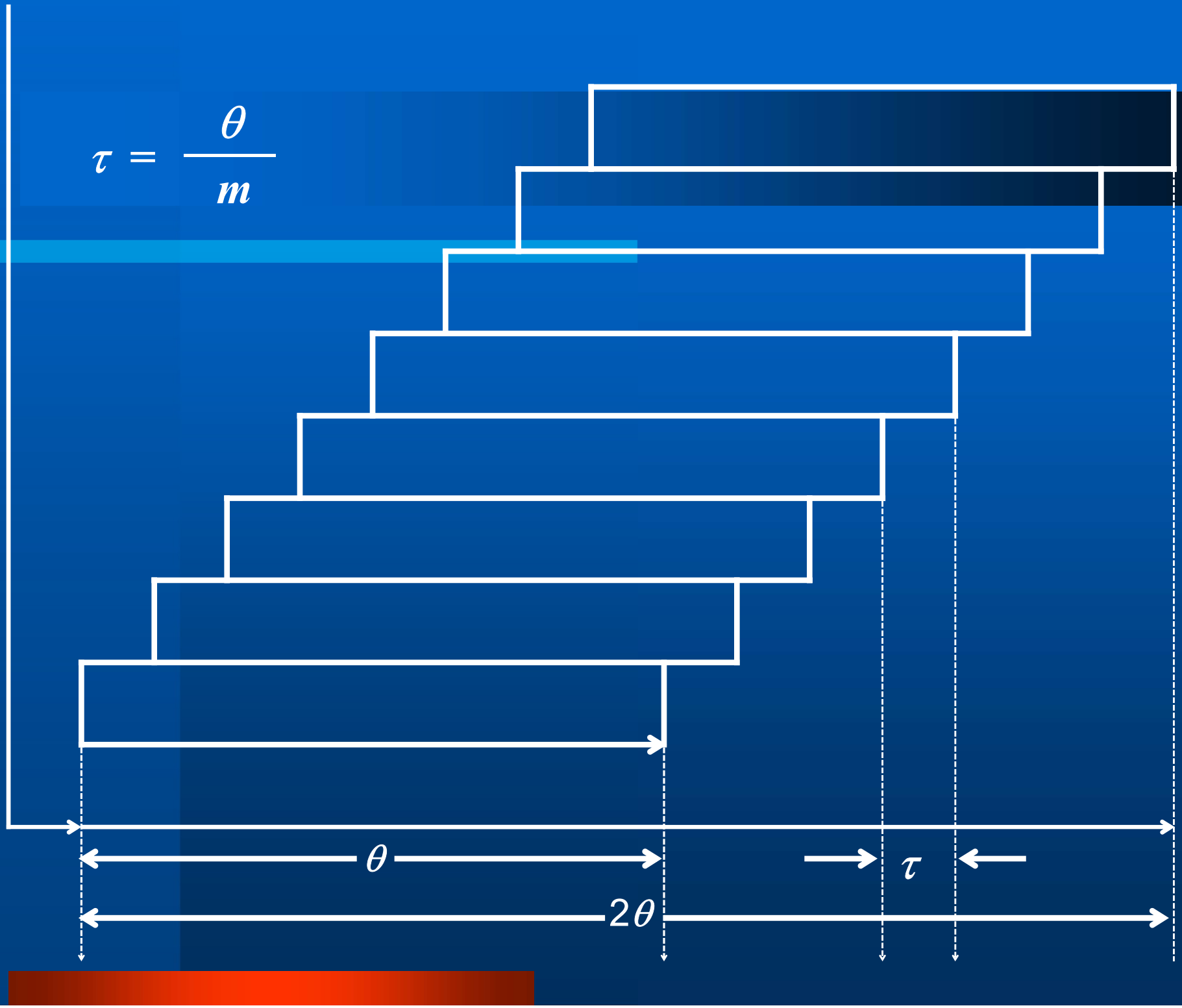
High-order interleaving



Some Definations

- **Major Cycle θ**
  - **The total time required to complete the access of a single word from a module**
- **Minor Cycle τ**
  - **The actual time needed to produce one word, assuming overlapped access of successive memory modules**
- **Degree of interleaving *m***
  - **Number of modules**

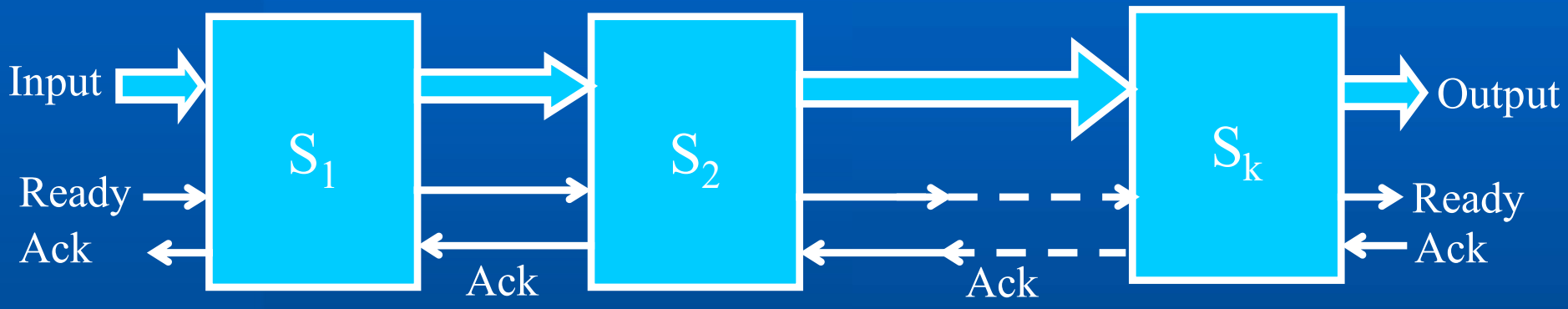
$$\tau = \frac{\theta}{m}$$



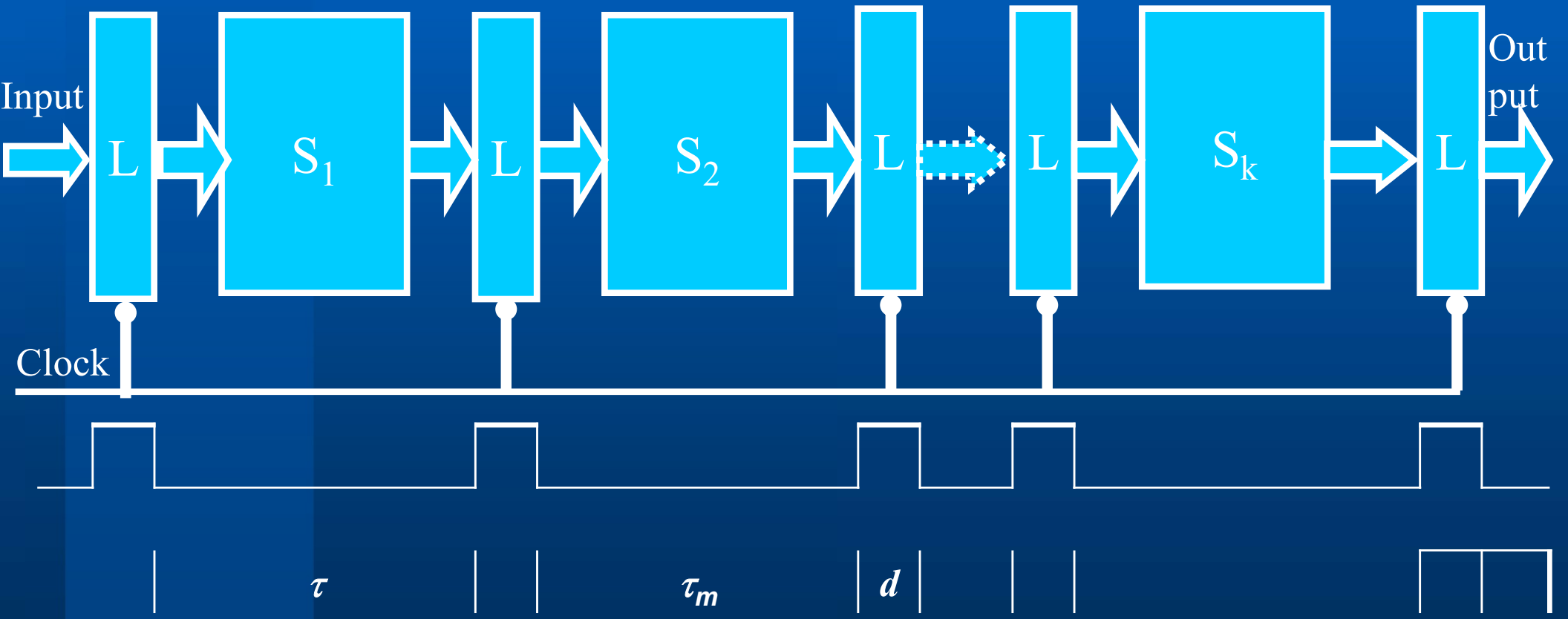
Linear Pipeline Processors

- ~ cascade of processing stages which are linearly connected to perform a fixed function over a stream of data flowing from one end to the other.
- Example: Instruction Execution, Arithmetic computation, Memory Access Operation
- Two Types: Asynchronous, Synchronous

Asynchronous Pipeline Model



Synchronous Pipeline Model



	1	2	3	4
S_1	X			
S_2		X		
S_3			X	
S_4				X

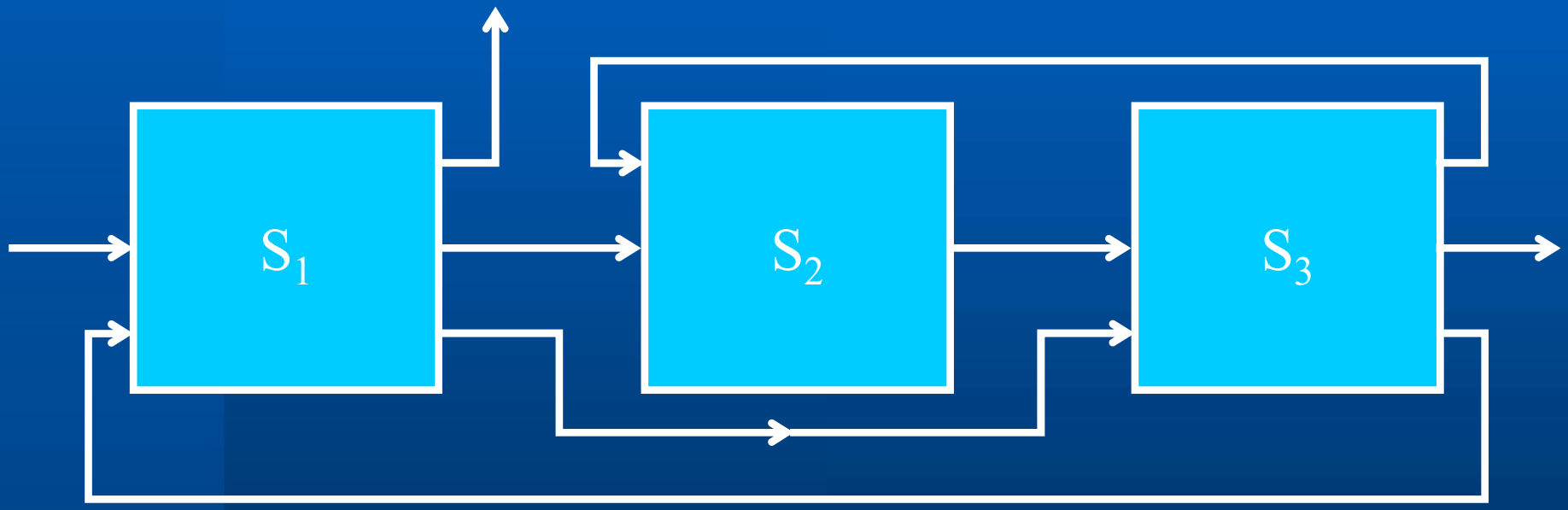
Reservation table of a four-stage linear pipeline

Speedup, Efficiency & Throughput

Nonlinear Pipeline Processor

- ~ allows feedforward and feedback connections in addition to the streamline connections, to perform variable functions at different times.
- ~ also called dynamic pipeline

Nonlinear Pipeline



Reservation Table

	1	2	3	4	5	6	7	8
S_1	X					X		X
S_2		X		X				
S_3			X		X		X	

Reservation Table

	1	2	3	4	5	6
S_1	Y				Y	
S_2			Y			
S_3		Y		Y		Y

Latency Analysis

- **Latency**
  - **The number of time units (clock cycle) between two initiations of a pipeline**
  - **Any attempt by two or more initiations to use the same pipeline stage at the same time will cause a collision**
  - **Latencies that cause collisions are called forbidden latencies**

Latency Analysis

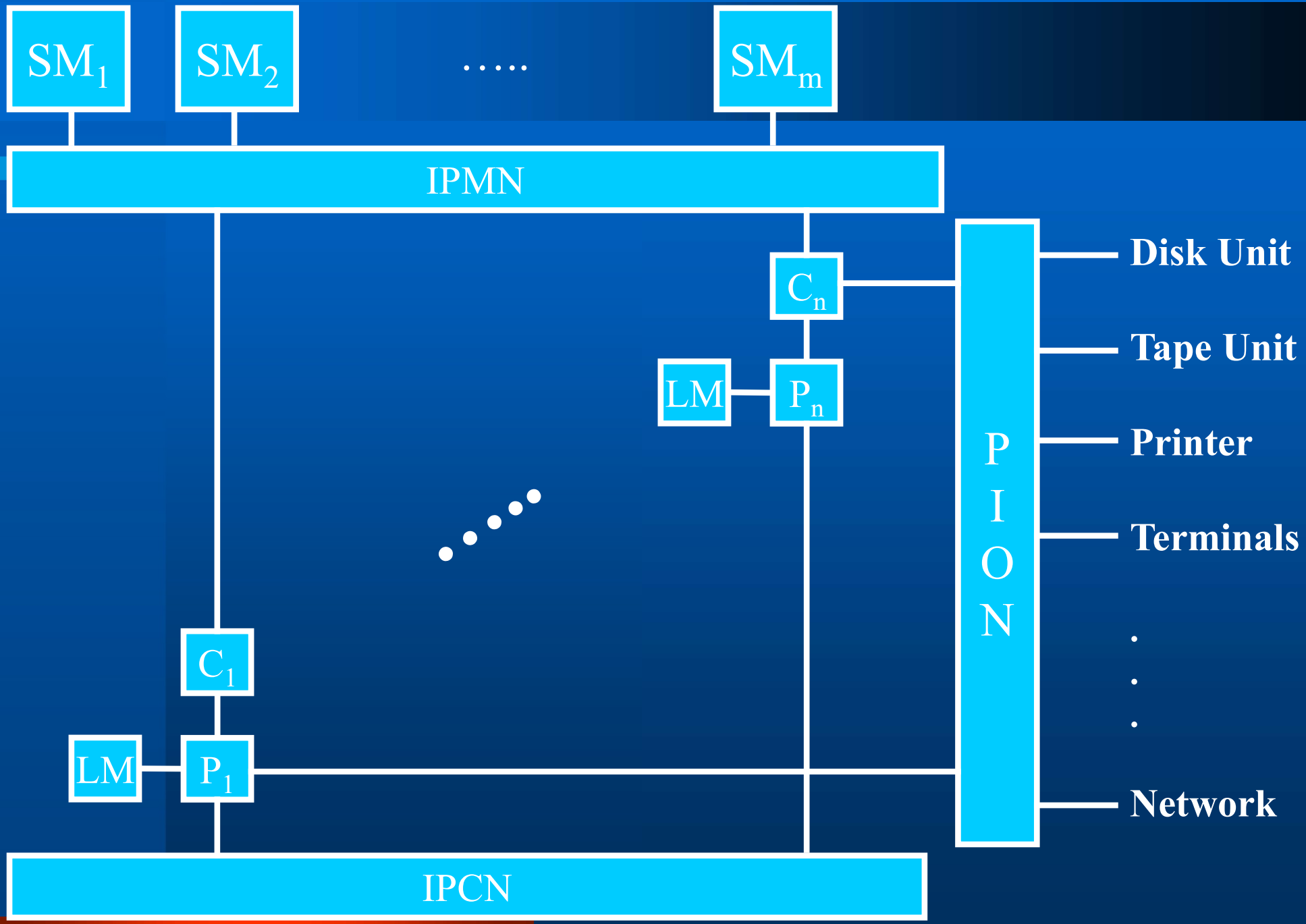
	1	2	3	4	5	6	7	8	9	10	11
S_1	x_1		x_2		x_3	x_1	x_4	$x_1 x_2$		$x_2 x_3$	
S_2		x_1		$x_1 x_2$		$x_2 x_3$		$x_3 x_4$		x_4	
S_3			x_1		$x_1 x_2$		$x_1 x_2 x_3$		$x_2 x_3 x_4$		

- **Latency Sequence**
  - Sequence of all permissible non forbidden latencies between successive task initiations
- **Latency Cycle**
  - Latency Sequence which repeats the same subsequence indefinitely

Collision-free scheduling

- **Collision Vector**
- **State Diagram**

Multiprocessor System Interconnects



Hierarchical Bus System

Class Test- 01, Course: CSE-402, Total Marks: 15

Answer Any Three

5
5
5
5

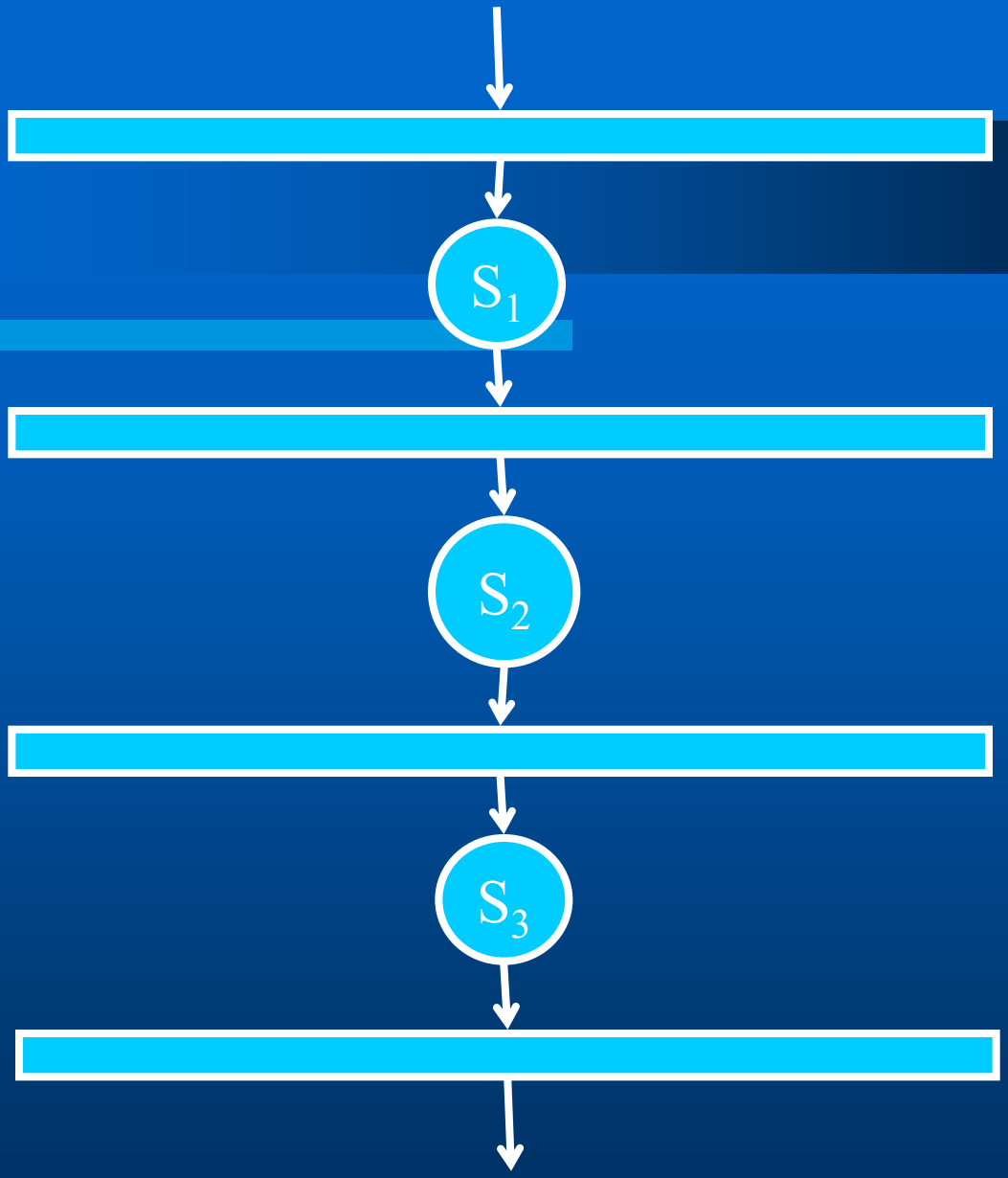
1. Define terms: **a.** Parallel Processing, **b.** Distributed System, **c.** Multiprogramming, **d.** Time-sharing, **e.** Multi-processing.
2. What are the mechanisms of implementing parallel processing in uni-processor system?
3. Draw and discuss the basic structure of an Array Computer?
4. How Flynn classify computers? Discuss with necessary diagrams.

Parallel Processing

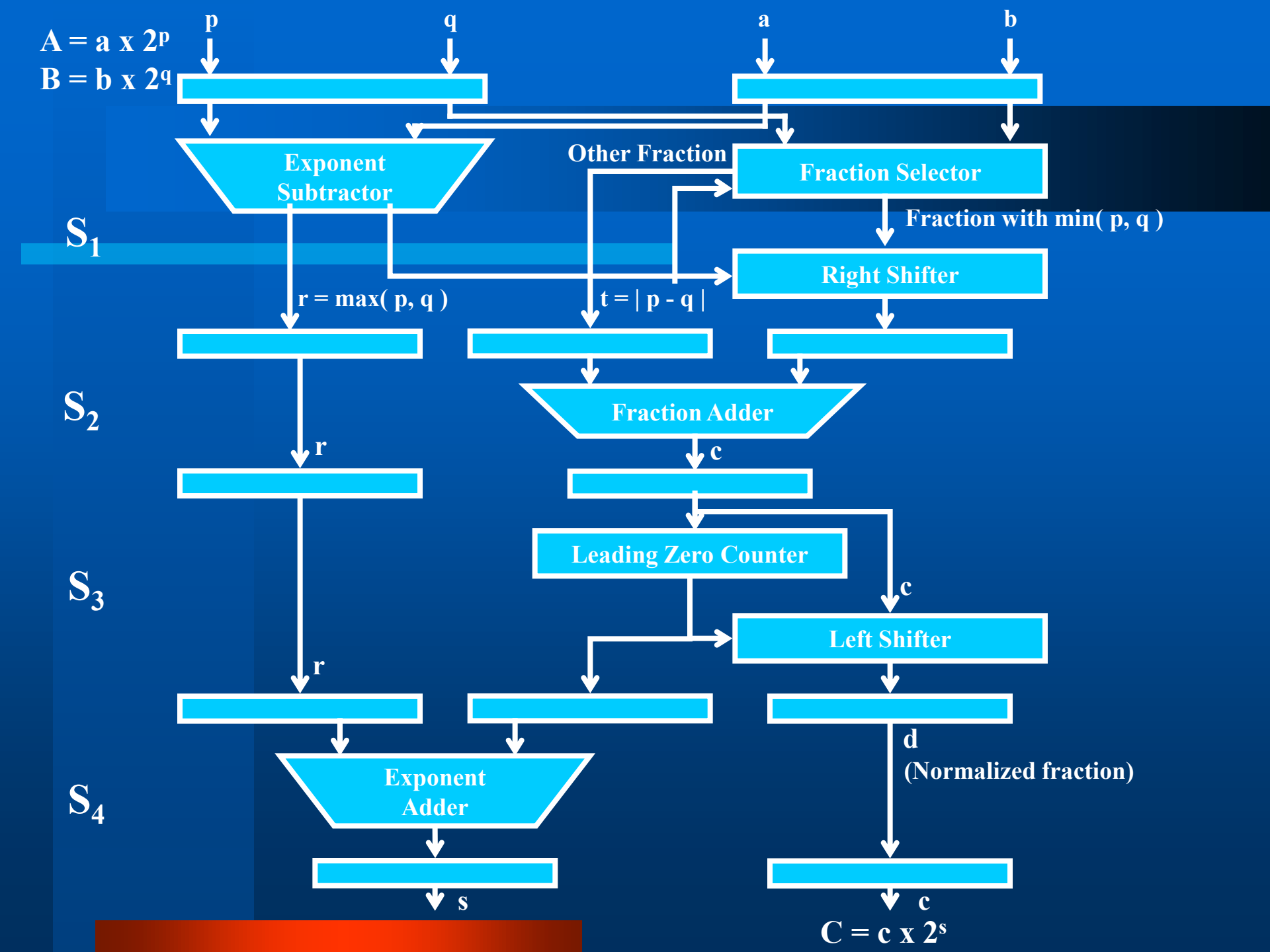
Pipelining and Vector Processing

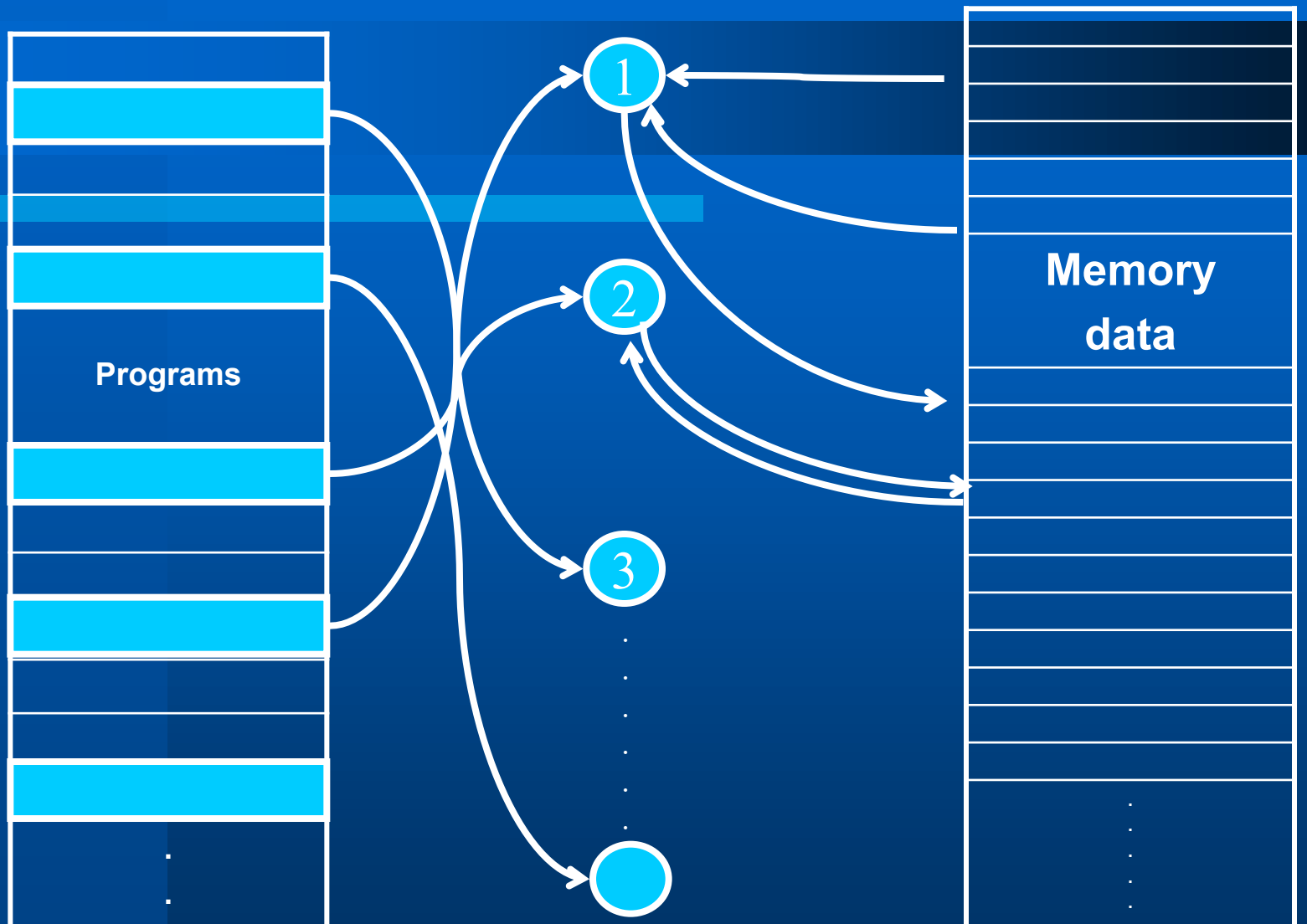
Classification of Pipeline Processor

- **According to the lavels of processing**
  - **Arithmetic pipelining**
  - **Instruction pipelining**
  - **Processor pipelining**

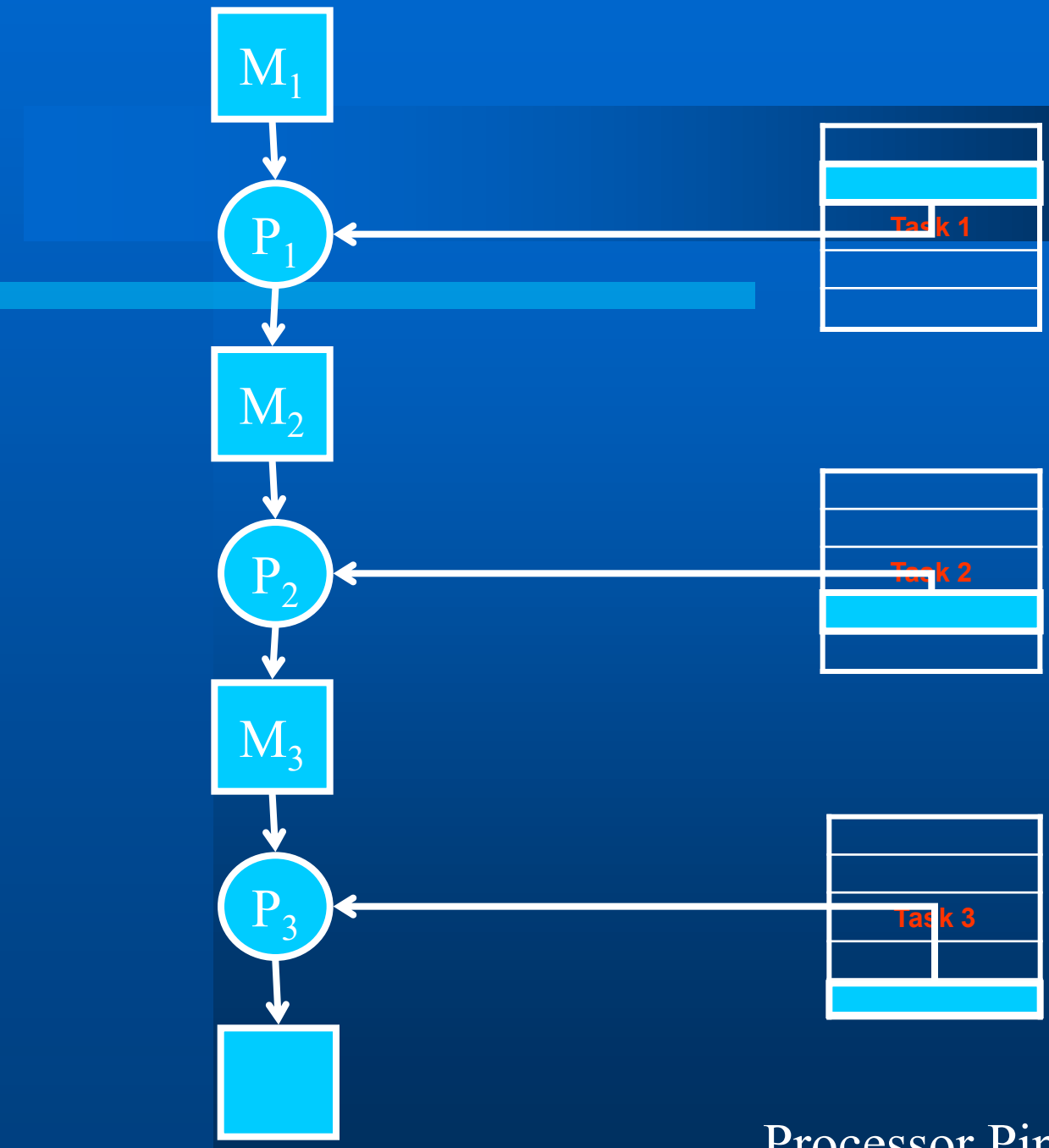


Arithmetic pipelining


A = a x 2^p
B = b x 2^q
p
q
a
b
Exponent Subtractor
Other Fraction
Fraction Selector
Fraction with min(p, q)
S1
Right Shifter
r = max(p, q)
t = | p - q |
S2
Fraction Adder
r
c
Leading Zero Counter
S3
c
Left Shifter
r
d
(Normalized fraction)
Exponent Adder
S4
s
c
C = c x 2^s



Instruction pipelining

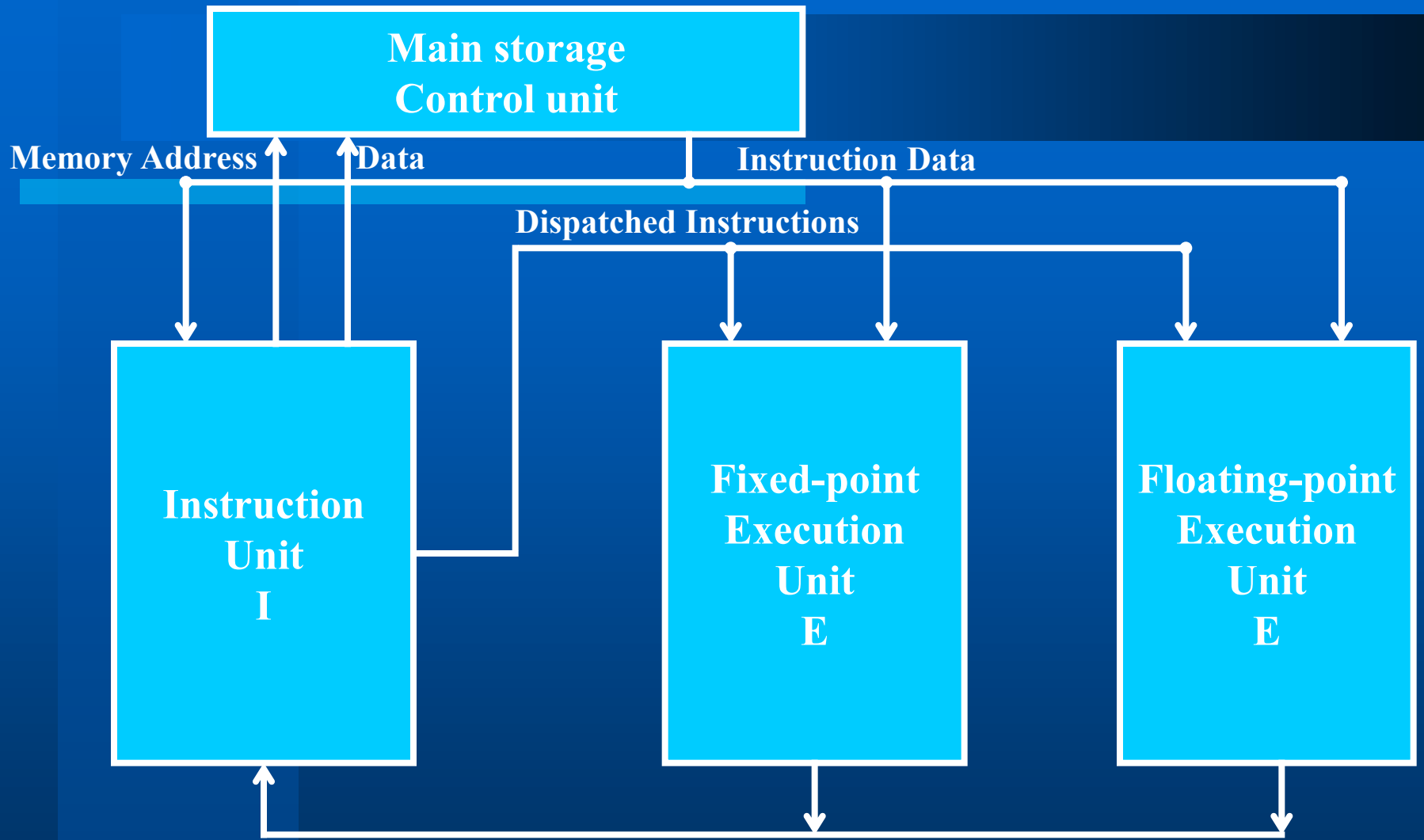


Processor Pipelining

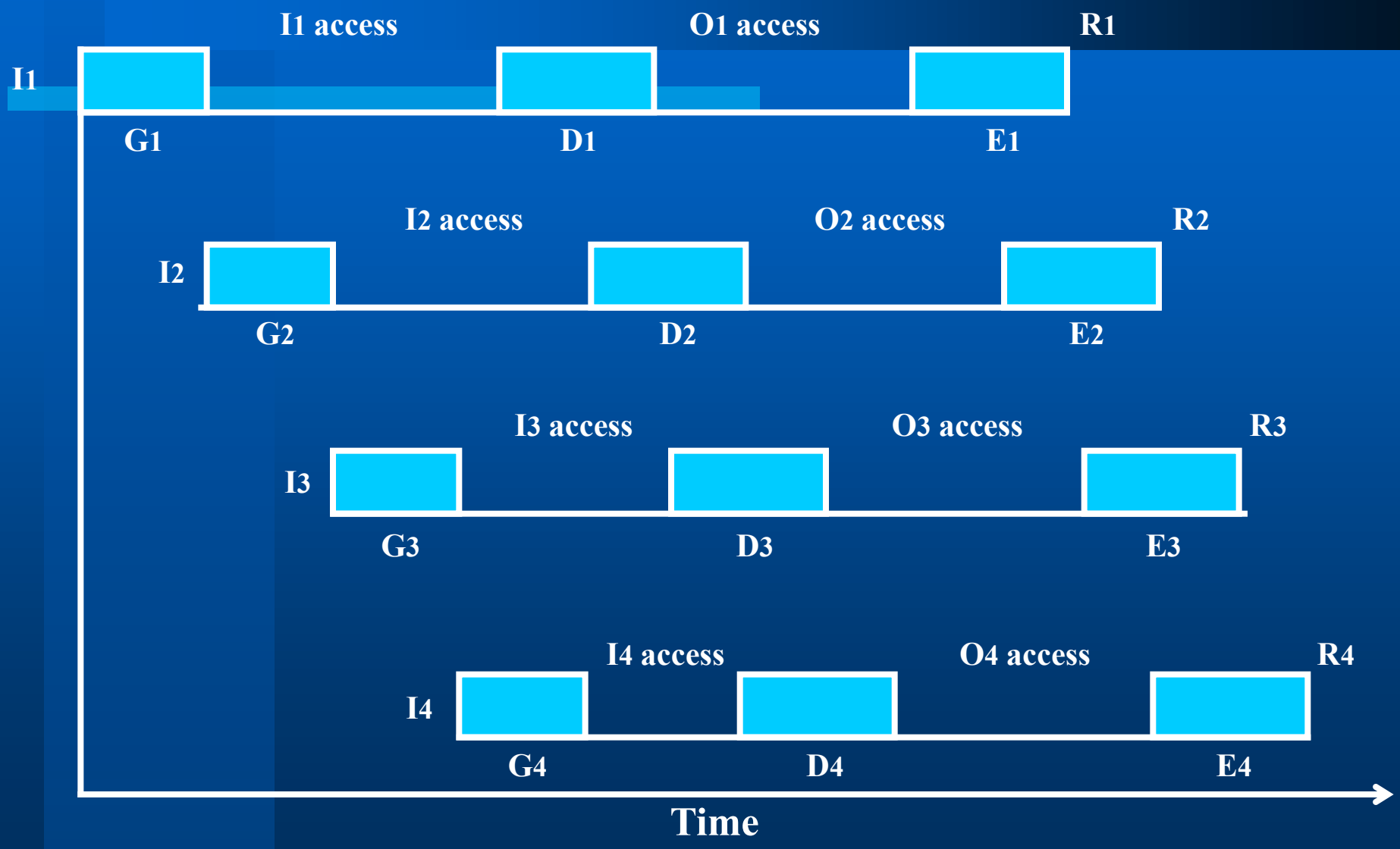
Classification of Pipeline Processor

- **According to pipeline configurations and control strategies**
  - **Unifunction vs. multifunction pipelines**
  - **Static vs. dynamic pipelines**
  - **Scalar vs. vector pipelines**

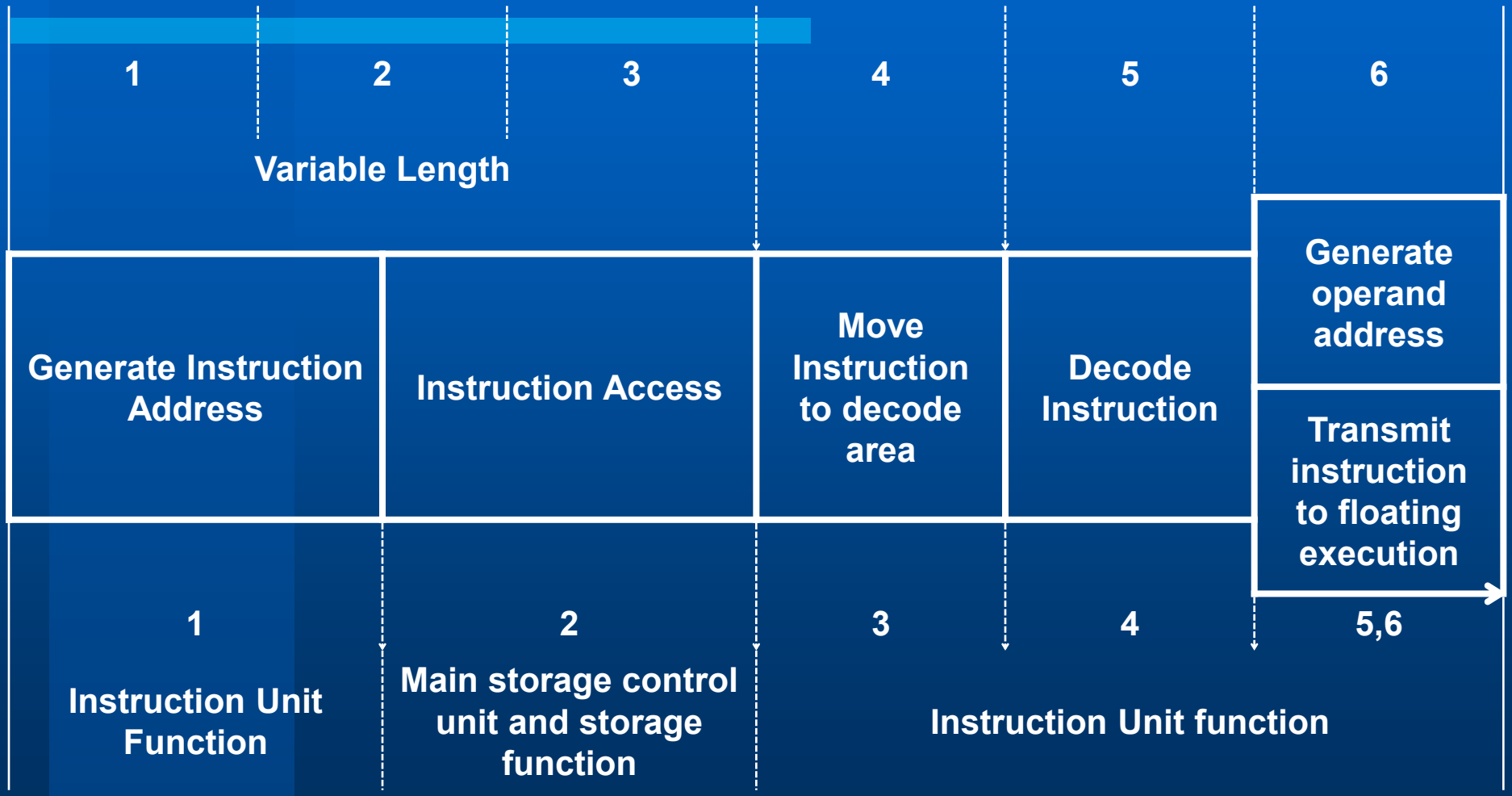
Instruction Pipelines



CPU of IBM System 360/91

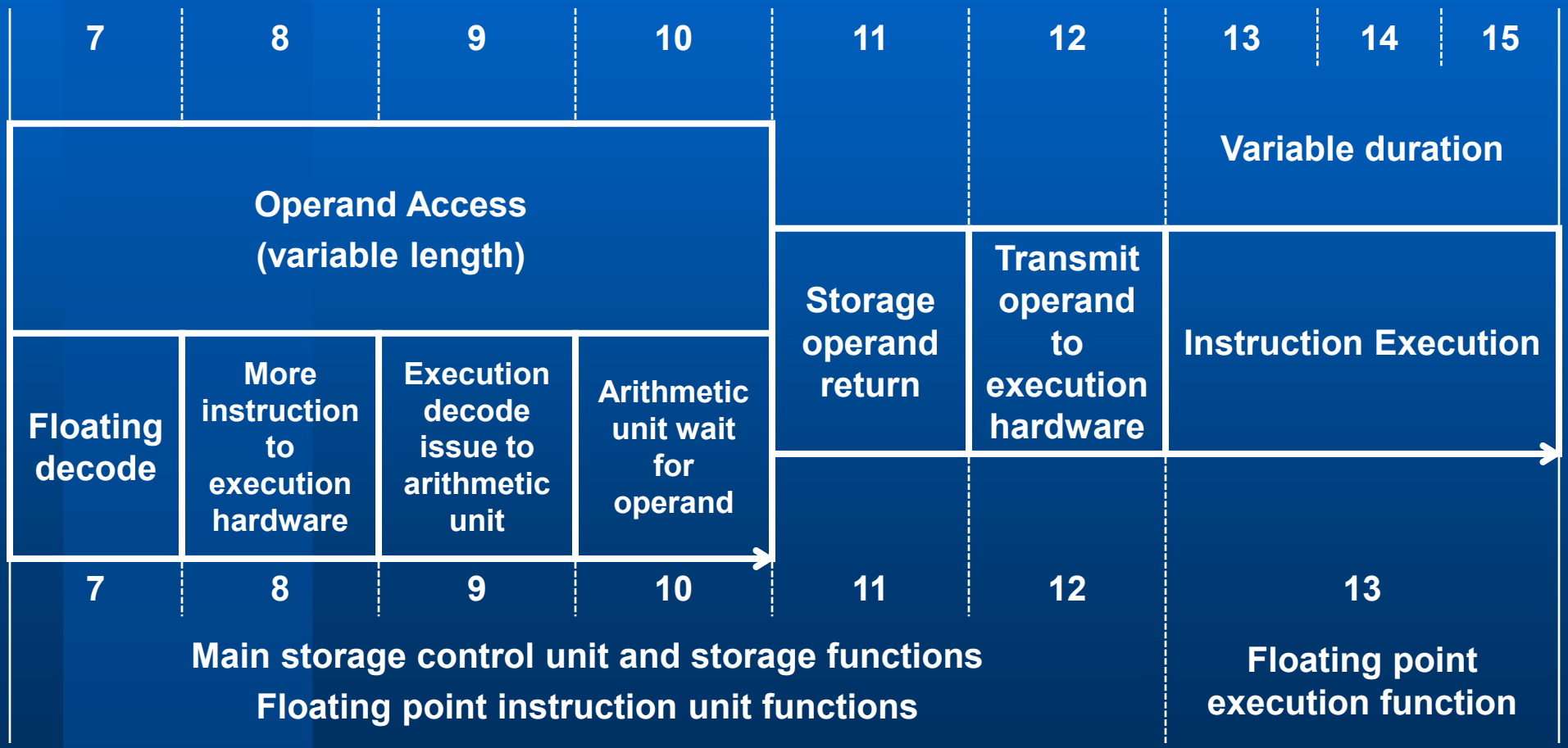

I1 access
O1 access
R1
I1
G1
D1
E1
I2 access
O2 access
R2
I2
G2
D2
E2
I3 access
O3 access
R3
I3
G3
D3
E3
I4 access
O4 access
R4
I4
G4
D4
E4
Time

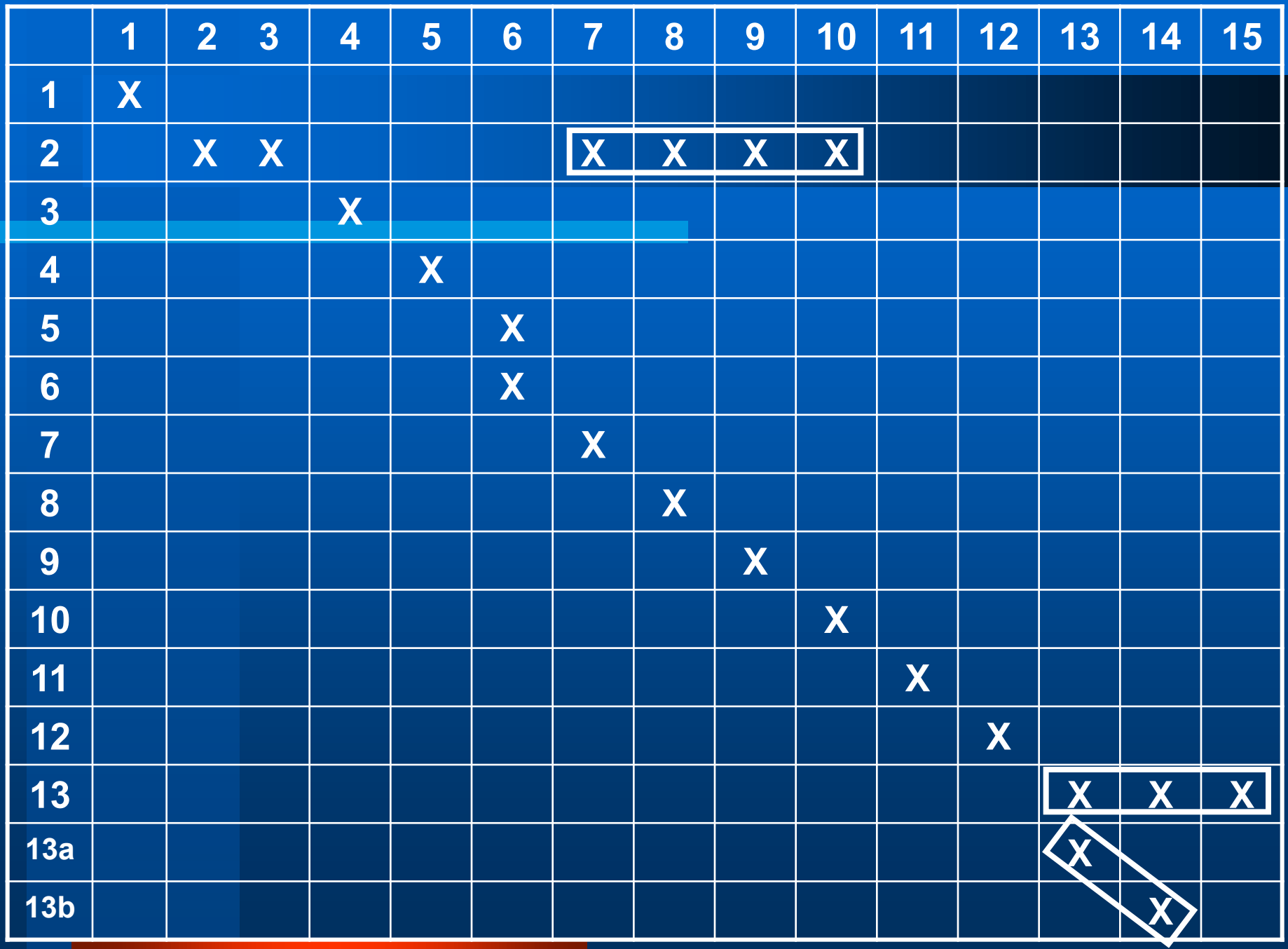
Functional segmentation of typical storage-to-register floating point instruction in the IBM 360/91



Continue…

Functional segmentation of typical storage-to-register floating point instruction in the IBM 360/91




1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 X
2 X X X X X X
3 X
4 X
5 X
6 X
7 X
8 X
9 X
10 X
11 X
12 X
13 X X X
13a X
13b X

Arithmetic Pipeline Design

General Binary Multication

						a_5	a_4	a_3	a_2	a_1	a_0	= A
					x)	b_1	b_4	b_3	b_2	b_1	b_0	= B
						a_5b_0	a_4b_0	a_3b_0	a_2b_0	a_1b_0	a_0b_0	= W_1
					a_5b_1	a_4b_1	a_3b_1	a_2b_1	a_1b_1	a_0b_1		= W_2
				a_5b_2	a_4b_2	a_3b_2	a_2b_2	a_1b_2	a_0b_2			= W_3
			a_5b_3	a_4b_3	a_3b_3	a_2b_3	a_1b_3	a_0b_3				= W_4
		a_5b_4	a_4b_4	a_3b_4	a_2b_4	a_1b_4	a_0b_4					= W_5
+)	a_5b_5	a_4b_5	a_3b_5	a_2b_5	a_1b_5	a_0b_5						= W_6
P_{11}	P_{10}	P_9	P_8	P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0	= AxB = P

Carry Save Adder (CSA)

	A	=		1	1	1	1	0	1
	B	=		0	1	0	1	1	0
+)	D	=		1	1	0	1	1	1
	C	=	1	1	0	1	1	1	
⊕)	S	=		0	1	1	1	0	0
A + B + D = C ⊕ S		=	1	1	1	0	0	1	0

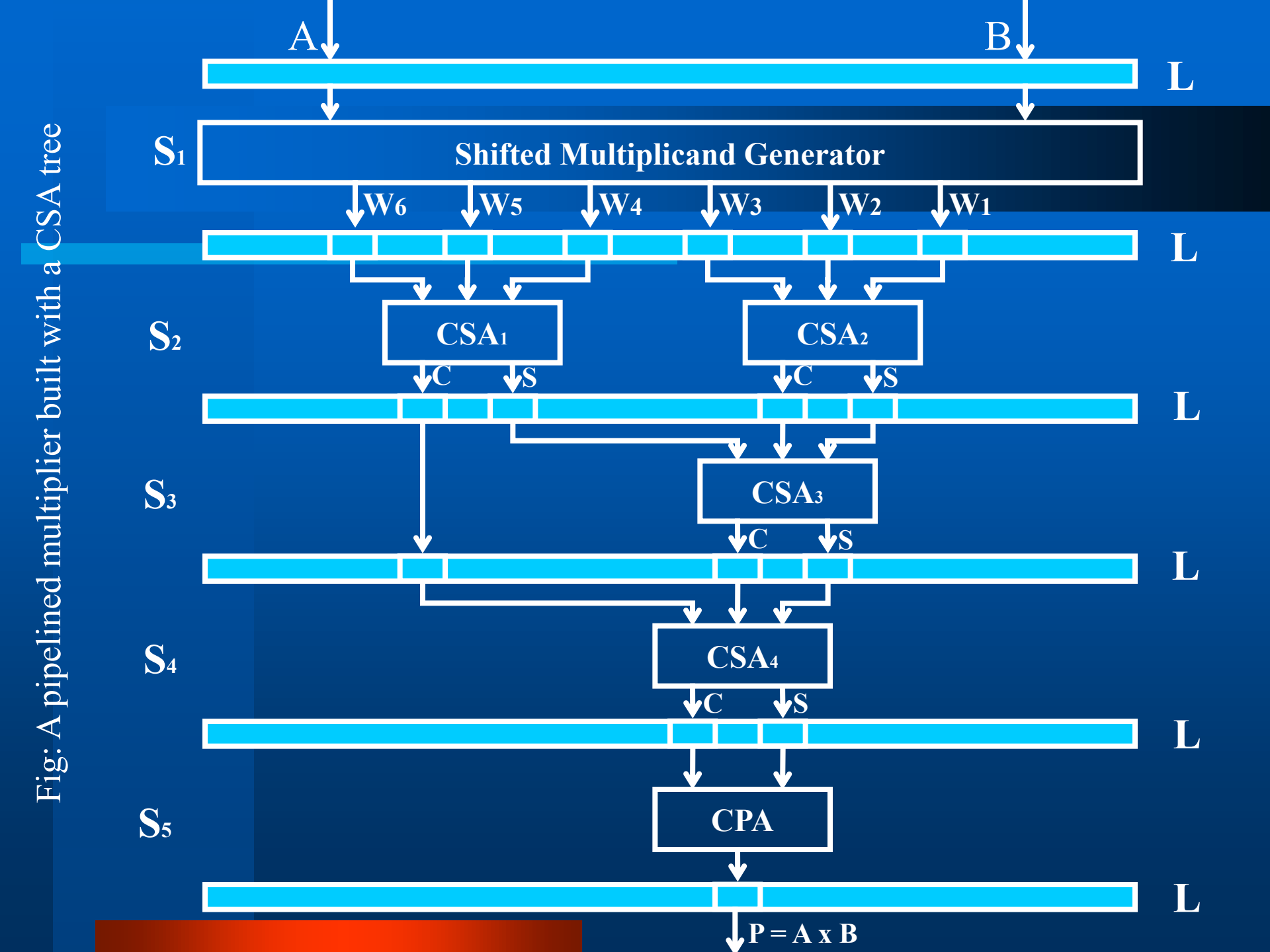


Fig: A pipelined multiplier built with a CSA tree

S_1 : Shifted Multiplicand Generator

A=101101 B=110101

W_1 =						1	0	1	1	0	1
W_2 =					0	0	0	0	0	0	
W_3 =				1	0	1	1	0	1		
W_4 =			0	0	0	0	0	0			
W_5 =		1	0	1	1	0	1				
W_6 =	1	0	1	1	0	1					

S_2 : CSA

CSA_2

W_1	=	0	0	0	0	0	1	0	1	1	0	1
W_2	=	0	0	0	0	0	0	0	0	0	0	0
W_3	=	0	0	0	1	0	1	1	0	1	0	0
C_2	=	0	0	0	0	1	0	0	1	0	0	0
S_2	=	0	0	0	1	0	0	1	1	0	0	1

CSA_1

W_4	=	0	0	0	0	0	0	0	0	0	0	0
W_5	=	0	1	0	1	1	0	1	0	0	0	0
W_6	=	1	0	1	1	0	1	0	0	0	0	0
C_1	=	0	0	1	0	0	0	0	0	0	0	0
S_1	=	1	1	1	0	1	1	1	0	0	0	0

S_3, S_4 : CSA

CSA3

C_2	=	0	0	0	0	1	0	0	1	0	0	0
S_2	=	0	0	0	1	0	0	1	1	0	0	1
S_1	=	1	1	1	0	1	1	1	0	0	0	0
C_3	=	0	0	0	1	0	1	1	0	0	0	0
S_3	=	1	1	1	1	0	1	0	0	0	0	1

CSA4

C_3	=	0	0	0	1	0	1	1	0	0	0	0
S_3	=	1	1	1	1	0	1	0	0	0	0	1
C_1	=	0	0	1	0	0	0	0	0	0	0	0
C_4	=	0	1	1	0	1	0	0	0	0	0	0
S_4	=	1	1	0	0	0	0	1	0	0	0	1

S_5: CPA

CPA

C_4	=	0	1	1	0	1	0	0	0	0	0
S_4	=	1	1	0	0	0	0	1	0	0	1
P	=	1	0	0	1	0	1	0	1	0	0	0	1

- A *v*-level CSA tree can add up to $N(v)$ input numbers.

$$N(v) = \left\lfloor \frac{N(v-1)}{2} \right\rfloor \times 3 + N(v-1) \bmod 2$$

with

$$N(1) = 3$$

  - 10-level CSA tree adds 64 to 94 numbers in one pass

- Using one pass CSA tree for multiplying two 32-bit numbers requires
  - 8 CSA stages
  - 30 CSAs and
  - 11 clock cycles.
    - 1 cycle for the input stage
    - 8 cycles for the one pass CSA tree stages
    - 2 cycles for the CPA stage

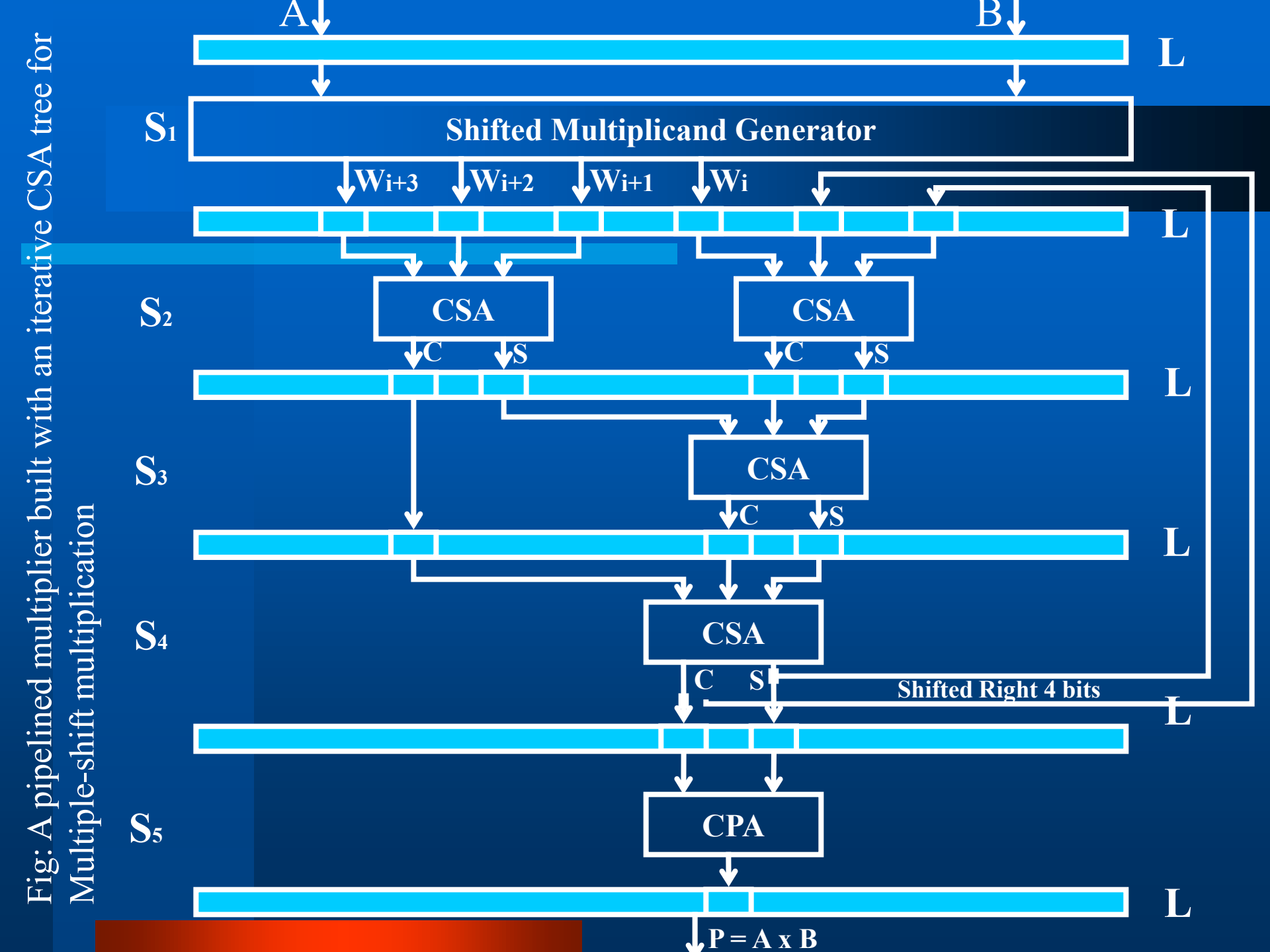


Fig: A pipelined multiplier built with an iterative CSA tree for Multiple-shift multiplication

- Using iterative CSA tree for multiplying two 32-bit numbers requires
  - 3 iterative CSA stages
  - 4 CSAs
    - (30 CSAs for one pass CSA tree)
  - 26 clock cycles
    - (11 cycles for one pass CSA tree)

Reservation table of iterative CSA tree for multiple-shift multiplication

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
S_1	X			X			X			X			X			X			X			X					
S_2		X			X			X			X			X			X			X			X				
S_3			X			X			X			X			X			X			X			X				
S_4				X			X			X			X			X			X			X			X			
S_5																										X	X

Vector Processing Principles

- **Vector: A vector is a set of scalar data items, all of the same type, stored in memory.**
- **Stride: The vector elements are ordered to have a fixed addressing increment between successive elements called the stride.**

Vector Processing Principles

- **Vector processor: A vector processor is an ensemble of hardware resources, including vector registers, functional pipelines, processing elements, and register counters, for performing vector operations.**
- **Vector processing: ~ occurs when arithmetic or logical operations are applied to vector.**
- **Vectorization: The conversion from scalar code to vector code is called vecttorization.**

Vector instruction types

- **Vector-vector instruction**
  - $f_1: V_i \rightarrow V_j$
  - $f_2: V_j \times V_k \rightarrow V_i$
- **Vector-scalar instruction**
  - $f_3: s \circ V_i \rightarrow V_j$
- **Vector-memory instructions**
  - $f_4: M \rightarrow V$
  - $f_5: V \rightarrow M$

Vector instruction types

- **Vector reduction instructions**
  - $f_6: V_i \rightarrow s_j$
  - $f_7: V_i \text{ o } V_j \rightarrow s_k$
- **Gather and scatter instructions**
  - $f_8: M \rightarrow V_1 \times V_0$ Gather
  - $f_9: V_1 \times V_0 \rightarrow M$ Scatter
- **Masking instructions**
  - $f_{10}: V_0 \times V_m \rightarrow V_1$

Compound Vector Processing

- Compound Vector Operations
- Multi-pipeline chaining and networking techniques
- Graph transformation for setting up pipeline network

Compound Vector Operations

- ***Compound vector function:***
  - **A composite function of vector operations converted from a looping structure of linked scalar operations.**
  - **A compound vector function called SAXPY code.**

```
      DO 10 I 1=N
            Load        R1, X(I)
            Load        R2, Y(I)
            Multiply    R1, S
            Add         R2, R1
            Store       Y(I), R2
10    Continue
```

SAXPY Code

M(1:N) → V1	*Vector load*
M(x:x+N-1) → V2	*Vector load*
S X V1 → V1	*Vector multiply*
V2 + V1 → V2	*Vector add*
V2 → M(y:y+N-1)	*Vector store*

$$Y(I) = S * X(I) + Y(I)$$

Compound Vector Functions

Table 8.6 Representative Compound Vector Functions

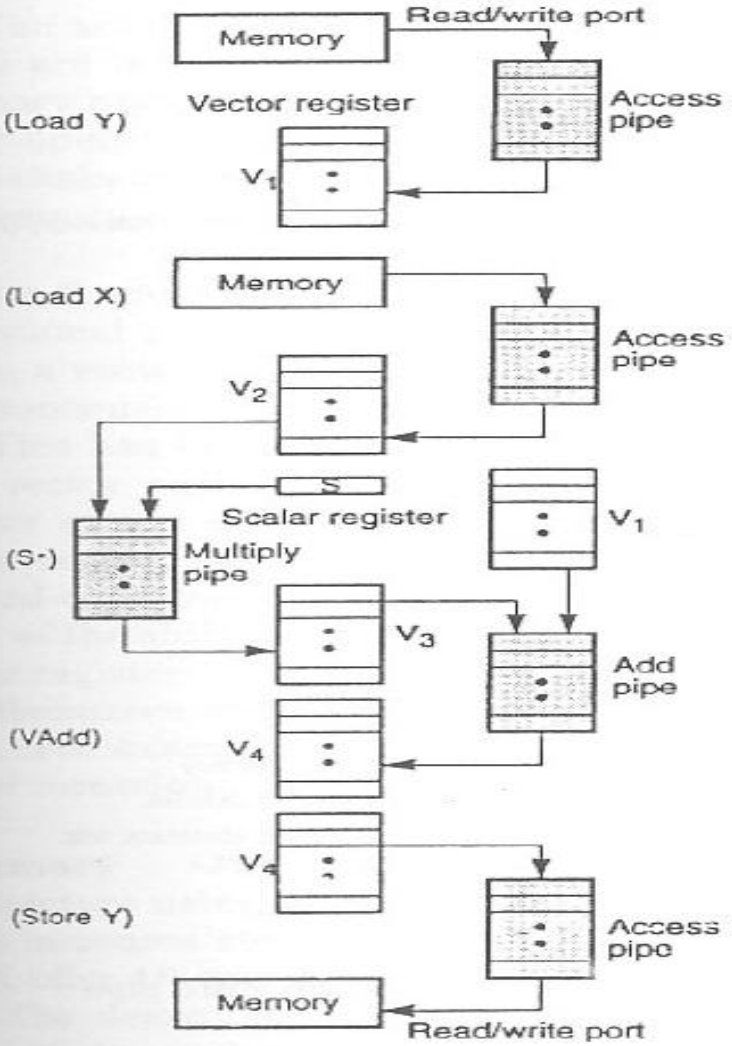
One-dimensional compound vector functions	Maximum chaining degree
V1(I) = V2(I) + V3(I) × V4(I)	2
V1(I) = B(I) + C(I)	3
A(I) = V1(I) × S + B(I)	4
A(I) = V1(I) + B(I) + C(I)	5
A(I) = B(I) + S × C(I)	5
A(I) = B(I) + C(I) + D(I)	6
A(I) = Q × V1(I) × (R × B(I) + C(I))	7
A(I) = B(I) × C(I) + D(I) × V1(I)	7
A(I) = V1(I) + (1 / A(I) + 1 / B(I)) + Log(V2(I))	8
A(I) = $\sqrt{V2(I)}$ + Sin(B(I) + C(I)) + V3(I)	8
A(I) = B(I) × C(I) + D(I) × E(I) × S	9
A(I) = (A(I) + B(I) × C(I) + D(I)) × (I)	10

Note: Vi(I) are vector registers in the processor. A(I), B(I), C(I), D(I), and E(I) are vectors in memory. Scalars are indicated as Q, R, and S are available from scalar registers in the processor. The chaining degrees include both memory-access and functional pipeline operations.

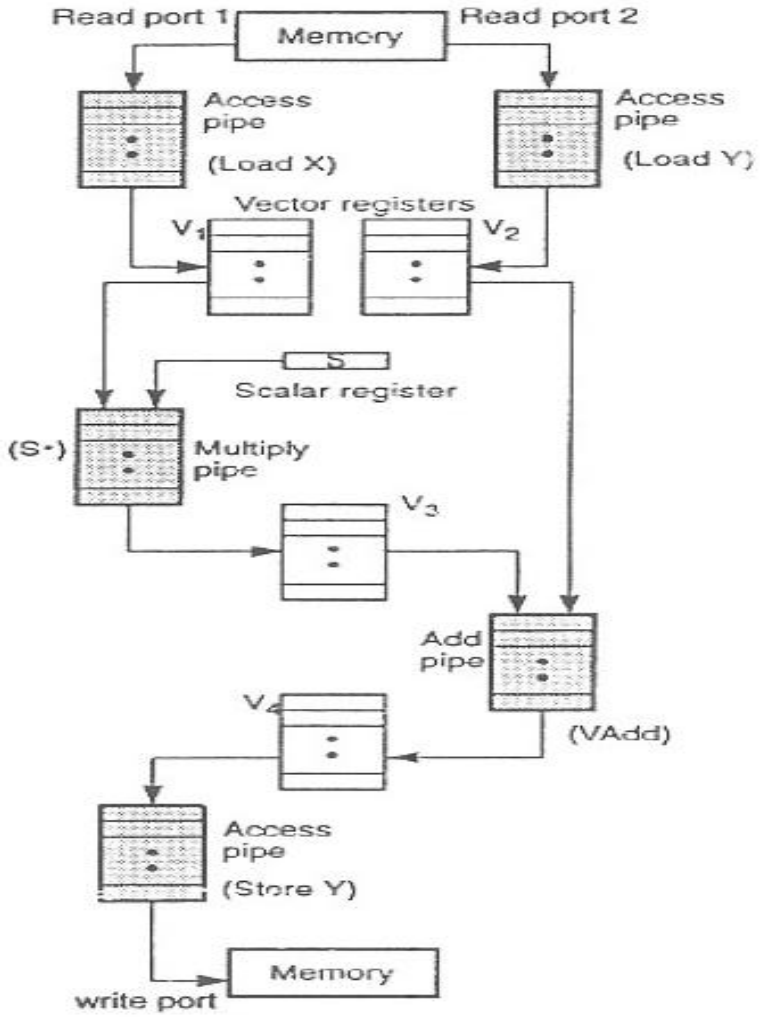
Vector Loop & Chaining

- **Vector loops or Strip-mining**
  - **When a vector has a length greater than that of the vector registers, segmentation of the long vector into fixed-length segments is necessary. This technique is called *strip-mining*.**
  - **The program construct for processing long vectors is called a vector loop.**

Vector Loop & Chaining



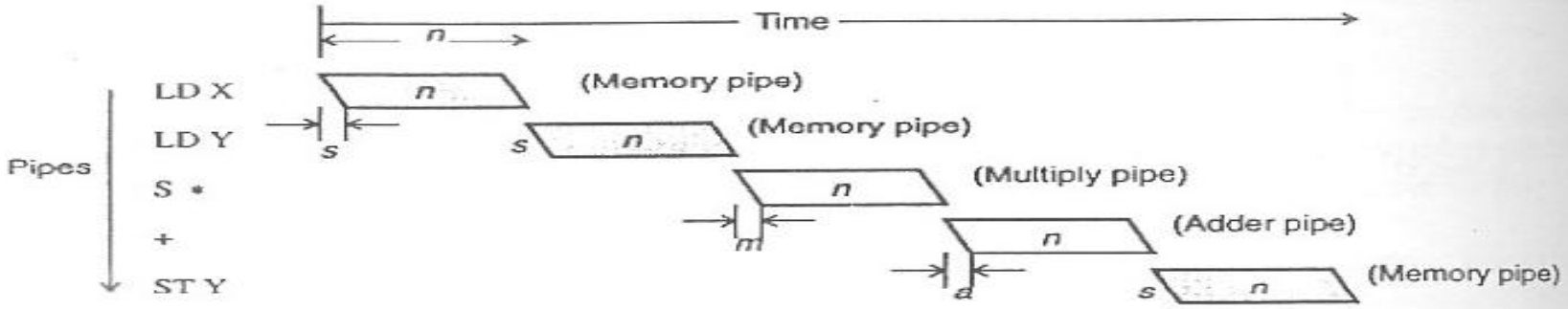
(a) Limited chaining using only one memory-access pipe in the Cray 1



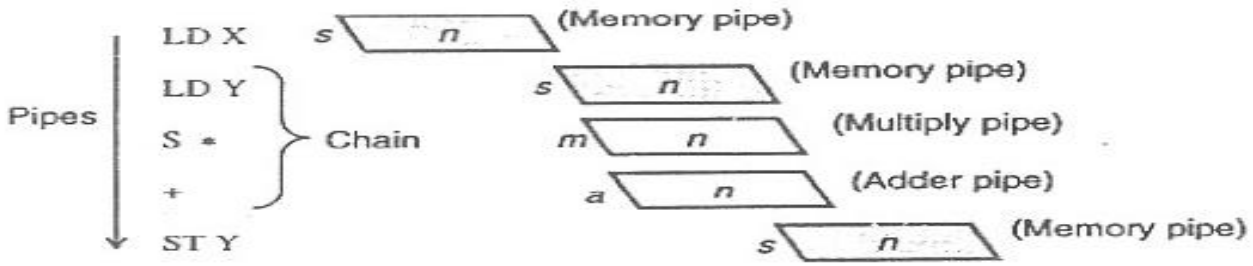
(b) Complete chaining using three memory-access pipes in the Cray X-MP

Figure 8.18 Multipipeline chaining on Cray 1 and Cray X-MP for executing the SAXPY code: Y(1:N) = S × X(1:N) + Y(1:N). (Courtesy of Cray Research, Inc., 1985)

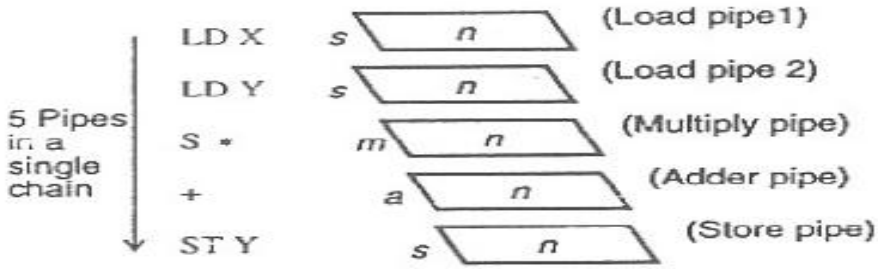
Vector Loop & Chaining



(a) Sequential execution without chaining



(b) Chaining of arithmetic pipes with only one memory pipe



Captions:

s = Memory access latency

m, a = Multiply/add latencies.

n = Time to process n elements, one per cycle.

(c) Chaining in two load pipes, two arithmetic pipes, and one store pipe

Figure 8.19 Timing for chaining the SAXPY code Y(1:N) = S × X(1:N) + Y(1:N) under different memory-access capabilities. (Courtesy of Cray Research, Inc., 1985)

Vector Loop & Chaining

- **Functional Units Independence**
- **Chaining Limitations**
  - **Vector chaining is limited by the small number of functional pipelines available in a vector processor.**
  - **The limited number of vector registers also imposes an additional limit on chaining.**

Vector Loop & Chaining

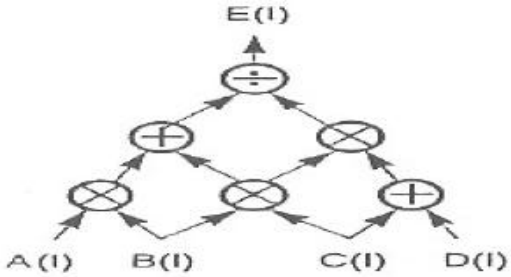
- **Vector Recurrence**
  - **A special class of vector loops in which the outputs of a functional pipeline may be feed back into one of its own source vector registers.**
  - **In other words, a vector register is used for holding the source operands and the result elements simultaneously**

Multi-pipeline Networking

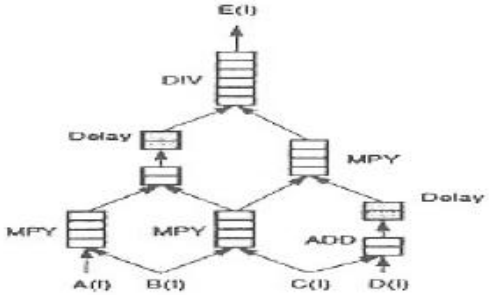
- **Pipeline Net (Pipenet)**
- **Example:**

$$E(I) = \frac{A(I) \times B(I) + B(I) \times C(I)}{B(I) \times C(I) \times [C(I) + D(I)]}$$

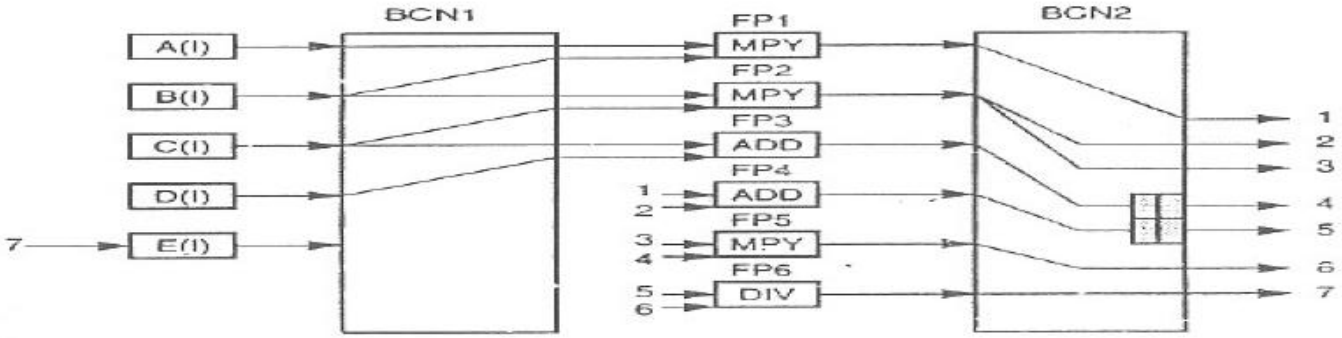
Multi-pipeline Networking



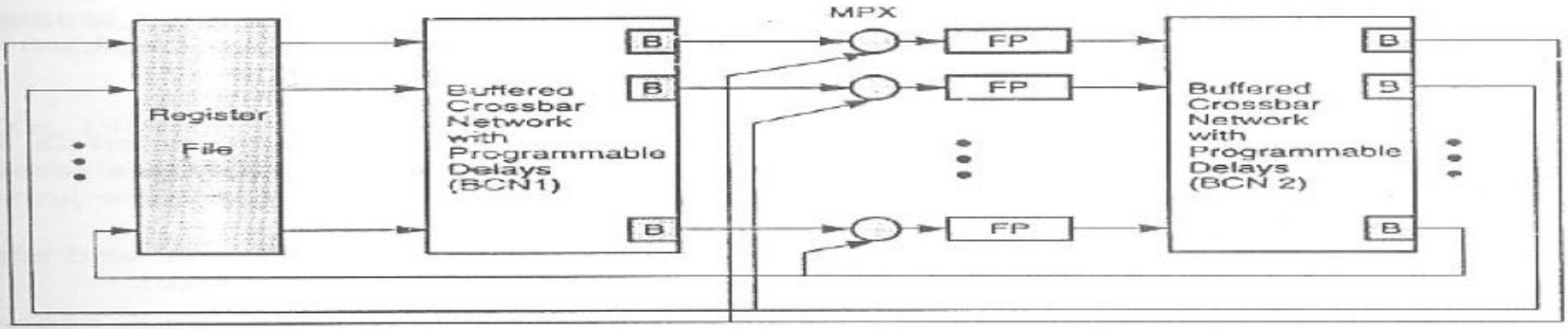
(a) A program graph



(b) The pipenet



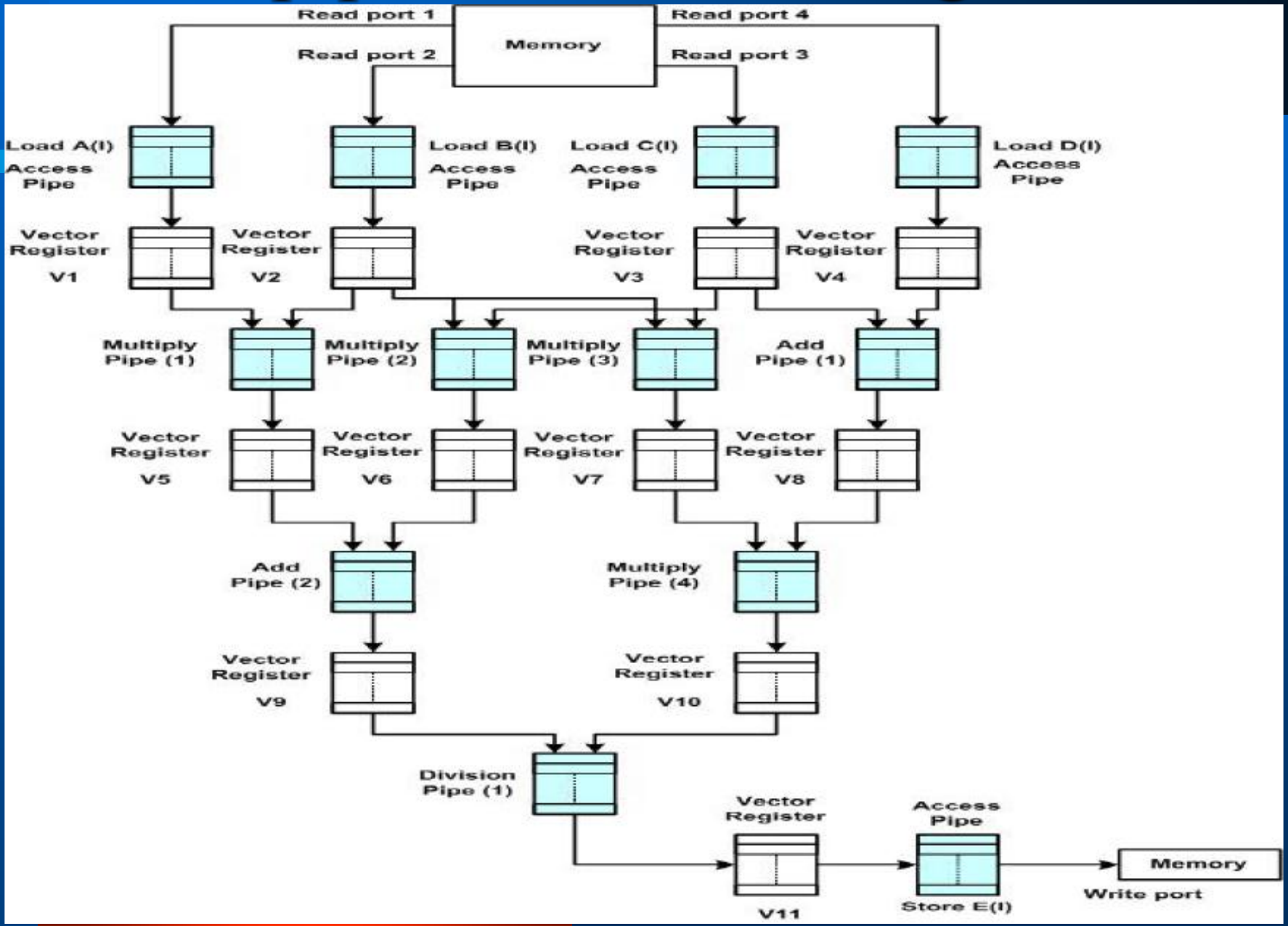
(c) A crossbar implementation



(d) A generalized pipenet model

Figure 8.20 The concept of a pipenet and its implementation model. (Reprinted from Hwang and Xu, *IEEE Transactions on Computers*, Jan. 1988)

Multi-pipeline Networking



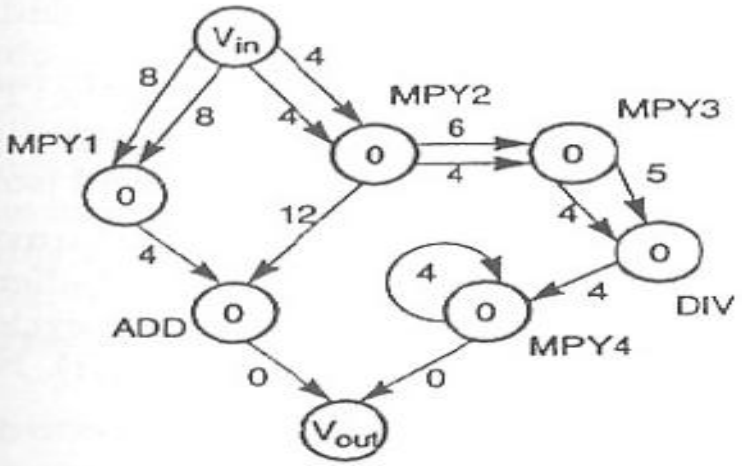
Program Graph Transformations

- Node delays corresponds to appropriate FPs
- Edge delays are the signal flow delays along the connecting path between FPs

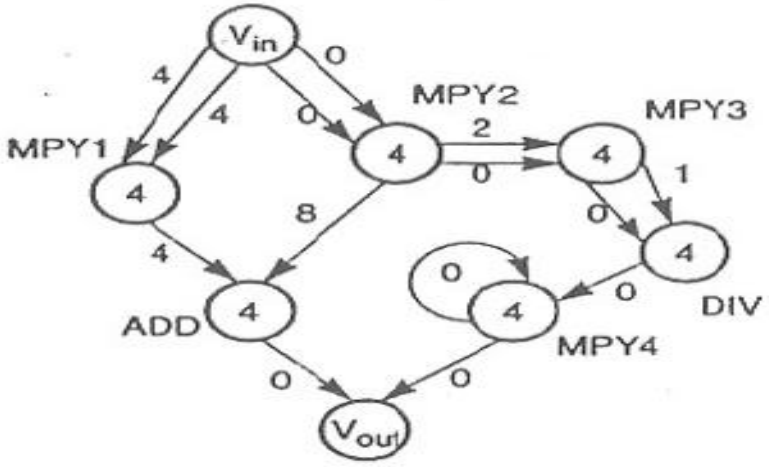
Convertion Process

- **Lemma 1**: Adding ***k***-delays to any node in a systolic program graph and subtracting k-delays from all incoming edges to that node will produce an equivalent program graph.
- **Lemma 2**: An equivalent program graph is generated if all nodal and edge delays are multiplied by the same positive integer, called the ***scaling constant***.

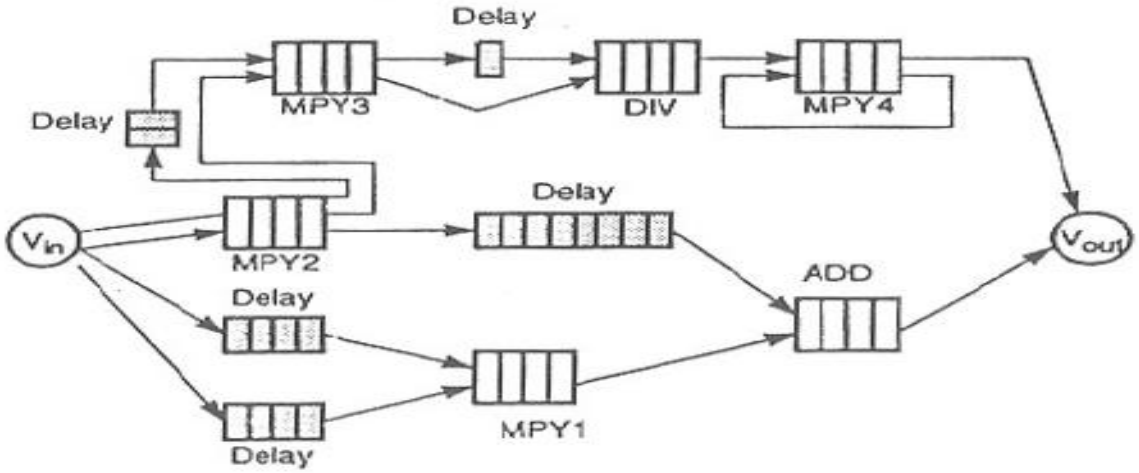
Multi-pipeline Networking



(a) A systolic program graph



(b) After graph transformation



(c) Pipenet implementation with inserted delays between pipelines

Figure 8.21 From synchronous program graph to pipenet implementation. (Reprinted from Hwang and Xu, *IEEE Transactions on Computers*, Jan. 1988)

Vector Processing Requirements

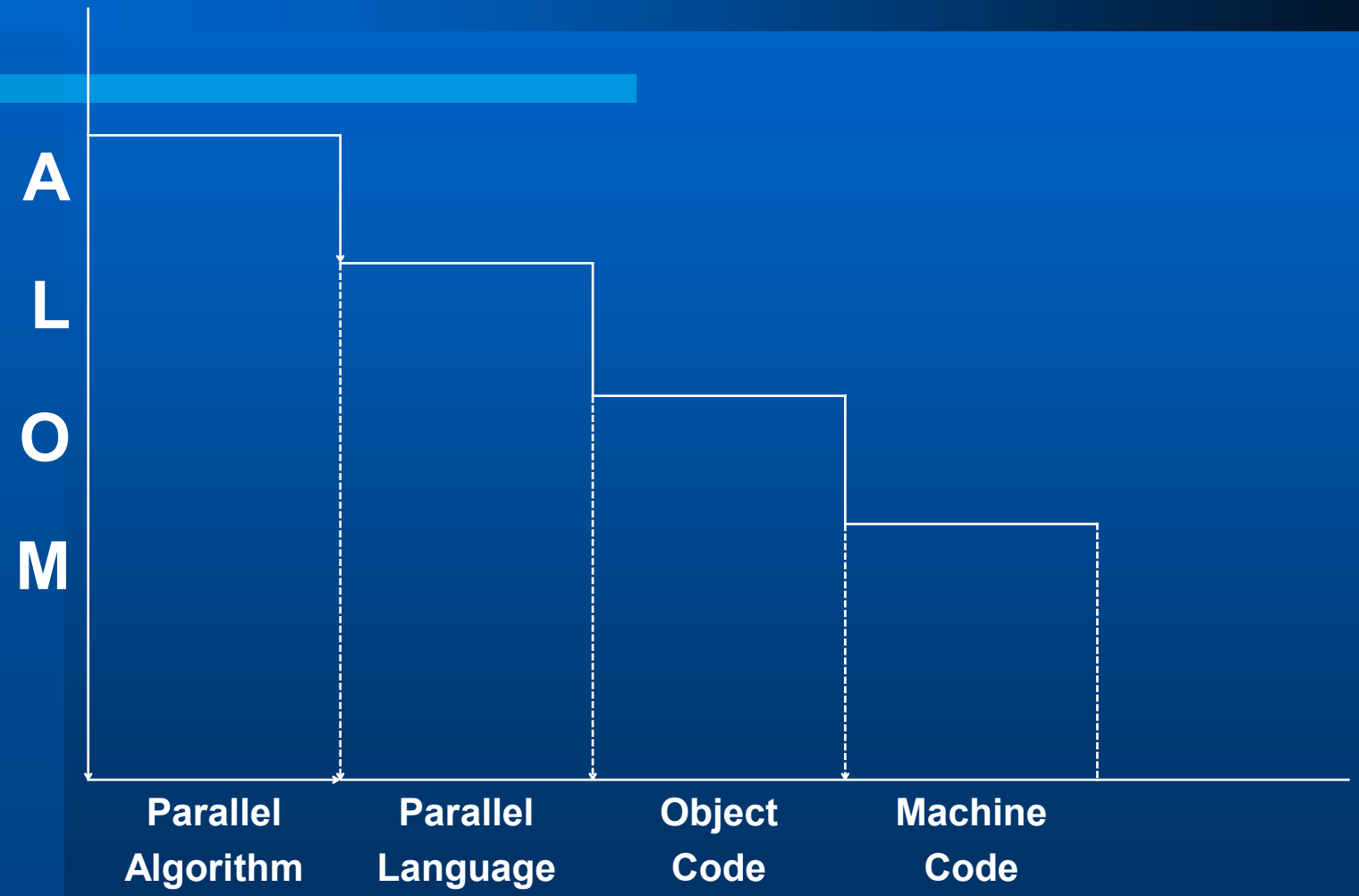
- Characteristic of Vector Processing
- Multiple Vector Task Dispatching
- Pipelined Vector Processing Method

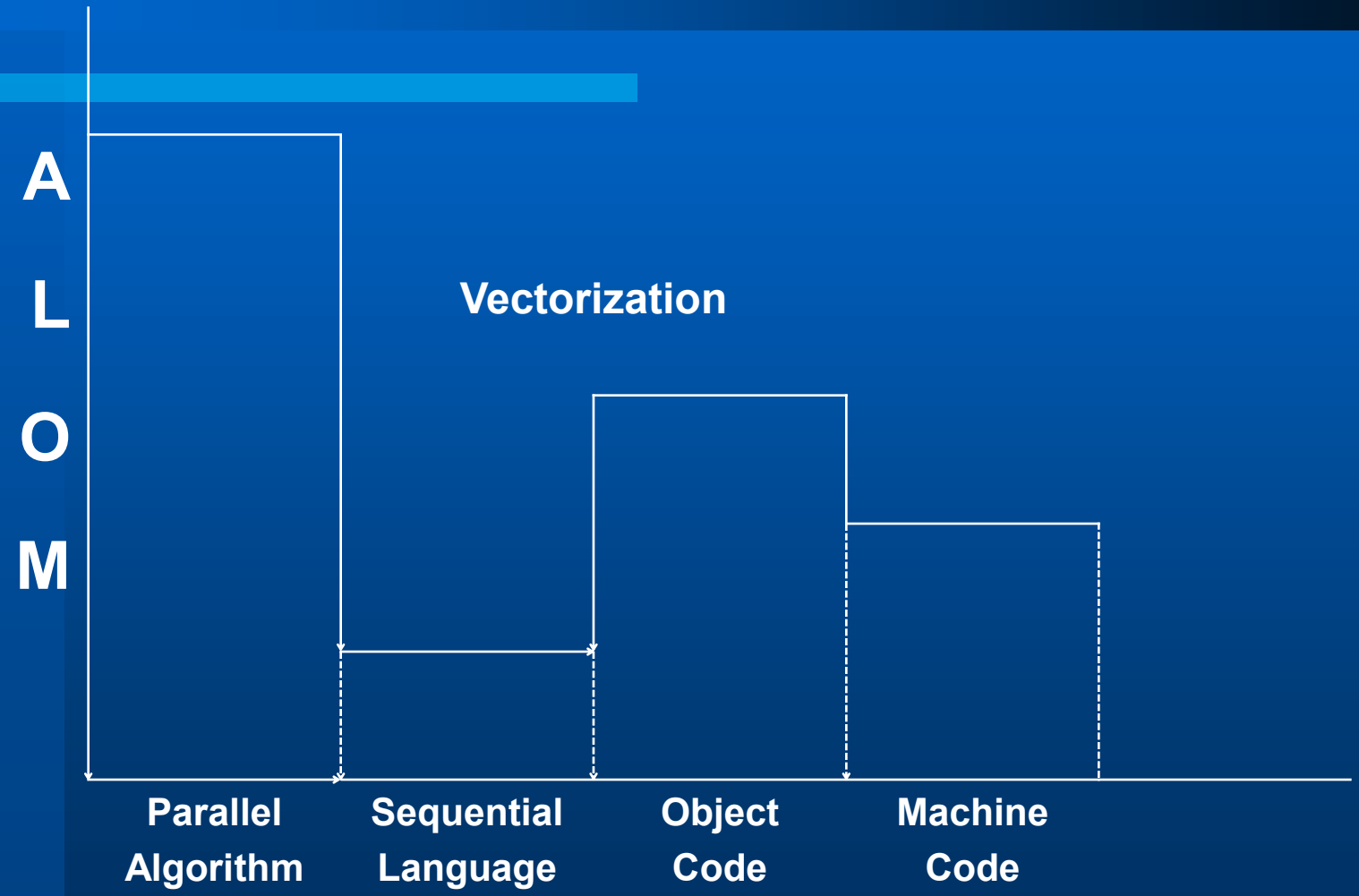
Characteristic of Vector Processing

- **Enhancing the vector-processing capability**
  - **Enrich the vector instruction set**
  - **Combine scalar instructions**
  - **Choose suitable algorithm**
  - **Use a vectorizing compiler**
    - **Developing of parallelism in advanced programming**

Vectorizing compiler

- Parallel Algorithm
- High-level language
- Efficient object code
- Target machine code


A
L
O
M
Parallel Algorithm
Parallel Language
Object Code
Machine Code


A
L
O
M
Vectorization
Parallel Algorithm
Sequential Language
Object Code
Machine Code

Multiple Vector Task Dispatching

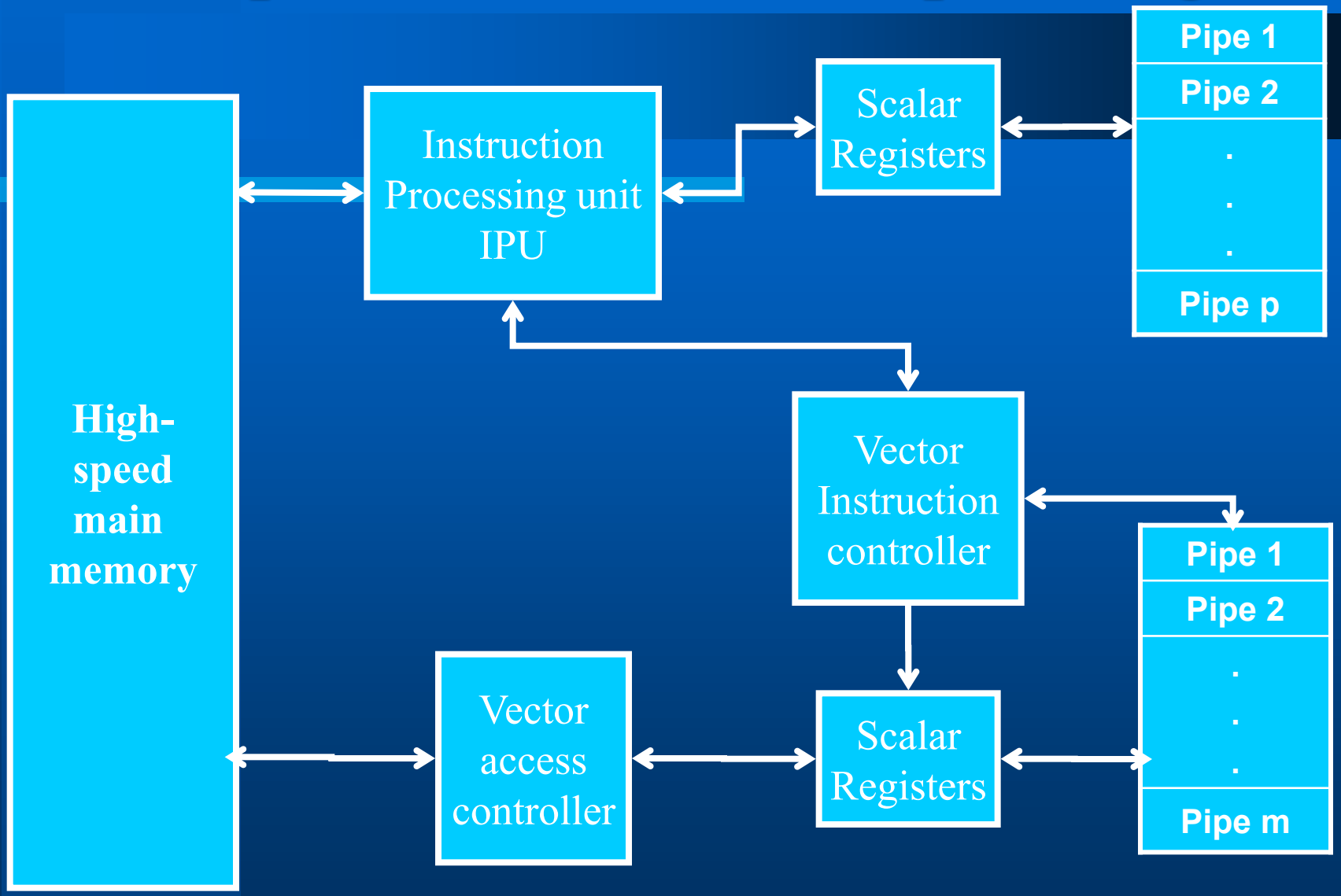


Fig: Architecture of a typical vector processor with multiple functional pipes

Vector Task System, V

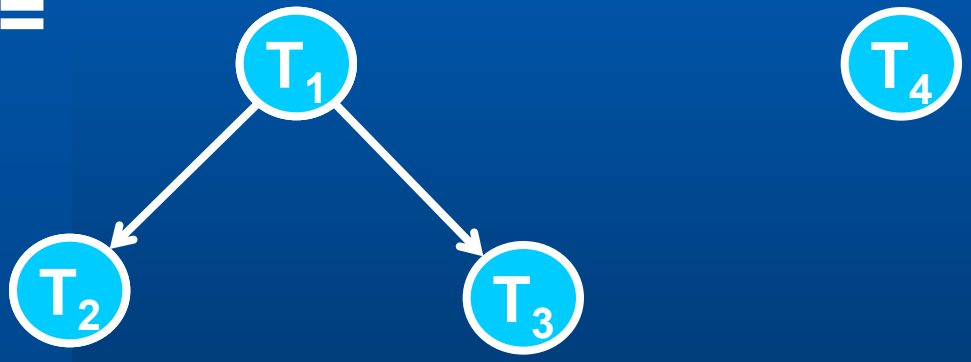
- $V = (T, <, \tau)$
- $T = \{ T_1, T_2, \ldots T_n \}$ is a set of n vector task
- $<$ is a partial ordering relation
- τ: $T \rightarrow R^+$ is a time function defining the production delay $\tau (T_i)$ for each T_i

Vector Task System, V

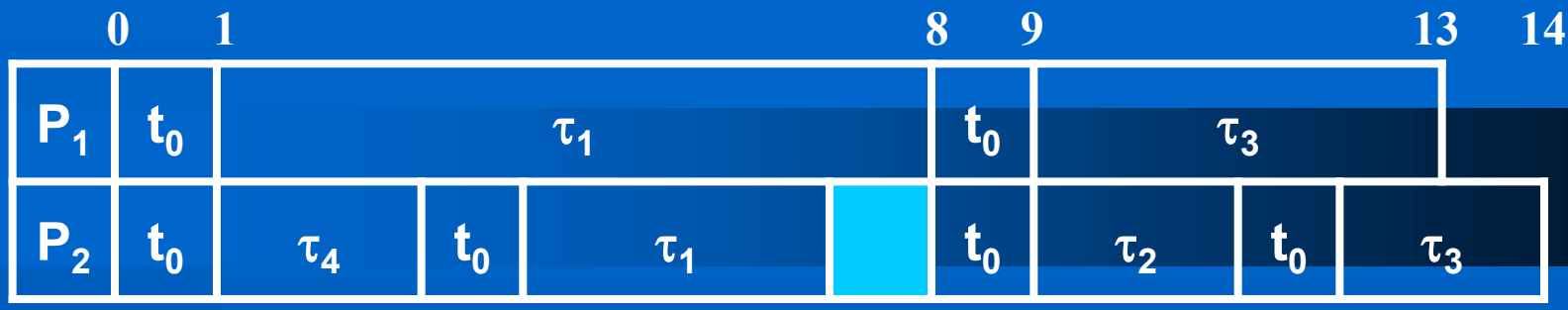
- $P = \{ P_1, P_2, \ldots P_m \}$ is a set of vector pipelines
- R^2 is the set of possible time interval
- Utilization of a pipeline P_i within interval [x, y] denoted by $P_i (x, y)$
- *Resource space* (set of all possible pipeline utilization patterns)
  - $P \times R^2 = \{ P_i (x, y) \mid P_i \in P \text{ and } (x, y) \in R^2 \}$
- Parallel schedule f for V is defined as
  - $f : T \rightarrow 2^{P \times R^2}$

Vector Task System

- $T = \{ T_1, T_2, T_3, T_4 \}$
- $\tau = \{10, 2, 6, 2\}$
- $<=$



- $P = \{ P_1, P_2 \}$



- $f(T_1) = \{P_1(0,8), P_2(3,7)\}$ with $S(T_1) = 0$ & $F(T_1) = 8$
- $f(T_2) = \{P_2(8,11)\}$ with $S(T_2) = 8$ & $F(T_2) = 11$
- $f(T_3) = \{P_1(8,13), P_2(11,14)\}$ with $S(T_3)=8$ & $F(T_3)=14$
- $f(T_4) = \{P_2(0,3)\}$ with $S(T_4) = 0$ & $F(T_4) = 3$

- $\omega \equiv MAX\{8, 11, 14, 3\} = 14$

Pipelined Vector Processing Methods

- **Horizontal processing**
  - **Vector computations are performed horizontally from left to right in row fashion**
- **Vertical processing**
  - **Vector computations are carried out vertically from top to bottom in column fashion**
- **Vector looping**
  - **Segmented vector loop computations are performed from left to right and top to bottom in a combined horizontal and vartical method**

Let

- $\{a_3$ for $1 \leq i \leq n\}$ be n scalar constant
- $\mathbf{x_j} = (x_{1j}, x_{2j}, \ldots, x_{mj})^T$ for j = 1, 2, …, n be n column vector
- $\mathbf{y} = (y_1, y_2, \ldots, y_m)\ )^T$ be a column vector for m components

- $\mathbf{y} = a_1.\mathbf{x_1}+a_2.\mathbf{x_2}+a_3.\mathbf{x_3}+\ldots.+a_n.\mathbf{x_n}$

- **Let $z_{ij} = a_j.x_{ij}$**

$y_1 = z_{11}+z_{12}+\ldots\ldots\ldots\ldots+z_{1n}$

$y_2 = z_{21}+z_{22}+\ldots\ldots\ldots\ldots+z_{2n}$

$\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots$

$\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots$

$y_m = z_{m1}+z_{12}+\ldots\ldots\ldots\ldots+z_{mn}$

- If each addition or multiplication requires $5t$ (t = clock period or one stage delay) then for nonpipeline scalar processor the total execution time,
  - $T_1 = [mn + m(n-1)] \times 5t$

    $= (10mn - 5m)t$

If we assume that all multiplication have already been carried out by the pipeline in T_m pipeline cycles

T_m = $5t$ (setup time) + $5t$ (time for the first product to comeout of the pipe) + $(mn - 1)t$ (time for producing remaining product)

$T_m = mnt + 9t$

Horizontal Vector Processing

- Each y_i are calculated in sequential order.
- $y_i=\sum z_{ij}$ involving n-1 additions must be completed before switching to the evaluation of next summation $y_{i+1}=\sum z_{i+1,j}$
- Each evaluation y_i requires $(n + 14)t$ clock period.
- The total add time for m output equals
  - $T_a(\text{horizontal})=(mn + 14m)t$

Horizontal Vector Processing

- **Speed up**

$$S_{horizontal} = \frac{T_i}{T_m + T_a(horizontal)}$$

$$\Rightarrow S_{horizontal} = \frac{(10mn - 5m)t}{(2mn + 14m + 9)t}$$

$$\Rightarrow S_{horizontal} = \frac{10mn - 5m}{2mn + 14m + 9}$$

Vartical Vector Processing

- Step-1:
  - Compute the partial sums $(z_{i1} + z_{i2}) = y^{i}_{12}$ [i=1,2,…,m]
- Step-2:
  - Compute the partial sums $y^{i}_{12} + z_{i3}$ for i=1,2,…,m
- Step-3 to n – 1:
  - Repeat step-2 for n-3 times by feeding successive columns $(z_{1j}, z_{2j}, \ldots, z_{mj})$ for j=4,5,…,n into the second input port. The values of y_j, for i=1,2,…,m emerge from the pipeline at the end of Step n – 1.
- The total add time for m output equals
  - $T_a(\text{vartical}) = (mn - m + 10)T$

Vartical Vector Processing

- **Speed up**

$$S_{vartical} = \frac{T_i}{T_m + T_a(vartical)}$$

$$\Rightarrow S_{vartical} = \frac{(10mn - 5m)t}{(2mn - m + 19)t}$$

$$\Rightarrow S_{vartical} = \frac{10mn - 5m}{2mn - m + 19}$$

Vector Looping Method

- Step-1:
  - Apply the vertical processing method to generate the 1st block of five outputs $y_1, y_2, \ldots, y_5$ in column fashion
- Step-2 tp step k:
  - Repeat step 1 for generating the remaining five output blocks as
    - Step-2: $y_6, y_7, \ldots, y_{10}$
    - Step-3: $y_{11}, y_{12}, \ldots, y_{15}$
    - ……
    - Step-k: $y_{5k-4}, y_{5k-3}, \ldots, y_{5k}$
- Step-k+1:
  - Repeat step-1 for generating the last block of r outputs, $y_{5k+1}, y_{5k+2}, \ldots$and y_{5k+r} where m = 5k + r [k=(m-r)/5] and 0 < r < 5
- The total add time for m output equals
  - $T_a(\text{vector looping}) = 5t + (5n - 1)t + (k - 1)[(5(n - 1)]t + 5t$

$$= mnt - mt - nrt + rt + 14t$$

Vector Looping Method

- **Speed up**

$$S_{vector-looping} = \frac{T_i}{T_m + T_a(vectorlooping)}$$

$$\Rightarrow S_{vector-looping} = \frac{10mn - 5m}{2mn - m + rn + r + 23}$$

Vactorization & Optimization Methods

Guidelines to design a Vectorizer

- Determining the flow pattern between subprograms
- Checking the precedence relationship among the subprograms
- Checking the locality of variables
- Determining the loop variables
- Checking the independence of variables
- Replacing the inner loop with vector instructions

Conversion of a DO-loop

```
DO 20 I = 8, 120, 2
20 A(I) = B(I+3) + C(I+1)

A(8:120:2) = B(11:123:2) + C(9:121:2)
```

Recurrence computation

```
A(0) = X
DO 20 I = 0, N, 1
20 A(I) = A(I – 1) * B(I) + C(I + 1)
```

```
A(1 : N) = A(0 : N-1 : 1) * B(1 : N) + C(2 : N +
   1)
```

Replacing IF statement by WHERE

```
DO 20 I = 1, N
20 IF (L(I), NE, 0) A(I) = A(I) – 1
```

```
WHERE (L(I), NE, 0) A(1 : N) = A(1 : N) – 1
```

Exchanging execution sequence

```
DO 20 I = 1, N
A(I) = B(I – 1)
20 B(I) = 2 * B(I)
```

```
B(1 : N) = 2 * B(1 : N)
A(1 : N) = B(0 : N – 1)
```

Using temporary storage

```
DO 20 I = 1, N
A(I) = B(I) + C(I)
20 B(I) = 2 * A(I + 1)
```

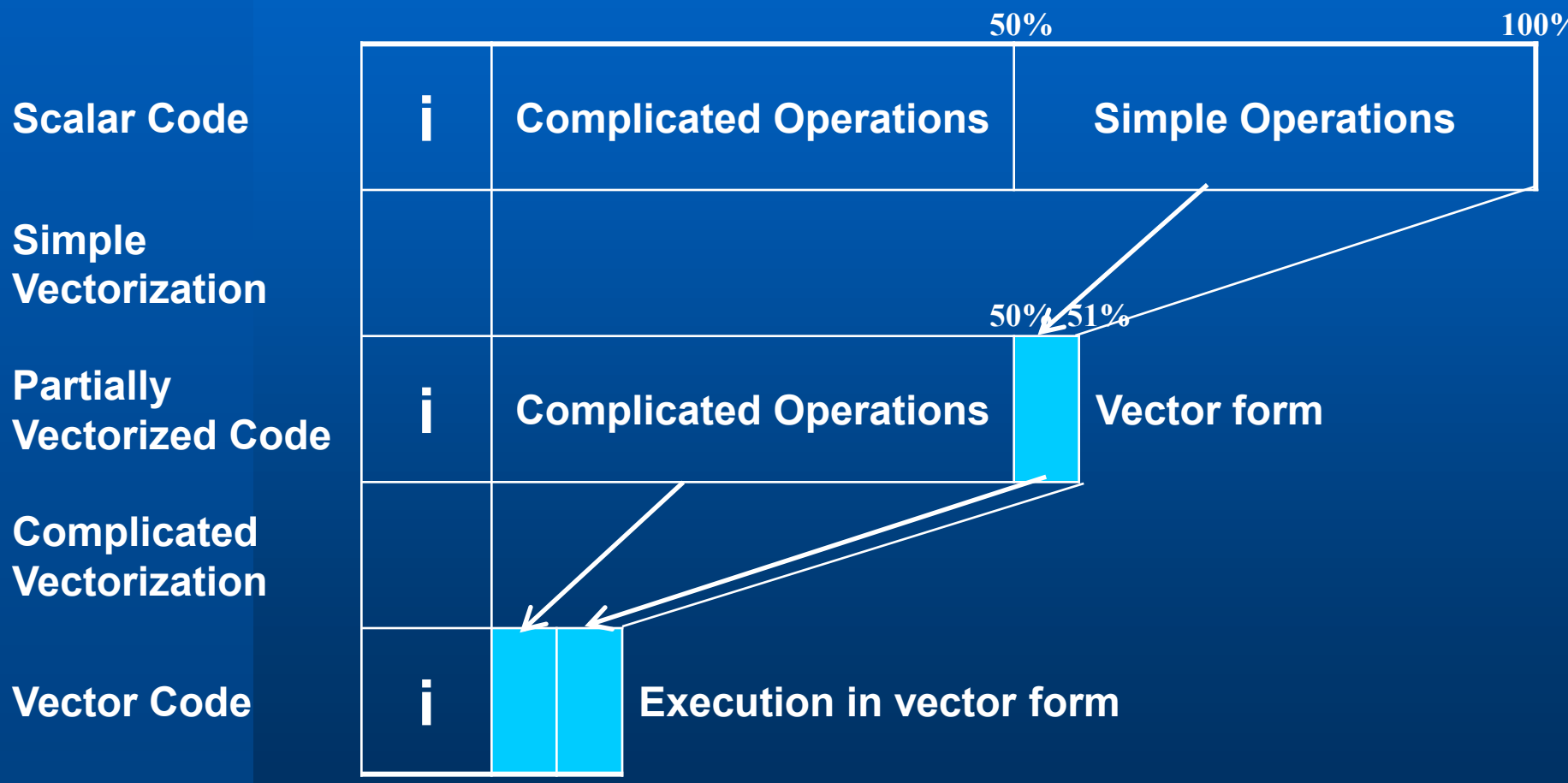
```
TEMP(1 : N) = A(2 : N+1)
A(1 : N) = B(1 : N) + C(1 : N)
B(1 : N) = 2 * TEMP(1 :N)
```

Two levels of array computation

```
DO 20 I = 1, 80
DO 20 J = 1, 80
20 A(I, J) = C(I, J) + C(I, J)


DO 20 J = 1, 80
DO 20 I = 1, 80
20 A(I, J) = C(I, J) + C(I, J)
```

Vectorization ration and saving in execution time



Major optimizing functions

- General optimization
  - Common expression elimination
  - Invariant expression movement
  - Strength reduction
  - Register optimization
  - Constant folding

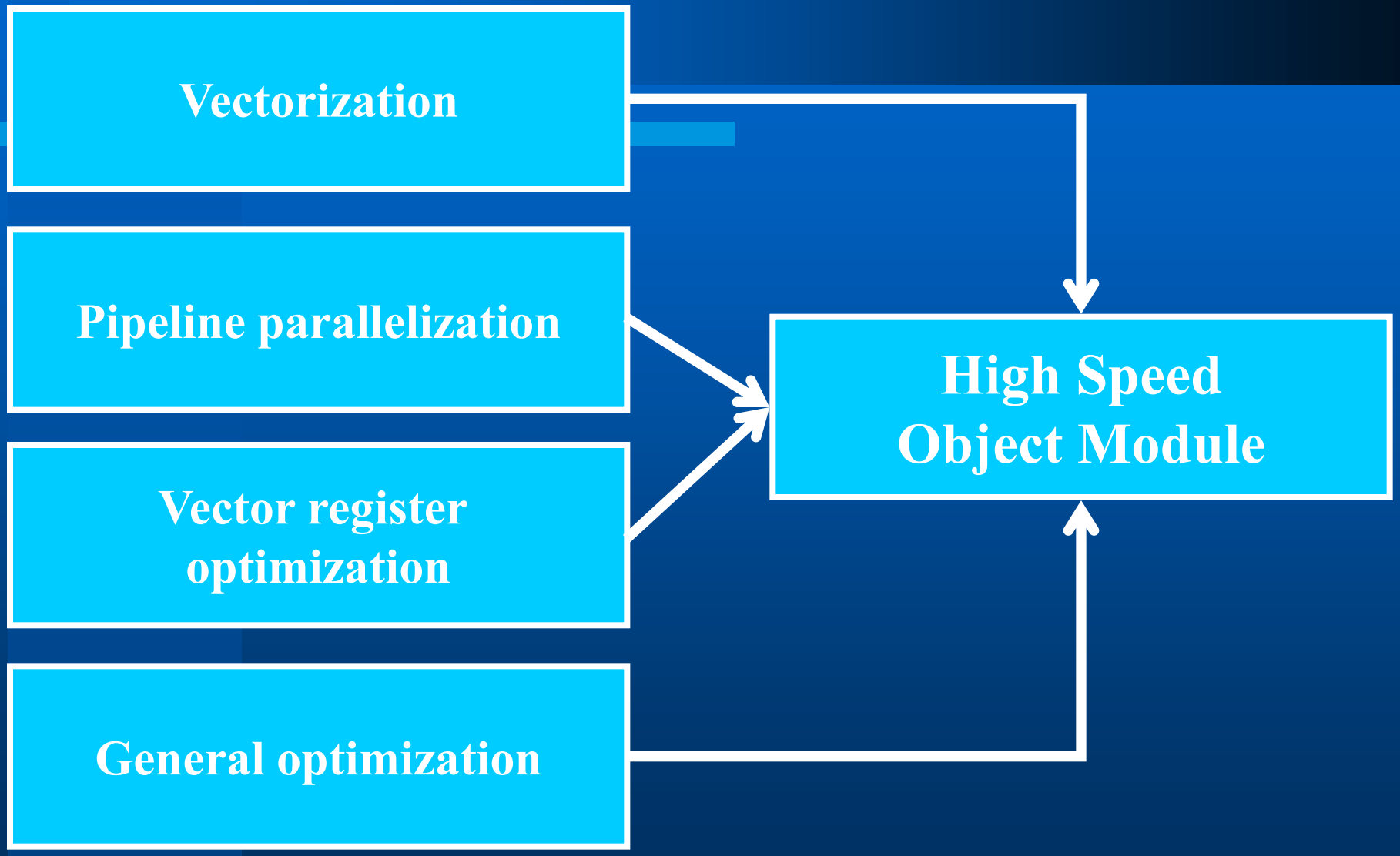
Major optimizing functions

- **Extended optimization**
  - **Intrinsic function integration. i.e. SQRT, SINE, etc.**
  - **User function subprogram integration**
  - **Reduction of iteration numbers in nested DO loops**
  - **Reorder of execution sequence to reduce pipeline overhead**
  - **Temporal storage management**
  - **Code avoidance**

Major optimizing functions

- **Vector Extended optimization**
  - **Full optimization**
  - **Pipeline chaining**
  - **Pipeline anti-chaining**
  - **Vector register optimization**
  - **Parallelization**

Vectorizing compiler design techniques



Optimization of Vector Function

- **Redundant Expression Elimination**

```
DIMENSION A(100), B(50), C(50)
C(1 : 50) = A(1 : 99 : 2) * B(1 : 50) + A(1 : 99 : 2) * C(1 : 50)
```

```
V1 ← A(1 : 99 : 2)
V2 ← B(1 : 50)
V3 ← V1 * V2
V4 ← A(1 : 99 : 2)
V5 ← C(1 : 50)
V6 ← V4 * V5
V7 ← V3 + V6
C(1 : 50) ← V7
```

If B array and C array are identical

```
V1 ← A(1 : 99 : 2)
V2 ← B(1 : 50)
V1 ← V1 * V2
V1 ← 2 * V1
C(1 : 50) ← V1
```

Optimization of Vector Function

- **Constant folding at compile time**

```
DO 20 I = 1, 100
A(I) = I
```

Can be replaced by constant vector (1, 2, … , 100) initialized in the compile time

Optimization of Vector Function

- **Invariant expression movement**

```
DO 20 I = 1, N
……
DO 20 J = 1, M
……
B(I, J) = B(I, J) + A(J) * C(J)
20 CONTINUE
```

```
DO 20 I = 1, N
……
B(I, *) = B(I, *) + A(*) * C(*)
DO 20 J = 1, M
……
20 CONTINUE
```

Optimization of Vector Function

- **Invariant expression movement**

```
DO 20 I = 1, N
……
B(I, *) = B(I, *) + A(*) * C(*)
DO 20 J = 1, M
……
20 CONTINUE
```

```
T(*) = A(*) * C(*)
DO 20 I = 1, N
……
B(I, *) = B(I, *) + T(*)
DO 20 J = 1, M
……
20 CONTINUE
```

Optimization of Vector Function

- **Invariant expression movement**

```
DO 20 I = 1, N
……
B(I, *) = B(I, *) + A(*) * C(*)
DO 20 J = 1, M
……
20 CONTINUE
```

```
T(*) = A(*) * C(*)
DO 20 I = 1, N
……
B(I, *) = B(I, *) + T(*)
DO 20 J = 1, M
……
20 CONTINUE
```

Optimization of Vector Function

- **Pipeline chaining and parallelization**

```
DO 20 I = 1, N
20  A(I) = B(I) * C(I) + D(I) * E(I) + F(I) * G(I)
```

Optimization of Vector Function

- **Vector Register Allocation**

V1 ← A
V2 ← B
V3 ← C
V2 ← V2 * V3
V1 ← V1 – V2

V1 ← B
V2 ← C
V1 ← V1 * V2
V2 ← A
V2 ← V2 – V1

Optimization of Vector Function

- **Reorder the execution sequence**

A1 ← A2 + A3
A4 ← A5 * A6
B1 ← A7 + A8
B2 ← B3 * B4
B5 ← C1 + C2

A1 ← A2 + A3
B1 ← A7 + A8
B5 ← C1 + C2
A4 ← A5 * A6
B2 ← B3 * B4

Optimization of Vector Function

- **Temporary storage management**

```
DO 20 K = 1, 4000, 64
L = MIN(K +63, 4000)
A(K, L) = A(K, L) * B(K, L) + C(K, L)
20 CONTINUE
```

Optimization of Vector Function

- **Code Avoidance**

```
A(1:50) = B(1:99:2)
.....
.....
C(1:50) = 2.0 * A(1:50)
```

```
V1 ← B(1:99:2)
A(1:50) ← 2 * V1
C(1:50) ← V1
.....
V1 ← A(1:50)
V1 ← 2 * V1
C(1:50) ← V1
```

Optimization of Vector Function

- **Tuning for interactive vectorization**

```
DO 20 I = 1, N
    IF(A(I), GE, 10) GO TO 20
    T = C1 * A(I) + C2 * B(I)
    A(I) = SIN(T)
20 CONTINUE
```

Distributed Processing

Introduction

- *Distributed processing: ~ describe system with multiple processors*
- *Distributed system: A distributed system is a collection of independent computers that appear to the users of the system as a single system.*
- *Example: Network of workstations, Network of branch office computers*

Local vs Remote Distribution

- *Interconnected by bus to form a single computer complex*
- *Interconnected by cables in a machine room*
- *Interconnected by in-plant wiring*
- *Interconnected by permanent common carrier telecommunications*
- *Interconnected by intermittent common carrier telecommunications*
- *Not physically connected*

Some Definitions

- *Software path length: The number of software instructions executed for a transaction*
- *Teleprocessing: Use of telecommunication facilities for accessing processing power*

Categories of Data Distribution

- *Central data, single host*
- *Central data, multiple host*
- *Dependent hierarchical data*
- *Independent hierarchical data*
- *Split data*
- *Separate data*
- *Replicated data*
- *Heterogeneous data system*
- ***Combinations***

Criteria for dividing data within a Distributed system

- *Geography*
- *Type of data*
- *Type of usage*

Factors, causing the trend to Distributed Processing

- *Low cost of processor*
- *Grosch's low overruled*
- *Increasing sophistication of end users*
- *End-user dissatisfaction*
- *Need for higher availability*
- *End-user imagination*

Factors, causing the trend to Distributed Processing (Cont.)

- *High cost of phone lines*
- *Low bandwidth of phone lines*
- *Batter dialogue*
- *Central control of remote development*
- *System interlinking*
- *Remote database access*

Factors, causing the trend to Distributed Processing (Cont.)

- *Networking software*
- *Security*
- *Excessive CPU load*
- *Programmer shortage*
- *Centralization and decentralization*

Types of Distributed Systems

- *Horizontal vs Vertical Distribution*
- *Function Distribution vs System Distribution*

Reasons for Function Dist.

- *Fall into three categories*
    1. *Reasons associated with the host*
    2. *Reasons associated with the network*
    3. *Reasons associated with the end user*

Reasons for Function Dist.

1. *Psychologically effective dialogues*
    - *Local interaction*
    - *Local panel storage*
    - *Speed*

Reasons for Function Dist.

2. *Reduction of telecommunications costs*
    - *Reduction of number of messages*
    - *Reduction of message size*
    - *Reduction of number of line turnarounds*
    - *Bulk transmission*
    - *Data compaction*
    - *Minimum cost routing*
    - *Controlled network access*

Reasons for Function Dist.

3. *Reliability*
    - *Local autonomy*
    - *Automatic dial backup*
    - *Automatic alternate routing*
    - *Control procedures*
    - *Automatic load balancing*

Reasons for Function Dist.

4. *Less load on host*
    - *Parallel operations*
    - *Permits large number of terminals*
5. *Fast response time*
    - *Process mechanisms*
    - *Human mechanisms*
    - *Dialogue response time*

Reasons for Function Dist.

6. *Data collection*
    - *Data entry terminals*
    - *Local error checking*
    - *Instrumentation*
7. *More attractive output*
    - *Local editing*
8. *Peaks*

Reasons for Function Dist.

9. *Security*
    - *Cryptography*
    - *Access Control*
10. *Network independence*
    - *Network transparency*
    - *Network evolution*
    - *New networks*

Reasons for Function Dist.

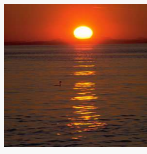
11. *Terminal independency*
    - *New terminals*
    - *Virtual terminal features*



Chapter 22: Distributed Databases

Database System Concepts, 5th Ed.

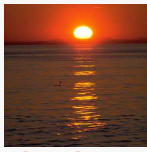
©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use





Chapter 22: Distributed Databases

- Heterogeneous and Homogeneous Databases
- Distributed Data Storage
- Distributed Transactions
- Commit Protocols
- Concurrency Control in Distributed Databases
- Availability
- Distributed Query Processing
- Heterogeneous Distributed Databases
- Directory Systems

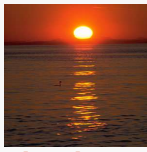


©Silberschatz, Korth and Sudarshan



Distributed Database System

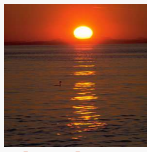
- A distributed database system consists of loosely coupled sites that share no physical component
- Database systems that run on each site are independent of each other
- Transactions may access data at one or more sites





Homogeneous Distributed Databases

- In a homogeneous distributed database
  - All sites have identical software
  - Are aware of each other and agree to cooperate in processing user requests.
  - Each site surrenders part of its autonomy in terms of right to change schemas or software
  - Appears to user as a single system
- In a heterogeneous distributed database
  - Different sites may use different schemas and software
    - Difference in schema is a major problem for query processing
    - Difference in software is a major problem for transaction processing
  - Sites may not be aware of each other and may provide only limited facilities for cooperation in transaction processing

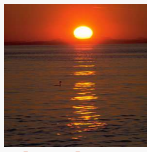


©Silberschatz, Korth and Sudarshan



Distributed Data Storage

- Assume relational data model
- Replication
  - System maintains multiple copies of data, stored in different sites, for faster retrieval and fault tolerance.
- Fragmentation
  - Relation is partitioned into several fragments stored in distinct sites
- Replication and fragmentation can be combined
  - Relation is partitioned into several fragments: system maintains several identical replicas of each such fragment.

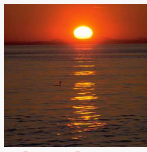


©Silberschatz, Korth and Sudarshan



Data Replication

- A relation or fragment of a relation is **replicated** if it is stored redundantly in two or more sites.
- Full replication of a relation is the case where the relation is stored at all sites.
- Fully redundant databases are those in which every site contains a copy of the entire database.

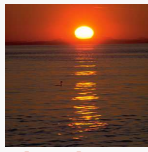


©Silberschatz, Korth and Sudarshan



Data Replication (Cont.)

- Advantages of Replication
  - **Availability**: failure of site containing relation *r* does not result in unavailability of *r* is replicas exist.
  - **Parallelism**: queries on *r* may be processed by several nodes in parallel.
  - **Reduced data transfer**: relation *r* is available locally at each site containing a replica of *r*.
- Disadvantages of Replication
  - Increased cost of updates: each replica of relation *r* must be updated.
  - Increased complexity of concurrency control: concurrent updates to distinct replicas may lead to inconsistent data unless special concurrency control mechanisms are implemented.
    - One solution: choose one copy as **primary copy** and apply concurrency control operations on primary copy

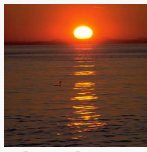


©Silberschatz, Korth and Sudarshan



Data Fragmentation

- Division of relation r into fragments r_1, r_2, …, r_n which contain sufficient information to reconstruct relation r.
- **Horizontal fragmentation**: each tuple of *r* is assigned to one or more fragments
- **Vertical fragmentation**: the schema for relation *r* is split into several smaller schemas
  - All schemas must contain a common candidate key (or superkey) to ensure lossless join property.
  - A special attribute, the tuple-id attribute may be added to each schema to serve as a candidate key.
- Example : relation account with following schema
- *Account* = (*branch_name*, *account_number, balance*)



©Silberschatz, Korth and Sudarshan



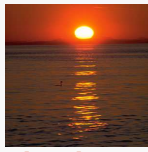
Horizontal Fragmentation of *account* Relation

branch_name	*account_number*	*balance*
Hillside	A-305	500
Hillside	A-226	336
Hillside	A-155	62

$$account_1 = \sigma_{branch_name = \text{“Hillside”}}(account)$$

branch_name	*account_number*	*balance*
Valleyview	A-177	205
Valleyview	A-402	10000
Valleyview	A-408	1123
Valleyview	A-639	750

$$account_2 = \sigma_{branch_name = \text{“Valleyview”}}(account)$$



Vertical Fragmentation of *employee_info* Relation

branch_name	*customer_name*	*tuple_id*
Hillside	Lowman	1
Hillside	Camp	2
Valleyview	Camp	3
Valleyview	Kahn	4
Hillside	Kahn	5
Valleyview	Kahn	6
Valleyview	Green	7

$deposit_1 = \Pi_{branch_name,\ customer_name,\ tuple_id}(employee_info)$

account_number	*balance*	*tuple_id*
A-305	500	1
A-226	336	2
A-177	205	3
A-402	10000	4
A-155	62	5
A-408	1123	6
A-639	750	7

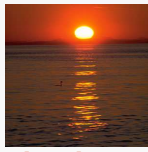
$deposit_2 = \Pi_{account_number,\ balance,\ tuple_id}(employee_info)$

©Silberschatz, Korth and Sudarshan



Advantages of Fragmentation

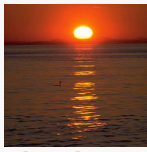
- Horizontal:
  - allows parallel processing on fragments of a relation
  - allows a relation to be split so that tuples are located where they are most frequently accessed
- Vertical:
  - allows tuples to be split so that each part of the tuple is stored where it is most frequently accessed
  - tuple-id attribute allows efficient joining of vertical fragments
  - allows parallel processing on a relation
- Vertical and horizontal fragmentation can be mixed.
  - Fragments may be successively fragmented to an arbitrary depth.





Data Transparency

- **Data transparency**: Degree to which system user may remain unaware of the details of how and where the data items are stored in a distributed system
- Consider transparency issues in relation to:
  - Fragmentation transparency
  - Replication transparency
  - Location transparency

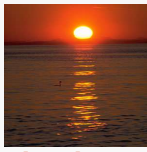


©Silberschatz, Korth and Sudarshan



Naming of Data Items - Criteria

1. Every data item must have a system-wide unique name.
2. It should be possible to find the location of data items efficiently.
3. It should be possible to change the location of data items transparently.
4. Each site should be able to create new data items autonomously.

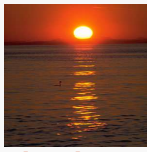


©Silberschatz, Korth and Sudarshan



Centralized Scheme - Name Server

- Structure:
  - name server assigns all names
  - each site maintains a record of local data items
  - sites ask name server to locate non-local data items
- Advantages:
  - satisfies naming criteria 1-3
- Disadvantages:
  - does not satisfy naming criterion 4
  - name server is a potential performance bottleneck
  - name server is a single point of failure

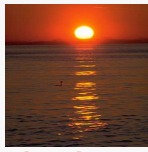


©Silberschatz, Korth and Sudarshan



Use of Aliases

- Alternative to centralized scheme: each site prefixes its own site identifier to any name that it generates i.e., *site* 17.a*ccount.*
  - Fulfills having a unique identifier, and avoids problems associated with central control.
  - However, fails to achieve network transparency.
- Solution: Create a set of **aliases** for data items; Store the mapping of aliases to the real names at each site.
- The user can be unaware of the physical location of a data item, and is unaffected if the data item is moved from one site to another.

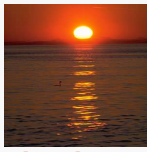


©Silberschatz, Korth and Sudarshan



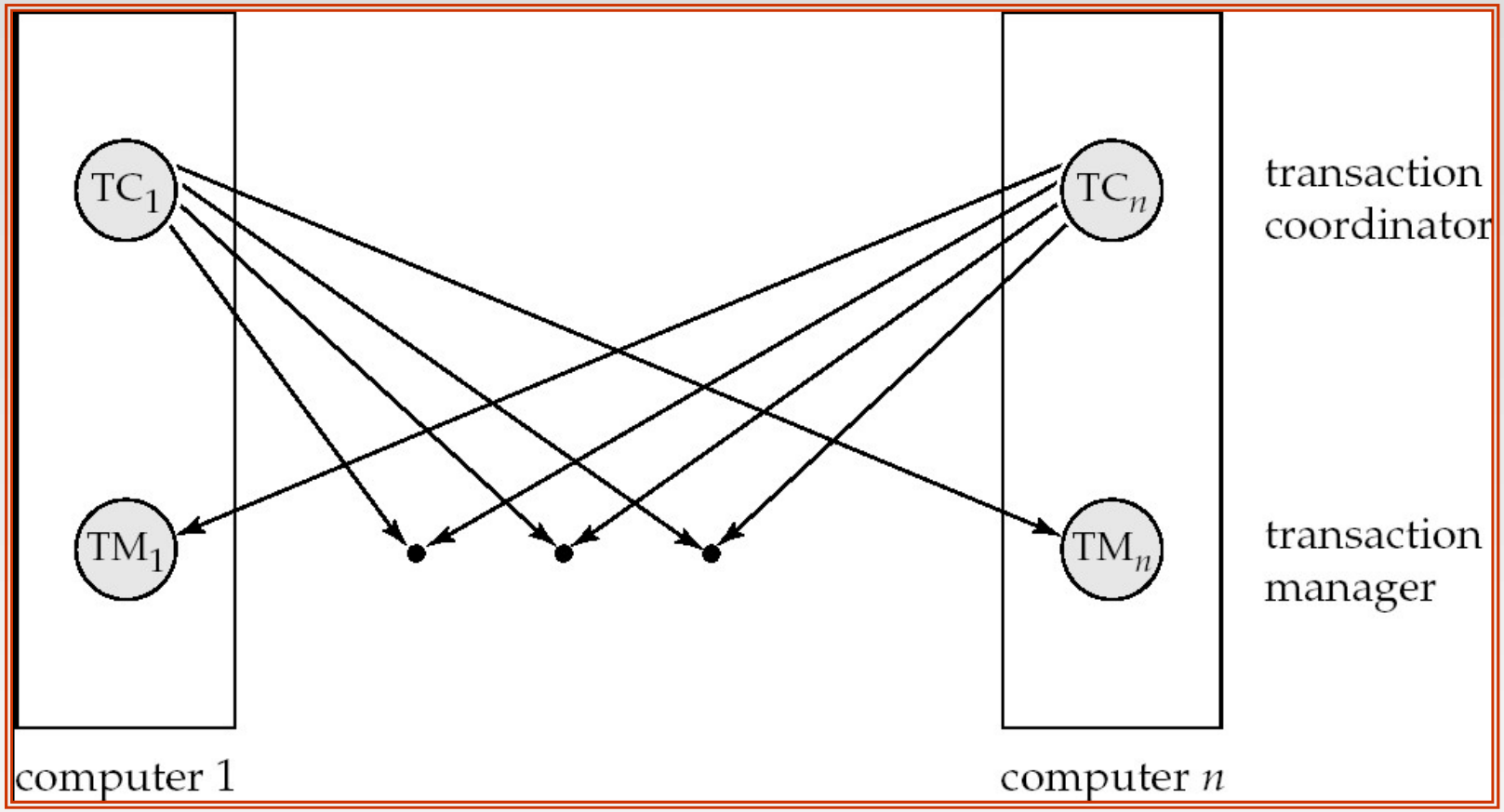
Distributed Transactions

- Transaction may access data at several sites.
- Each site has a local transaction manager responsible for:
  - Maintaining a log for recovery purposes
  - Participating in coordinating the concurrent execution of the transactions executing at that site.
- Each site has a transaction coordinator, which is responsible for:
  - Starting the execution of transactions that originate at the site.
  - Distributing subtransactions at appropriate sites for execution.
  - Coordinating the termination of each transaction that originates at the site, which may result in the transaction being committed at all sites or aborted at all sites.



©Silberschatz, Korth and Sudarshan

Transaction System Architecture

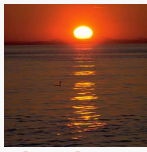


 ©Silberschatz, Korth and Sudarshan



System Failure Modes

- Failures unique to distributed systems:
  - Failure of a site.
  - Loss of massages
    - Handled by network transmission control protocols such as TCP-IP
  - Failure of a communication link
    - Handled by network protocols, by routing messages via alternative links
  - **Network partition**
    - A network is said to be **partitioned** when it has been split into two or more subsystems that lack any connection between them
      - Note: a subsystem may consist of a single node
- Network partitioning and site failures are generally indistinguishable.

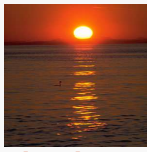


©Silberschatz, Korth and Sudarshan



Commit Protocols

- Commit protocols are used to ensure atomicity across sites
  - a transaction which executes at multiple sites must either be committed at all the sites, or aborted at all the sites.
  - not acceptable to have a transaction committed at one site and aborted at another
- The *two-phase commit* (2PC) protocol is widely used
- The *three-phase commit* (3PC) protocol is more complicated and more expensive, but avoids some drawbacks of two-phase commit protocol. This protocol is not used in practice.

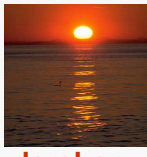


©Silberschatz, Korth and Sudarshan



Two Phase Commit Protocol (2PC)

- Assumes **fail-stop** model – failed sites simply stop working, and do not cause any other harm, such as sending incorrect messages to other sites.
- Execution of the protocol is initiated by the coordinator after the last step of the transaction has been reached.
- The protocol involves all the local sites at which the transaction executed
- Let T be a transaction initiated at site S_i, and let the transaction coordinator at S_i be C_i

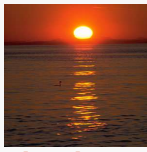


©Silberschatz, Korth and Sudarshan



Phase 1: Obtaining a Decision

- Coordinator asks all participants to *prepare* to commit transaction T_i.
  - C_i adds the records <**prepare** *T*> to the log and forces log to stable storage
  - sends **prepare** *T* messages to all sites at which *T* executed
- Upon receiving message, transaction manager at site determines if it can commit the transaction
  - if not, add a record <**no** *T*> to the log and send **abort** *T* message to C_i
  - if the transaction can be committed, then:
  - add the record <**ready** *T*> to the log
  - force *all records* for *T* to stable storage
  - send **ready** *T* message to C_i

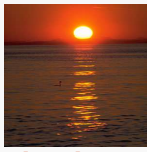


©Silberschatz, Korth and Sudarshan



Phase 2: Recording the Decision

- *T* can be committed of C_i received a **ready** *T* message from all the participating sites: otherwise *T* must be aborted.
- Coordinator adds a decision record, <**commit** *T*> or <a**bort** *T*>, to the log and forces record onto stable storage. Once the record stable storage it is irrevocable (even if failures occur)
- Coordinator sends a message to each participant informing it of the decision (commit or abort)
- Participants take appropriate action locally.



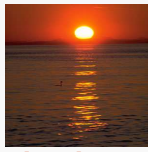
©Silberschatz, Korth and Sudarshan



Handling of Failures - Site Failure

When site S_i recovers, it examines its log to determine the fate of transactions active at the time of the failure.

- Log contain <**commit** *T*> record: site executes **redo** (*T*)
- Log contains <**abort** *T*> record: site executes **undo** (*T*)
- Log contains <**ready** *T*> record: site must consult C_i to determine the fate of *T*.
  - If *T* committed, **redo** (*T*)
  - If *T* aborted, **undo** (*T*)
- The log contains no control records concerning *T* replies that S_k failed before responding to the **prepare** *T* message from C_i
  - since the failure of S_k precludes the sending of such a response C_1 must abort *T*
  - S_k must execute **undo** (*T*)

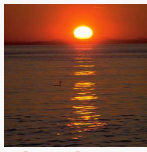


©Silberschatz, Korth and Sudarshan



Handling of Failures- Coordinator Failure

- If coordinator fails while the commit protocol for *T* is executing then participating sites must decide on *T*'s fate:
  1. If an active site contains a <**commit** *T*> record in its log, then *T* must be committed.
  2. If an active site contains an <**abort** *T*> record in its log, then *T* must be aborted.
  3. If some active participating site does not contain a <**ready** *T*> record in its log, then the failed coordinator C_i cannot have decided to commit *T*. Can therefore abort *T*.
  4. If none of the above cases holds, then all active sites must have a <**ready** *T*> record in their logs, but no additional control records (such as <**abort** *T*> of <**commit** *T*>). In this case active sites must wait for C_i to recover, to find decision.
- **Blocking problem** : active sites may have to wait for failed coordinator to recover.

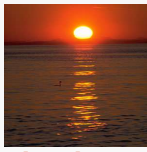


©Silberschatz, Korth and Sudarshan



Handling of Failures - Network Partition

- If the coordinator and all its participants remain in one partition, the failure has no effect on the commit protocol.
- If the coordinator and its participants belong to several partitions:
  - Sites that are not in the partition containing the coordinator think the coordinator has failed, and execute the protocol to deal with failure of the coordinator.
    - No harm results, but sites may still have to wait for decision from coordinator.
- The coordinator and the sites are in the same partition as the coordinator think that the sites in the other partition have failed, and follow the usual commit protocol.
    - Again, no harm results

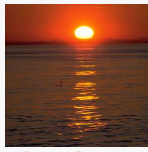


©Silberschatz, Korth and Sudarshan



Recovery and Concurrency Control

- **In-doubt transactions** have a <**ready** *T*>, but neither a <**commit** *T*>, nor an <**abort** *T*> log record.
- The recovering site must determine the commit-abort status of such transactions by contacting other sites; this can slow and potentially block recovery.
- Recovery algorithms can note lock information in the log.
  - Instead of <**ready** *T*>, write out <**ready** *T*, *L*> *L* = list of locks held by *T* when the log is written (read locks can be omitted).
  - For every in-doubt transaction *T*, all the locks noted in the <**ready** *T*, *L*> log record are reacquired.
- After lock reacquisition, transaction processing can resume; the commit or rollback of in-doubt transactions is performed concurrently with the execution of new transactions.

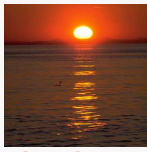


©Silberschatz, Korth and Sudarshan



Alternative Models of Transaction Processing

- Notion of a single transaction spanning multiple sites is inappropriate for many applications
  - E.g. transaction crossing an organizational boundary
  - No organization would like to permit an externally initiated transaction to block local transactions for an indeterminate period
- Alternative models carry out transactions by sending messages
  - Code to handle messages must be carefully designed to ensure atomicity and durability properties for updates
    - Isolation cannot be guaranteed, in that intermediate stages are visible, but code must ensure no inconsistent states result due to concurrency
  - Persistent messaging systems are systems that provide transactional properties to messages
    - Messages are guaranteed to be delivered exactly once
    - Will discuss implementation techniques later

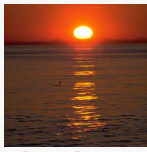


©Silberschatz, Korth and Sudarshan



Alternative Models (Cont.)

- Motivating example: funds transfer between two banks
  - Two phase commit would have the potential to block updates on the accounts involved in funds transfer
  - Alternative solution:
    - Debit money from source account and send a message to other site
    - Site receives message and credits destination account
  - Messaging has long been used for distributed transactions (even before computers were invented!)
- Atomicity issue
  - once transaction sending a message is committed, message must guaranteed to be delivered
    - Guarantee as long as destination site is up and reachable, code to handle undeliverable messages must also be available
      - e.g. credit money back to source account.
  - If sending transaction aborts, message must not be sent

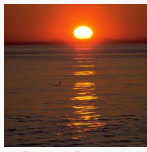


©Silberschatz, Korth and Sudarshan



Error Conditions with Persistent Messaging

- Code to handle messages has to take care of variety of failure situations (even assuming guaranteed message delivery)
  - E.g. if destination account does not exist, failure message must be sent back to source site
  - When failure message is received from destination site, or destination site itself does not exist, money must be deposited back in source account
    - Problem if source account has been closed
      - get humans to take care of problem
- User code executing transaction processing using 2PC does not have to deal with such failures
- There are many situations where extra effort of error handling is worth the benefit of absence of blocking
  - E.g. pretty much all transactions across organizations



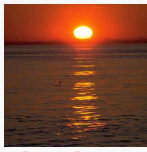
©Silberschatz, Korth and Sudarshan



Persistent Messaging and Workflows

- **Workflows** provide a general model of transactional processing involving multiple sites and possibly human processing of certain steps
  - E.g. when a bank receives a loan application, it may need to
    - Contact external credit-checking agencies
    - Get approvals of one or more managers

    and then respond to the loan application
  - We study workflows in Chapter 25
  - Persistent messaging forms the underlying infrastructure for workflows in a distributed environment

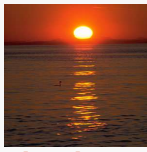


©Silberschatz, Korth and Sudarshan



Concurrency Control

- Modify concurrency control schemes for use in distributed environment.
- We assume that each site participates in the execution of a commit protocol to ensure global transaction automicity.
- We assume all replicas of any item are updated
  - Will see how to relax this in case of site failures later

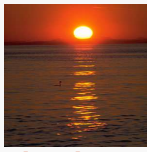


©Silberschatz, Korth and Sudarshan



Single-Lock-Manager Approach

- System maintains a *single* lock manager that resides in a *single* chosen site, say S_i
- When a transaction needs to lock a data item, it sends a lock request to S_i and lock manager determines whether the lock can be granted immediately
  - If yes, lock manager sends a message to the site which initiated the request
  - If no, request is delayed until it can be granted, at which time a message is sent to the initiating site

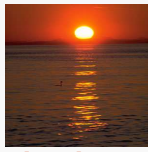


©Silberschatz, Korth and Sudarshan



Single-Lock-Manager Approach (Cont.)

- The transaction can read the data item from *any* one of the sites at which a replica of the data item resides.
- Writes must be performed on all replicas of a data item
- Advantages of scheme:
  - Simple implementation
  - Simple deadlock handling
- Disadvantages of scheme are:
  - Bottleneck: lock manager site becomes a bottleneck
  - Vulnerability: system is vulnerable to lock manager site failure.

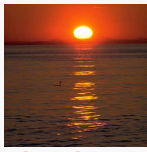


©Silberschatz, Korth and Sudarshan



Distributed Lock Manager

- In this approach, functionality of locking is implemented by lock managers at each site
  - Lock managers control access to local data items
    - But special protocols may be used for replicas
- Advantage: work is distributed and can be made robust to failures
- Disadvantage: deadlock detection is more complicated
  - Lock managers cooperate for deadlock detection
    - More on this later
- Several variants of this approach
  - Primary copy
  - Majority protocol
  - Biased protocol
  - Quorum consensus

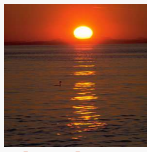


©Silberschatz, Korth and Sudarshan



Primary Copy

- Choose one replica of data item to be the **primary copy**.
  - Site containing the replica is called the **primary site** for that data item
  - Different data items can have different primary sites
- When a transaction needs to lock a data item *Q*, it requests a lock at the primary site of *Q*.
  - Implicitly gets lock on all replicas of the data item
- Benefit
  - Concurrency control for replicated data handled similarly to unreplicated data - simple implementation.
- Drawback
  - If the primary site of *Q* fails, *Q* is inaccessible even though other sites containing a replica may be accessible.

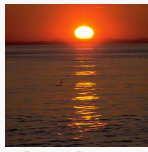


©Silberschatz, Korth and Sudarshan



Majority Protocol

- Local lock manager at each site administers lock and unlock requests for data items stored at that site.
- When a transaction wishes to lock an unreplicated data item *Q* residing at site S_i, a message is sent to S_i ‘s lock manager.
  - If *Q* is locked in an incompatible mode, then the request is delayed until it can be granted.
  - When the lock request can be granted, the lock manager sends a message back to the initiator indicating that the lock request has been granted.

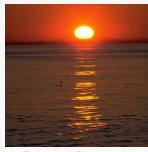


©Silberschatz, Korth and Sudarshan



Majority Protocol (Cont.)

- In case of replicated data
  - If *Q* is replicated at n sites, then a lock request message must be sent to more than half of the n sites in which *Q* is stored.
  - The transaction does not operate on *Q* until it has obtained a lock on a majority of the replicas of *Q*.
  - When writing the data item, transaction performs writes on *all* replicas.
- Benefit
  - Can be used even when some sites are unavailable
    - details on how handle writes in the presence of site failure later
- Drawback
  - Requires 2(*n*/2 + 1) messages for handling lock requests, and (*n*/2 + 1) messages for handling unlock requests.
  - Potential for deadlock even with single item - e.g., each of 3 transactions may have locks on 1/3rd of the replicas of a data.

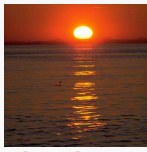


©Silberschatz, Korth and Sudarshan



Biased Protocol

- Local lock manager at each site as in majority protocol, however, requests for shared locks are handled differently than requests for exclusive locks.
- **Shared locks**. When a transaction needs to lock data item *Q*, it simply requests a lock on *Q* from the lock manager at one site containing a replica of *Q*.
- **Exclusive locks**. When transaction needs to lock data item *Q*, it requests a lock on *Q* from the lock manager at all sites containing a replica of *Q*.
- Advantage - imposes less overhead on **read** operations.
- Disadvantage - additional overhead on writes

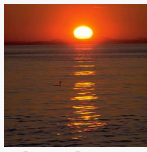


©Silberschatz, Korth and Sudarshan



Quorum Consensus Protocol

- A generalization of both majority and biased protocols
- Each site is assigned a weight.
  - Let S be the total of all site weights
- Choose two values read quorum Q_r and write quorum Q_w
  - Such that $Q_r + Q_w > S$ and $2 * Q_w > S$
  - Quorums can be chosen (and S computed) separately for each item
- Each read must lock enough replicas that the sum of the site weights is >= Q_r
- Each write must lock enough replicas that the sum of the site weights is >= Q_w
- For now we assume all replicas are written
  - Extensions to allow some sites to be unavailable described later

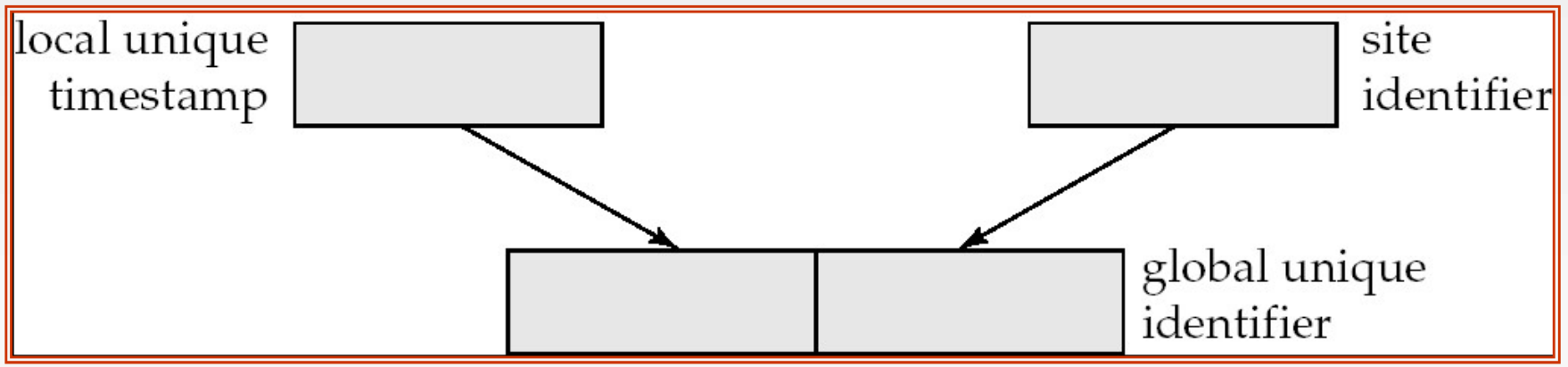


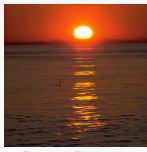
©Silberschatz, Korth and Sudarshan



Timestamping

- Timestamp based concurrency-control protocols can be used in distributed systems
- Each transaction must be given a unique timestamp
- Main problem: how to generate a timestamp in a distributed fashion
  - Each site generates a unique local timestamp using either a logical counter or the local clock.
  - Global unique timestamp is obtained by concatenating the unique local timestamp with the unique identifier.



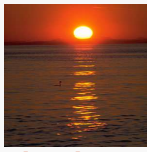


 ©Silberschatz, Korth and Sudarshan



Timestamping (Cont.)

- A site with a slow clock will assign smaller timestamps
  - Still logically correct: serializability not affected
  - But: "disadvantages" transactions
- To fix this problem
  - Define within each site S_i a **logical clock** (LC_i), which generates the unique local timestamp
  - Require that S_i advance its logical clock whenever a request is received from a transaction Ti with timestamp < *x,y*> and x is greater that the current value of LC_i.
  - In this case, site S_i advances its logical clock to the value $x + 1$.

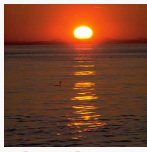


©Silberschatz, Korth and Sudarshan



Replication with Weak Consistency

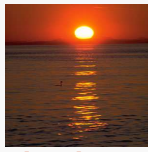
- Many commercial databases support replication of data with weak degrees of consistency (I.e., without a guarantee of serializabiliy)
- E.g.: **master-slave replication**: updates are performed at a single "master" site, and propagated to "slave" sites.
  - Propagation is not part of the update transaction: its is decoupled
    - May be immediately after transaction commits
    - May be periodic
  - Data may only be read at slave sites, not updated
    - No need to obtain locks at any remote site
  - Particularly useful for distributing information
    - E.g. from central office to branch-office
  - Also useful for running read-only queries offline from the main database





Replication with Weak Consistency (Cont.)

- Replicas should see a **transaction-consistent snapshot** of the database
  - That is, a state of the database reflecting all effects of all transactions up to some point in the serialization order, and no effects of any later transactions.
- E.g. Oracle provides a **create snapshot** statement to create a snapshot of a relation or a set of relations at a remote site
  - snapshot refresh either by recomputation or by incremental update
  - Automatic refresh (continuous or periodic) or manual refresh

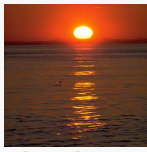


©Silberschatz, Korth and Sudarshan



Multimaster and Lazy Replication

- With multimaster replication (also called update-anywhere replication) updates are permitted at any replica, and are automatically propagated to all replicas
  - Basic model in distributed databases, where transactions are unaware of the details of replication, and database system propagates updates as part of the same transaction
    - Coupled with 2 phase commit
- Many systems support **lazy propagation** where updates are transmitted after transaction commits
  - Allows updates to occur even if some sites are disconnected from the network, but at the cost of consistency



©Silberschatz, Korth and Sudarshan



Deadlock Handling

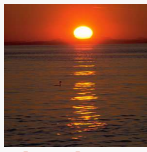
Consider the following two transactions and history, with item X and transaction T_1 at site 1, and item Y and transaction T_2 at site 2:

T_1: write (X)
write (Y)

T_2: write (Y)
write (X)

X-lock on X write (X)	
	X-lock on Y write (Y) wait for X-lock on X
Wait for X-lock on Y	

Result: deadlock which cannot be detected locally at either site

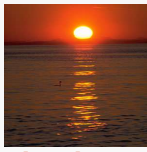


©Silberschatz, Korth and Sudarshan



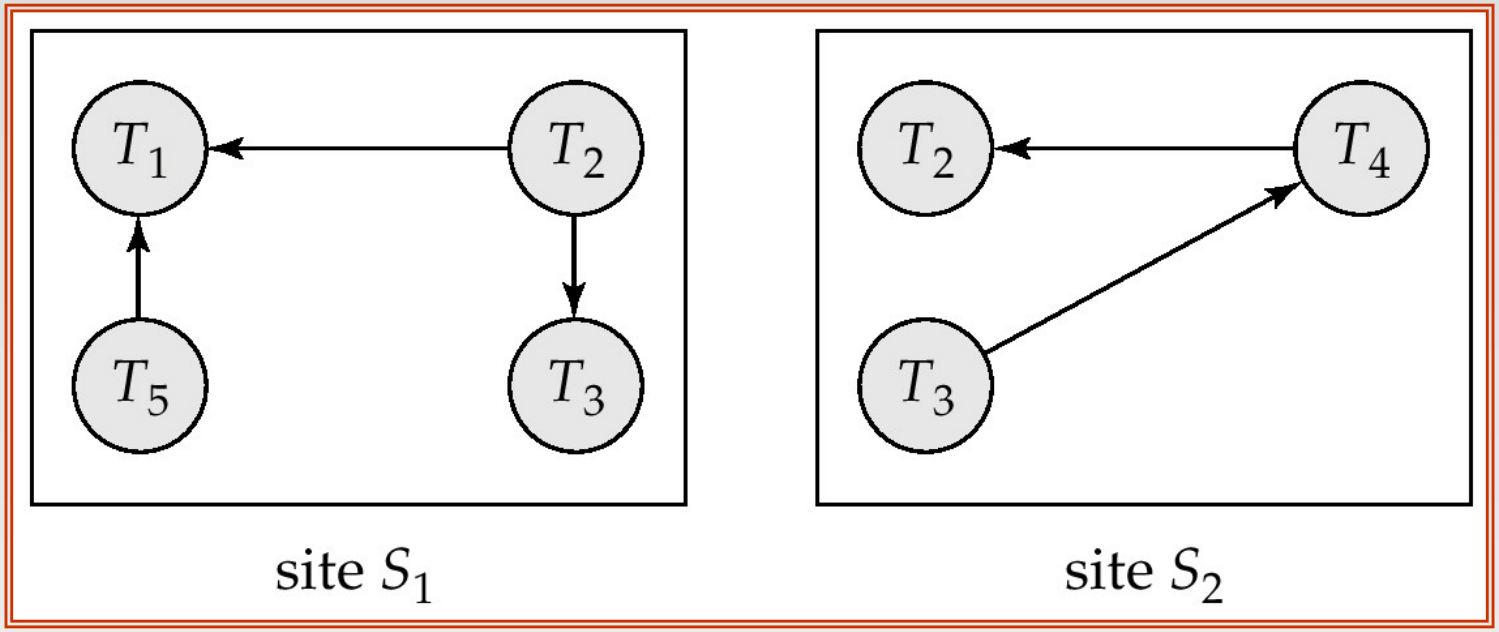
Centralized Approach

- A global wait-for graph is constructed and maintained in a *single* site; the deadlock-detection coordinator
  - *Real graph*: Real, but unknown, state of the system.
  - *Constructed graph*:Approximation generated by the controller during the execution of its algorithm .
- the global wait-for graph can be constructed when:
  - a new edge is inserted in or removed from one of the local wait-for graphs.
  - a number of changes have occurred in a local wait-for graph.
  - the coordinator needs to invoke cycle-detection.
- If the coordinator finds a cycle, it selects a victim and notifies all sites. The sites roll back the victim transaction.

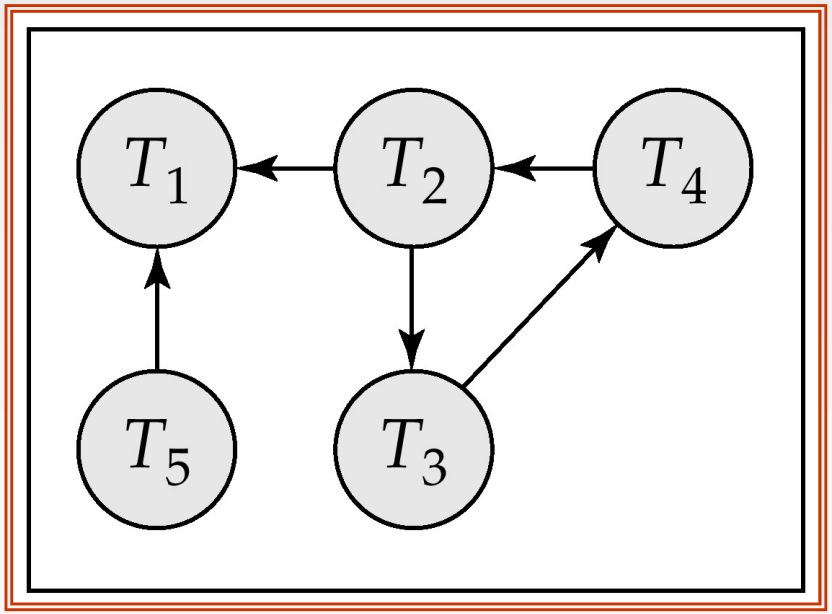


©Silberschatz, Korth and Sudarshan

Local and Global Wait-For Graphs



Local

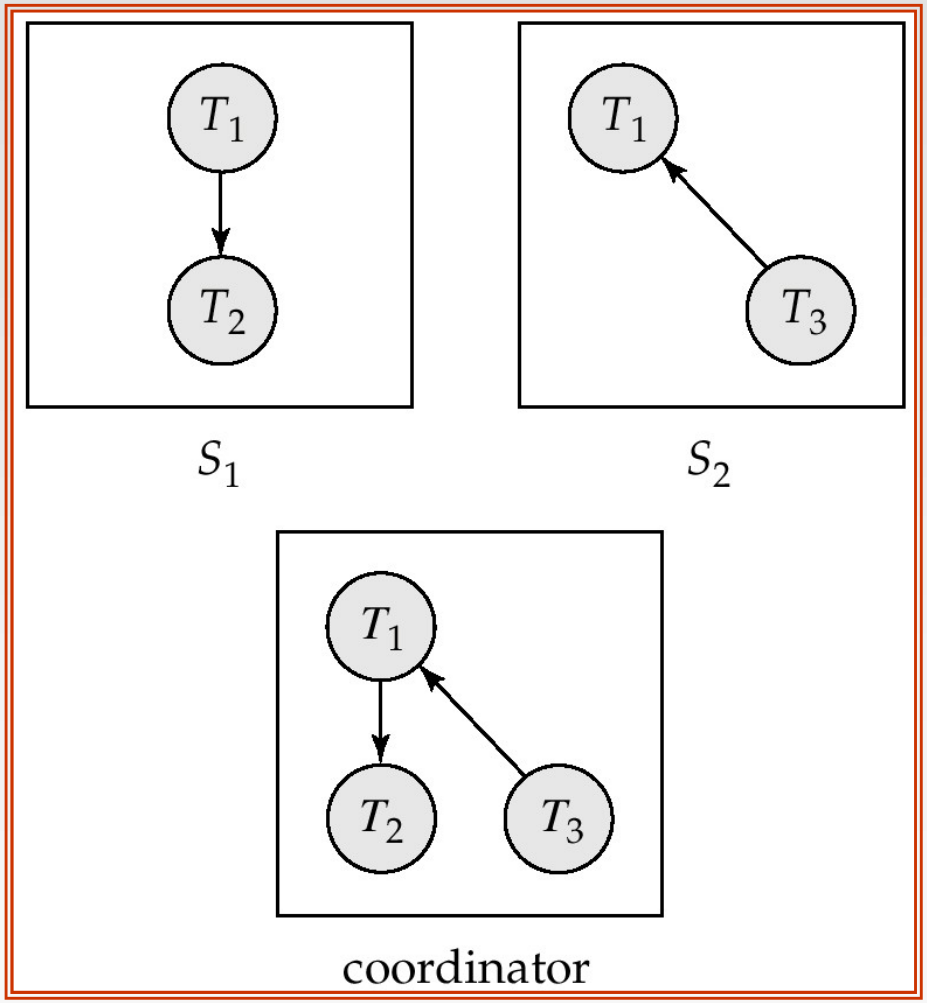


Global

©Silberschatz, Korth and Sudarshan

Example Wait-For Graph for False Cycles

Initial state:



©Silberschatz, Korth and Sudarshan

False Cycles (Cont.)

- Suppose that starting from the state shown in figure,
  1. T_2 releases resources at S_1
     - resulting in a message remove $T_1 \rightarrow T_2$ message from the Transaction Manager at site S_1 to the coordinator)
  2. And then T_2 requests a resource held by T_3 at site S_2
     - resulting in a message insert $T_2 \rightarrow T_3$ from S_2 to the coordinator
- Suppose further that the insert message reaches before the **delete** message
  - this can happen due to network delays
- The coordinator would then find a false cycle

$$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$$

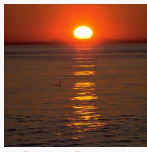
- The false cycle above never existed in reality.
- False cycles cannot occur if two-phase locking is used.

©Silberschatz, Korth and Sudarshan



Unnecessary Rollbacks

- Unnecessary rollbacks may result when deadlock has indeed occurred and a victim has been picked, and meanwhile one of the transactions was aborted for reasons unrelated to the deadlock.
- Unnecessary rollbacks can result from false cycles in the global wait-for graph; however, likelihood of false cycles is low.

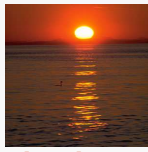


©Silberschatz, Korth and Sudarshan



Availability

- High availability: time for which system is not fully usable should be extremely low (e.g. 99.99% availability)
- Robustness: ability of system to function spite of failures of components
- Failures are more likely in large distributed systems
- To be robust, a distributed system must
  - Detect failures
  - Reconfigure the system so computation may continue
  - Recovery/reintegration when a site or link is repaired
- Failure detection: distinguishing link failure from site failure is hard
  - (partial) solution: have multiple links, multiple link failure is likely a site failure

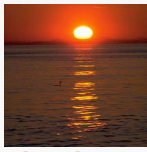


©Silberschatz, Korth and Sudarshan



Reconfiguration

- Reconfiguration:
  - Abort all transactions that were active at a failed site
    - Making them wait could interfere with other transactions since they may hold locks on other sites
    - However, in case only some replicas of a data item failed, it may be possible to continue transactions that had accessed data at a failed site (more on this later)
  - If replicated data items were at failed site, update system catalog to remove them from the list of replicas.
    - This should be reversed when failed site recovers, but additional care needs to be taken to bring values up to date
  - If a failed site was a central server for some subsystem, an **election** must be held to determine the new server
    - E.g. name server, concurrency coordinator, global deadlock detector

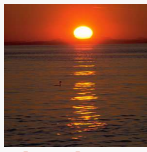


©Silberschatz, Korth and Sudarshan



Reconfiguration (Cont.)

- Since network partition may not be distinguishable from site failure, the following situations must be avoided
  - Two ore more central servers elected in distinct partitions
  - More than one partition updates a replicated data item
- Updates must be able to continue even if some sites are down
- Solution: majority based approach
  - Alternative of “read one write all available” is tantalizing but causes problems

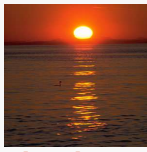


©Silberschatz, Korth and Sudarshan



Majority-Based Approach

- The majority protocol for distributed concurrency control can be modified to work even if some sites are unavailable
  - Each replica of each item has a **version number** which is updated when the replica is updated, as outlined below
  - A lock request is sent to at least ½ the sites at which item replicas are stored and operation continues only when a lock is obtained on a majority of the sites
  - Read operations look at all replicas locked, and read the value from the replica with largest version number
    - May write this value and version number back to replicas with lower version numbers (no need to obtain locks on all replicas for this task)

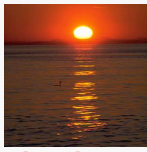


©Silberschatz, Korth and Sudarshan



Majority-Based Approach

- Majority protocol (Cont.)
  - Write operations
    - find highest version number like reads, and set new version number to old highest version + 1
    - Writes are then performed on all locked replicas and version number on these replicas is set to new version number
  - Failures (network and site) cause no problems as long as
    - Sites at commit contain a majority of replicas of any updated data items
    - During reads a majority of replicas are available to find version numbers
    - Subject to above, 2 phase commit can be used to update replicas
  - Note: reads are guaranteed to see latest version of data item
  - Reintegration is trivial: nothing needs to be done
- Quorum consensus algorithm can be similarly extended

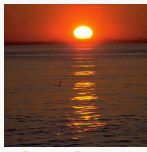


©Silberschatz, Korth and Sudarshan



Read One Write All (Available)

- Biased protocol is a special case of quorum consensus
  - Allows reads to read any one replica but updates require all replicas to be available at commit time (called **read one write all**)
- Read one write all available (ignoring failed sites) is attractive, but incorrect
  - If failed link may come back up, without a disconnected site ever being aware that it was disconnected
  - The site then has old values, and a read from that site would return an incorrect value
  - If site was aware of failure reintegration could have been performed, but no way to guarantee this
  - With network partitioning, sites in each partition may update same item concurrently
    - believing sites in other partitions have all failed

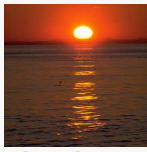


©Silberschatz, Korth and Sudarshan



Site Reintegration

- When failed site recovers, it must catch up with all updates that it missed while it was down
  - Problem: updates may be happening to items whose replica is stored at the site while the site is recovering
  - Solution 1: halt all updates on system while reintegrating a site
    - Unacceptable disruption
  - Solution 2: lock all replicas of all data items at the site, update to latest version, then release locks
    - Other solutions with better concurrency also available

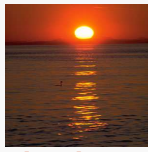


©Silberschatz, Korth and Sudarshan



Comparison with Remote Backup

- Remote backup (hot spare) systems (Section 17.10) are also designed to provide high availability
- Remote backup systems are simpler and have lower overhead
  - All actions performed at a single site, and only log records shipped
  - No need for distributed concurrency control, or 2 phase commit
- Using distributed databases with replicas of data items can provide higher availability by having multiple (> 2) replicas and using the majority protocol
  - Also avoid failure detection and switchover time associated with remote backup systems

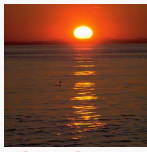


©Silberschatz, Korth and Sudarshan



Coordinator Selection

- Backup coordinators
  - site which maintains enough information locally to assume the role of coordinator if the actual coordinator fails
  - executes the same algorithms and maintains the same internal state information as the actual coordinator fails executes state information as the actual coordinator
  - allows fast recovery from coordinator failure but involves overhead during normal processing.
- Election algorithms
  - used to elect a new coordinator in case of failures
  - Example: Bully Algorithm - applicable to systems where every site can send a message to every other site.

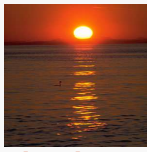


©Silberschatz, Korth and Sudarshan



Bully Algorithm

- If site S_i sends a request that is not answered by the coordinator within a time interval T, assume that the coordinator has failed S_i tries to elect itself as the new coordinator.
- S_i sends an election message to every site with a higher identification number, S_i then waits for any of these processes to answer within T.
- If no response within T, assume that all sites with number greater than i have failed, S_i elects itself the new coordinator.
- If answer is received S_i begins time interval T' , waiting to receive a message that a site with a higher identification number has been elected.

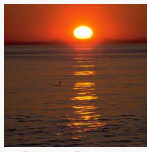


©Silberschatz, Korth and Sudarshan



Bully Algorithm (Cont.)

- If no message is sent within T' , assume the site with a higher number has failed; S_i restarts the algorithm.
- After a failed site recovers, it immediately begins execution of the same algorithm.
- If there are no active sites with higher numbers, the recovered site forces all processes with lower numbers to let it become the coordinator site, even if there is a currently active coordinator with a lower number.

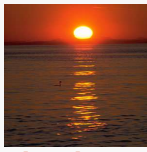


©Silberschatz, Korth and Sudarshan



Distributed Query Processing

- For centralized systems, the primary criterion for measuring the cost of a particular strategy is the number of disk accesses.
- In a distributed system, other issues must be taken into account:
  - The cost of a data transmission over the network.
  - The potential gain in performance from having several sites process parts of the query in parallel.



©Silberschatz, Korth and Sudarshan



Query Transformation

- Translating algebraic queries on fragments.
  - It must be possible to construct relation *r* from its fragments
  - Replace relation *r* by the expression to construct relation *r* from its fragments
- Consider the horizontal fragmentation of the *account* relation into

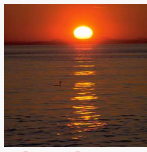
  $account_1 = \sigma_{branch_name = \text{“Hillside”}} (account)$

  $account_2 = \sigma_{branch_name = \text{“Valleyview”}} (account)$
- The query $\sigma_{branch_name = \text{“Hillside”}} (account)$ becomes

  $\sigma_{branch_name = \text{“Hillside”}} (account_1 \cup account_2)$

  which is optimized into

  $\sigma_{branch_name = \text{“Hillside”}} (account_1) \cup \sigma_{branch_name = \text{“Hillside”}} (account_2)$



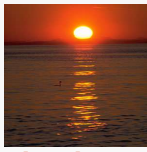
©Silberschatz, Korth and Sudarshan



Example Query (Cont.)

- Since $account_1$ has only tuples pertaining to the Hillside branch, we can eliminate the selection operation.
- Apply the definition of $account_2$ to obtain

  $\sigma_{branch_name = \text{“Hillside”}} (\sigma_{branch_name = \text{“Valleyview”}} (account))$
- This expression is the empty set regardless of the contents of the *account* relation.
- Final strategy is for the Hillside site to return $account_1$ as the result of the query.



©Silberschatz, Korth and Sudarshan

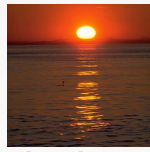


Simple Join Processing

- Consider the following relational algebra expression in which the three relations are neither replicated nor fragmented

  $account \bowtie depositor \bowtie branch$

- *account* is stored at site S_1
- *depositor* at S_2
- *branch* at S_3
- For a query issued at site S_I, the system needs to produce the result at site S_I

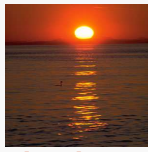


©Silberschatz, Korth and Sudarshan



Possible Query Processing Strategies

- Ship copies of all three relations to site S_I and choose a strategy for processing the entire locally at site S_I.
- Ship a copy of the account relation to site S_2 and compute $temp_1$ = *account* ⋈ *depositor at* S_2. Ship $temp_1$ from S_2 to S_3, and compute $temp_2$ = $temp_1$ ⋈ *branch* at S_3. Ship the result $temp_2$ to S_I.
- Devise similar strategies, exchanging the roles S_1, S_2, S_3
- Must consider following factors:
  - amount of data being shipped
  - cost of transmitting a data block between sites
  - relative processing speed at each site



©Silberschatz, Korth and Sudarshan



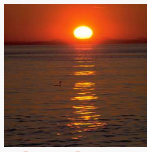
Semijoin Strategy

- Let r_1 be a relation with schema R_1 stores at site S_1

  Let r_2 be a relation with schema R_2 stores at site S_2
- Evaluate the expression $r_1 \bowtie r_2$ and obtain the result at S_1.

1. Compute $temp_1 \leftarrow \Pi_{R1 \cap R2}$ (r1) at $S1$.

- 2. Ship $temp_1$ from S_1 to S_2.
- 3. Compute $temp_2 \leftarrow r_2 \bowtie$ temp1 at S_2
- 4. Ship $temp_2$ from S_2 to S_1.
- 5. Compute $r_1 \bowtie temp_2$ at S_1. This is the same as $r_1 \bowtie r_2$.



©Silberschatz, Korth and Sudarshan

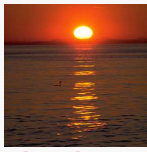


Formal Definition

- The **semijoin** of r_1 with r_2, is denoted by:

$$r_1 \ltimes r_2$$

- it is defined by:
- $\Pi_{R1} (r_1 \bowtie r_2)$
- Thus, $r_1 \ltimes r_2$ selects those tuples of r_1 that contributed to $r_1 \bowtie r_2$.
- In step 3 above, $temp_2 = r_2 \ltimes r_1$.
- For joins of several relations, the above strategy can be extended to a series of semijoin steps.

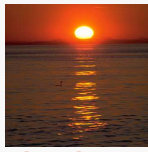


©Silberschatz, Korth and Sudarshan



Join Strategies that Exploit Parallelism

- Consider $r_1 \bowtie r_2 \bowtie r_3 \bowtie r_4$ where relation r_i is stored at site S_i. The result must be presented at site S_1.
- r_1 is shipped to S_2 and $r_1 \bowtie r_2$ is computed at S_2: simultaneously r_3 is shipped to S_4 and $r_3 \bowtie r_4$ is computed at S_4
- S_2 ships tuples of $(r_1 \bowtie r_2)$ to S_1 as they produced;
  S_4 ships tuples of $(r_3 \bowtie r_4)$ to S_1
- Once tuples of $(r_1 \bowtie r_2)$ and $(r_3 \bowtie r_4)$ arrive at S_1 $(r_1 \bowtie r_2) \bowtie (r_3 \bowtie r_4)$ is computed in parallel with the computation of $(r_1 \bowtie r_2)$ at S_2 and the computation of $(r_3 \bowtie r_4)$ at S_4.

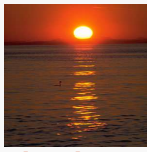


©Silberschatz, Korth and Sudarshan



Heterogeneous Distributed Databases

- Many database applications require data from a variety of preexisting databases located in a heterogeneous collection of hardware and software platforms
- Data models may differ (hierarchical, relational , etc.)
- Transaction commit protocols may be incompatible
- Concurrency control may be based on different techniques (locking, timestamping, etc.)
- System-level details almost certainly are totally incompatible.
- A **multidatabase system** is a software layer on top of existing database systems, which is designed to manipulate information in heterogeneous databases
  - Creates an illusion of logical database integration without any physical database integration

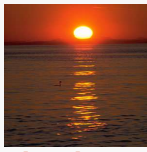


©Silberschatz, Korth and Sudarshan



Advantages

- Preservation of investment in existing
  - hardware
  - system software
  - Applications
- Local autonomy and administrative control
- Allows use of special-purpose DBMSs
- Step towards a unified homogeneous DBMS
  - Full integration into a homogeneous DBMS faces
    - Technical difficulties and cost of conversion
    - Organizational/political difficulties
      - Organizations do not want to give up control on their data
      - Local databases wish to retain a great deal of **autonomy**

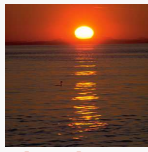


©Silberschatz, Korth and Sudarshan



Unified View of Data

- Agreement on a common data model
  - Typically the relational model
- Agreement on a common conceptual schema
  - Different names for same relation/attribute
  - Same relation/attribute name means different things
- Agreement on a single representation of shared data
  - E.g. data types, precision,
  - Character sets
    - ASCII vs EBCDIC
    - Sort order variations
- Agreement on units of measure
- Variations in names
  - E.g. Köln vs Cologne, Mumbai vs Bombay

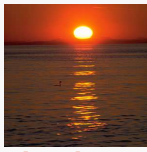


©Silberschatz, Korth and Sudarshan



Query Processing

- Several issues in query processing in a heterogeneous database
- Schema translation
  - Write a **wrapper** for each data source to translate data to a global schema
  - Wrappers must also translate updates on global schema to updates on local schema
- Limited query capabilities
  - Some data sources allow only restricted forms of selections
    - E.g. web forms, flat file data sources
  - Queries have to be broken up and processed partly at the source and partly at a different site
- Removal of duplicate information when sites have overlapping information
  - Decide which sites to execute query
- Global query optimization

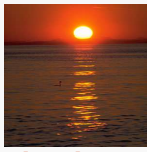


©Silberschatz, Korth and Sudarshan



Mediator Systems

- **Mediator** systems are systems that integrate multiple heterogeneous data sources by providing an integrated global view, and providing query facilities on global view
  - Unlike full fledged multidatabase systems, mediators generally do not bother about transaction processing
  - But the terms mediator and multidatabase are sometimes used interchangeably
  - The term **virtual database** is also used to refer to mediator/multidatabase systems

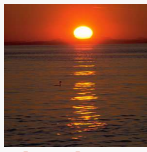


©Silberschatz, Korth and Sudarshan



Directory Systems

- Typical kinds of directory information
  - Employee information such as name, id, email, phone, office addr, ..
  - Even personal information to be accessed from multiple places
    - e.g. Web browser bookmarks
- White pages
  - Entries organized by name or identifier
    - Meant for forward lookup to find more about an entry
- Yellow pages
  - Entries organized by properties
  - For reverse lookup to find entries matching specific requirements
- When directories are to be accessed across an organization
  - Alternative 1: Web interface. Not great for programs
  - Alternative 2: Specialized **directory access protocols**
    - Coupled with specialized user interfaces

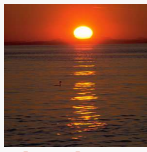


©Silberschatz, Korth and Sudarshan



Directory Access Protocols

- Most commonly used directory access protocol:
  - LDAP (Lightweight Directory Access Protocol)
  - Simplified from earlier X.500 protocol
- Question: Why not use database protocols like ODBC/JDBC?
- Answer:
  - Simplified protocols for a limited type of data access, evolved parallel to ODBC/JDBC
  - Provide a nice hierarchical naming mechanism similar to file system directories
    - Data can be partitioned amongst multiple servers for different parts of the hierarchy, yet give a single view to user
      - E.g. different servers for Bell Labs Murray Hill and Bell Labs Bangalore
  - Directories may use databases as storage mechanism

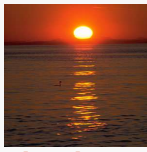


©Silberschatz, Korth and Sudarshan



LDAP: Lightweight Directory Access Protocol

- LDAP Data Model
- Data Manipulation
- Distributed Directory Trees

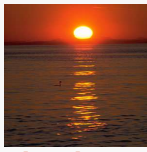


©Silberschatz, Korth and Sudarshan



LDAP Data Model

- LDAP directories store **entries**
  - Entries are similar to objects
- Each entry must have unique **distinguished name (DN)**
- DN made up of a sequence of **relative distinguished names (RDNs)**
- E.g. of a DN
  - cn=Silberschatz, ou-Bell Labs, o=Lucent, c=USA
  - Standard RDNs (can be specified as part of schema)
    - cn: common name ou: organizational unit
    - o: organization c: country
  - Similar to paths in a file system but written in reverse direction

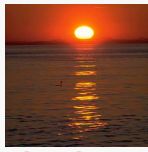


©Silberschatz, Korth and Sudarshan



LDAP Data Model (Cont.)

- Entries can have attributes
  - Attributes are multi-valued by default
  - LDAP has several built-in types
    - Binary, string, time types
    - Tel: telephone number PostalAddress: postal address
- LDAP allows definition of **object classes**
  - Object classes specify attribute names and types
  - Can use inheritance to define object classes
  - Entry can be specified to be of one or more object classes
    - No need to have single most-specific type

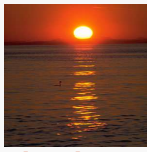


©Silberschatz, Korth and Sudarshan



LDAP Data Model (cont.)

- Entries organized into a **directory information tree** according to their DNs
  - Leaf level usually represent specific objects
  - Internal node entries represent objects such as organizational units, organizations or countries
  - Children of a node inherit the DN of the parent, and add on RDNs
    - E.g. internal node with DN c=USA
      - Children nodes have DN starting with c=USA and further RDNs such as o or ou
    - DN of an entry can be generated by traversing path from root
  - Leaf level can be an alias pointing to another entry
    - Entries can thus have more than one DN
      - E.g. person in more than one organizational unit

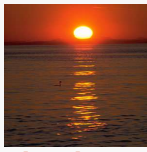


©Silberschatz, Korth and Sudarshan



LDAP Data Manipulation

- Unlike SQL, LDAP does not define DDL or DML
- Instead, it defines a network protocol for DDL and DML
  - Users use an API or vendor specific front ends
  - LDAP also defines a file format
    - LDAP Data Interchange Format (LDIF)
- Querying mechanism is very simple: only selection & projection

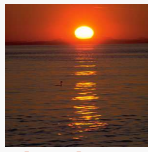


©Silberschatz, Korth and Sudarshan



LDAP Queries

- LDAP query must specify
  - Base: a node in the DIT from where search is to start
  - A search condition
    - Boolean combination of conditions on attributes of entries
      - Equality, wild-cards and approximate equality supported
  - A scope
    - Just the base, the base and its children, or the entire subtree from the base
  - Attributes to be returned
  - Limits on number of results and on resource consumption
  - May also specify whether to automatically dereference aliases
- LDAP URLs are one way of specifying query
- LDAP API is another alternative

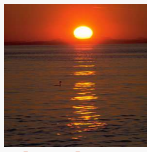


©Silberschatz, Korth and Sudarshan



LDAP URLs

- First part of URL specifis server and DN of base
  - ldap:://aura.research.bell-labs.com/o=Lucent,c=USA
- Optional further parts separated by ? symbol
  - ldap:://aura.research.bell-labs.com/o=Lucent,c=USA??sub?cn=Korth
  - Optional parts specify
    1. attributes to return (empty means all)
    2. Scope (sub indicates entire subtree)
    3. Search condition (cn=Korth)

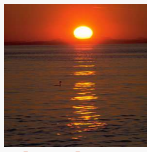


©Silberschatz, Korth and Sudarshan



C Code using LDAP API

```
#include <stdio.h>
#include <ldap.h>
main( ) {
        LDAP *ld;
        LDAPMessage *res, *entry;
        char *dn, *attr, *attrList [ ] = {"telephoneNumber", NULL};
        BerElement *ptr;
        int vals, i;
            // Open a connection to server
        ld = ldap_open("aura.research.bell-labs.com", LDAP_PORT);
        ldap_simple_bind(ld, "avi", "avi-passwd");
         ... actual query (next slide) ...
      ldap_unbind(ld);
}
```

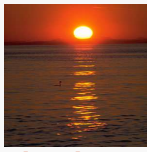


©Silberschatz, Korth and Sudarshan



C Code using LDAP API (Cont.)

```
ldap_search_s(ld, "o=Lucent, c=USA", LDAP_SCOPE_SUBTREE,
                    "cn=Korth", attrList, /* attrsonly*/ 0, &res);
          /*attrsonly = 1 => return only schema not actual results*/
printf("found%d entries", ldap_count_entries(ld, res));
for (entry=ldap_first_entry(ld, res); entry != NULL;
                    entry=ldap_next_entry(id, entry)) {
       dn = ldap_get_dn(ld, entry);
       printf("dn: %s", dn);   /* dn: DN of matching entry */
       ldap_memfree(dn);
       for(attr = ldap_first_attribute(ld, entry, &ptr); attr != NULL;
            attr = ldap_next_attribute(ld, entry, ptr))
      {                                  // for each attribute
          printf("%s:", attr);              // print name of attribute
          vals = ldap_get_values(ld, entry, attr);
          for (i = 0; vals[i] != NULL; i ++)
                    printf("%s", vals[i]);  // since attrs can be multivalued
          ldap_value_free(vals);
      }
}
ldap_msgfree(res);
```

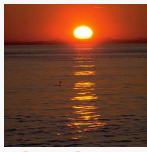


©Silberschatz, Korth and Sudarshan



LDAP API (Cont.)

- LDAP API also has functions to create, update and delete entries
- Each function call behaves as a separate transaction
  - LDAP does not support atomicity of updates

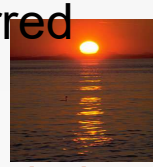


©Silberschatz, Korth and Sudarshan



Distributed Directory Trees

- Organizational information may be split into multiple directory information trees
  - Suffix of a DIT gives RDN to be tagged onto to all entries to get an overall DN
    - E.g. two DITs, one with suffix o=Lucent, c=USA
      and another with suffix o=Lucent, c=India
  - Organizations often split up DITs based on geographical location or by organizational structure
  - Many LDAP implementations support replication (master-slave or multi-master replication) of DITs (not part of LDAP 3 standard)
- A node in a DIT may be a **referral** to a node in another DIT
  - E.g. Ou= Bell Labs may have a separate DIT, and DIT for o=Lucent may have a leaf with ou=Bell Labs containing a referral to the Bell Labs DIT
  - Referalls are the key to integrating a distributed collection of directories
  - When a server gets a query reaching a referral node, it may either
    - Forward query to referred DIT and return answer to client, or
    - Give referral back to client, which transparently sends query to referred DIT (without user intervention)

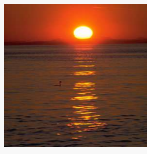


©Silberschatz, Korth and Sudarshan



End of Chapter

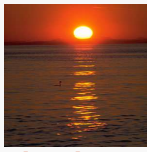
©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use





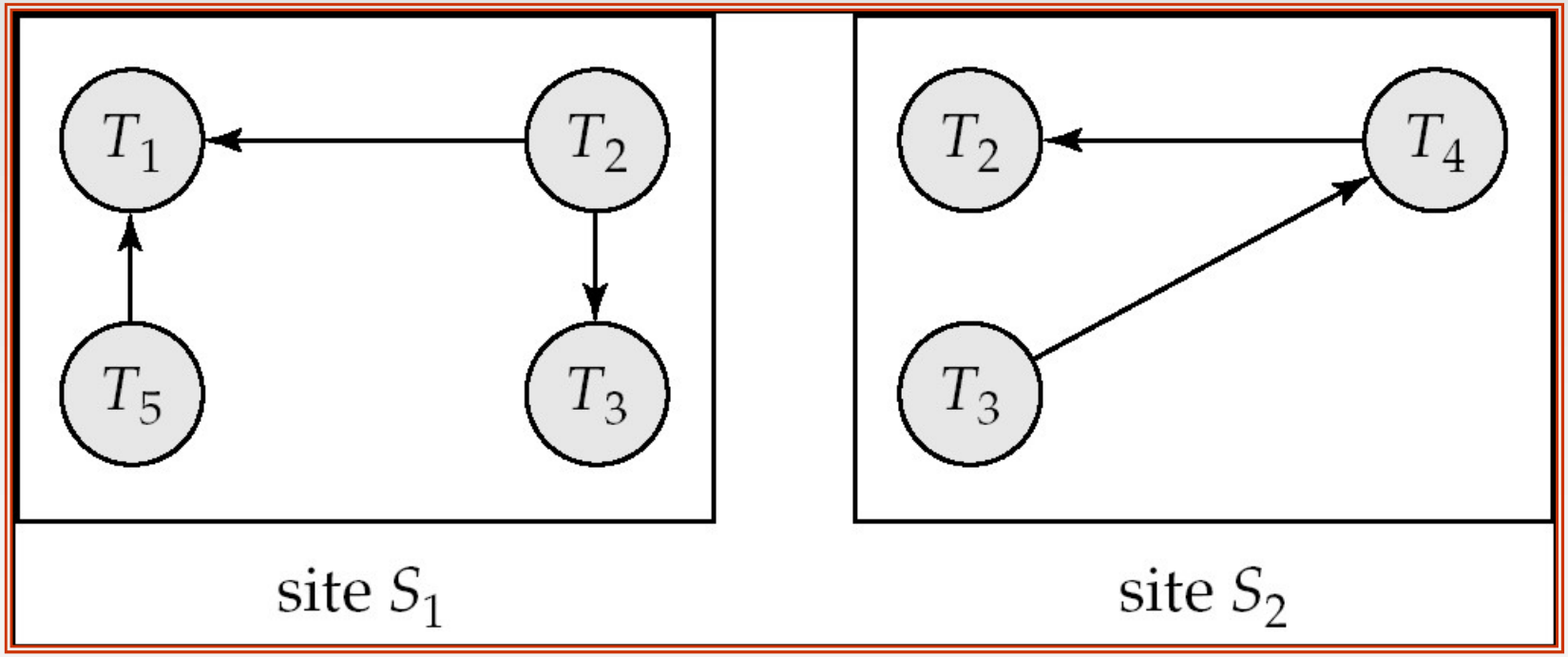
Three Phase Commit (3PC)

- Assumptions:
  - No network partitioning
  - At any point, at least one site must be up.
  - At most K sites (participants as well as coordinator) can fail
- Phase 1: Obtaining Preliminary Decision: Identical to 2PC Phase 1.
  - Every site is ready to commit if instructed to do so
- Phase 2 of 2PC is split into 2 phases, Phase 2 and Phase 3 of 3PC
  - In phase 2 coordinator makes a decision as in 2PC (called the pre-commit decision) and records it in multiple (at least K) sites
  - In phase 3, coordinator sends commit/abort message to all participating sites,
- Under 3PC, knowledge of pre-commit decision can be used to commit despite coordinator failure
  - Avoids blocking problem as long as < K sites fail
- Drawbacks:
  - higher overheads
  - assumptions may not be satisfied in practice



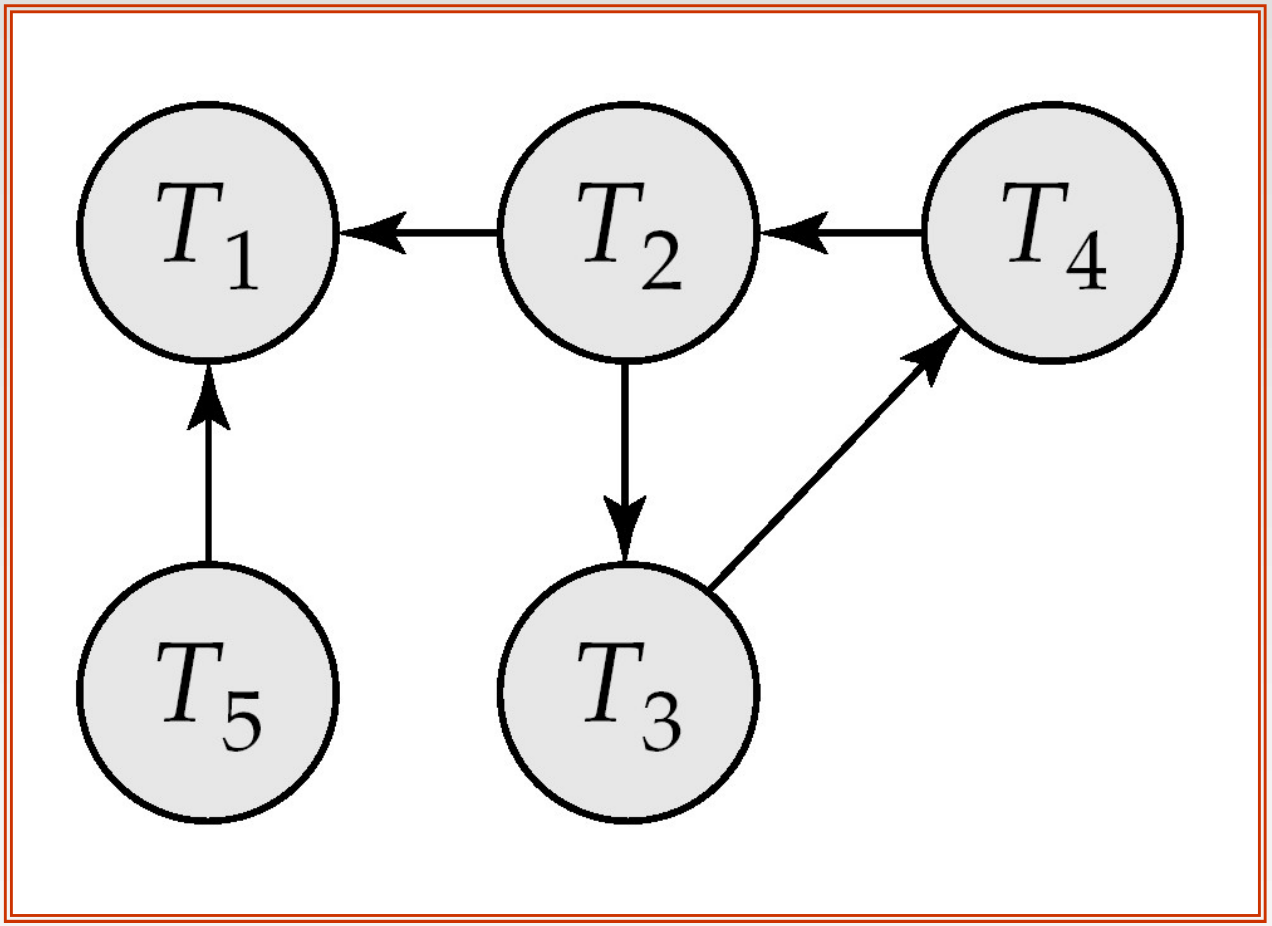
©Silberschatz, Korth and Sudarshan

Figure 22.3



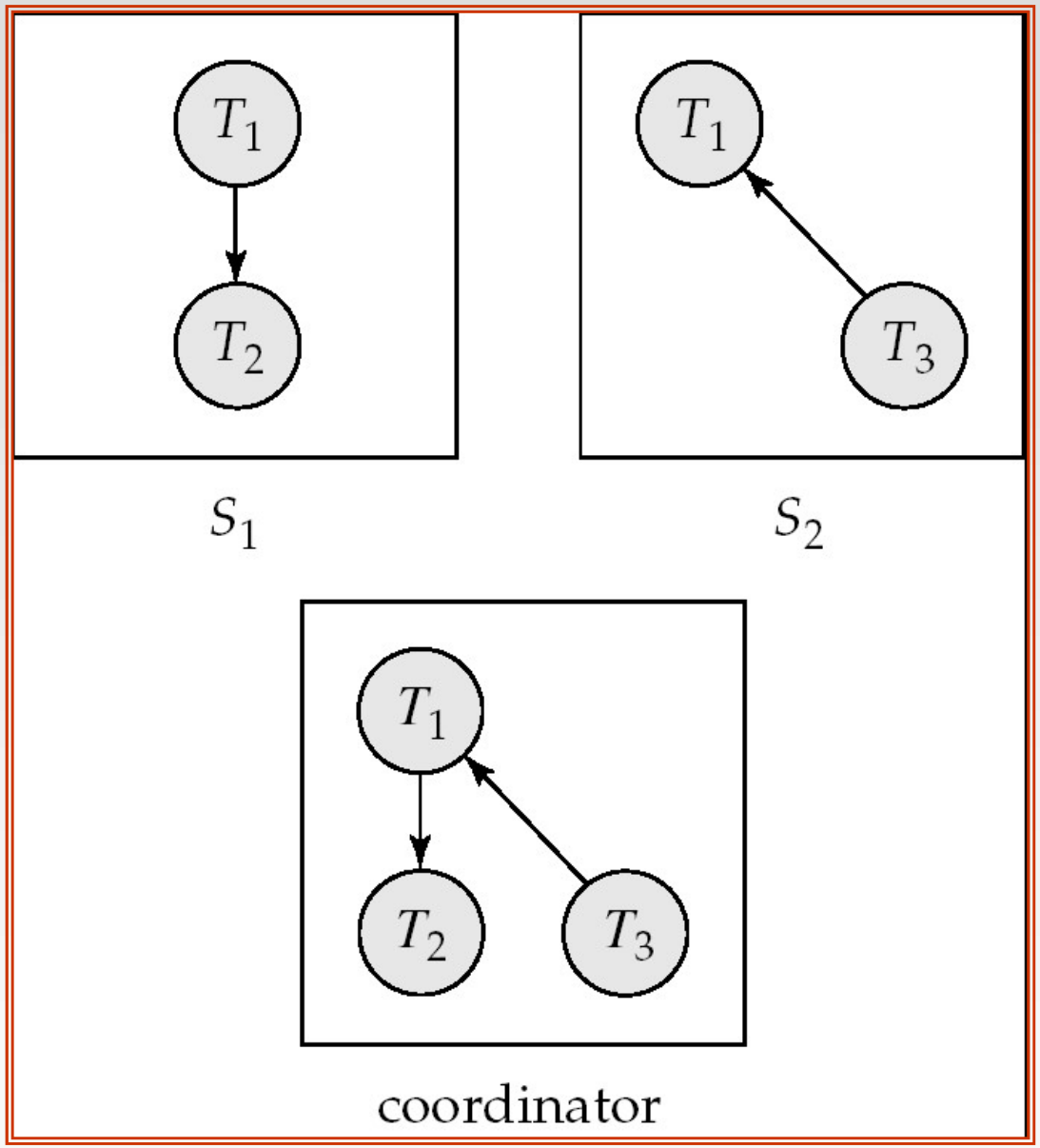
©Silberschatz, Korth and Sudarshan

Figure 22.4



 ©Silberschatz, Korth and Sudarshan

Figure 22.5



©Silberschatz, Korth and Sudarshan



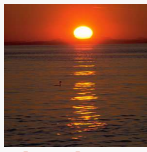
Figure 22.7

A	B	C
1	2	3
4	5	6
1	2	4
5	3	2
8	9	7

r

C	D	E
3	4	5
3	6	8
2	3	2
1	4	1
1	2	3

s



©Silberschatz, Korth and Sudarshan